

End-to-End Active Directory Cyberattack Detection, Containment, Eradication & Recovery Framework

Final Implementation Report

Participant: Chiamaka Odumegwu | Cybersecurity Engineer

Project Planning

1. **Defined project scope:** an end-to-end AD cyberattack detection, containment, eradication, and recovery framework, covering Password Spraying, Kerberoasting and the NIST SP 800-61 Incident Response lifecycle (Detection & Analysis → Containment → Eradication → Recovery → Lessons Learned).
2. **Identified required licenses and resources:** Azure subscription with Defender for Servers/Defender for Cloud enabled, Microsoft Defender for Identity and Microsoft Sentinel licensing, a Log Analytics workspace, and the three lab VMs (DC01 – Windows Server 2022, CLIENT01 – Windows 10/11, KALI01 – Kali Linux).
3. **Designed the solution architecture:** a single VNet (vnet-ad-cyberdefense, 10.10.0.0/16) hosting DC01, CLIENT01, and KALI01 on Servers-subnet, secured by an NSG scoped to the admin's public IP, with MDI/MDE feeding a dedicated Sentinel workspace and Logic Apps handling automated response .

Azure Infrastructure Deployment

1. Created a dedicated Azure Resource Group (AD_Cyberattack_Detection_Group) for the project.
2. Created a Virtual Network (vnet-ad-cyberdefense) with address space 10.10.0.0/16.
3. Created a subnet: Servers-subnet, 10.10.1.0/24.
4. Created a Network Security Group: NSG-AD-CYBERDEFENSE.
5. Configured inbound access rules: RDP (TCP 3389) and SSH (TCP 22).
6. Restricted administrative access to the authorized public IP address, to reduce the attack surface, prevent unauthorized access attempts, follow least-privilege principles, and demonstrate secure Azure networking practices.
7. Associated the Network Security Group with Servers-subnet.
8. Validated the subnet and NSG association.

Outcome: Azure network infrastructure was successfully deployed and secured. The environment is now ready for virtual machine deployment and Active Directory configuration. This foundation will support the deployment of DC01, CLIENT01, and KALI01 as defined in the project scope.



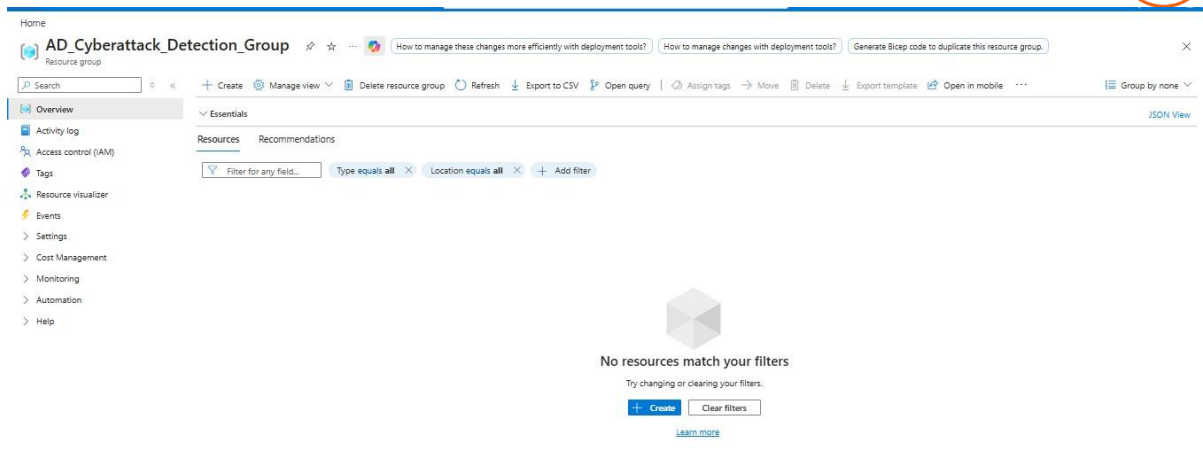


Figure 2.1 — The AD_Cyberattack_Detection_Group resource group created in Azure, shown empty before any resources have been provisioned into it.

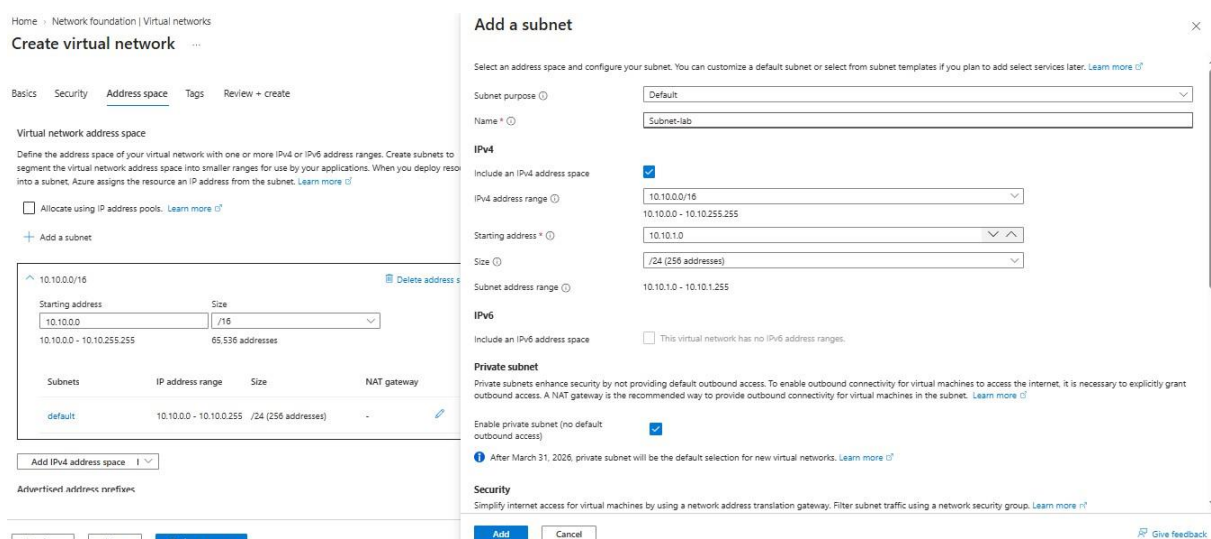


Figure 2.2 — Adding the Servers-subnet (10.10.1.0/24) inside the vnet-ad-cyberdefense address space (10.10.0.0/16) during virtual network creation.

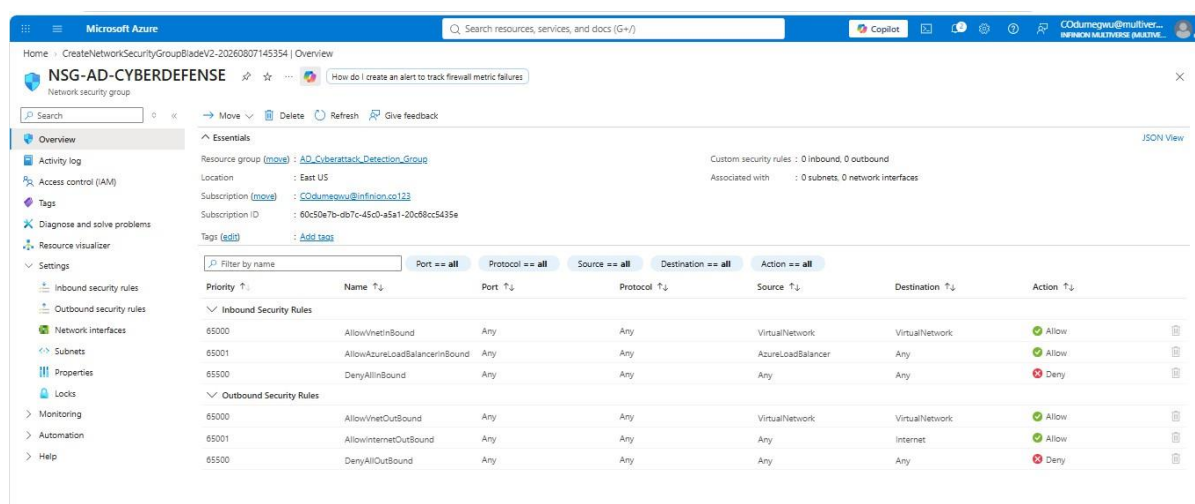


Figure 2.3 — The newly created NSG-AD-CYBERDEFENSE network security group, showing only the Azure default inbound/outbound rules before any custom rules were added.

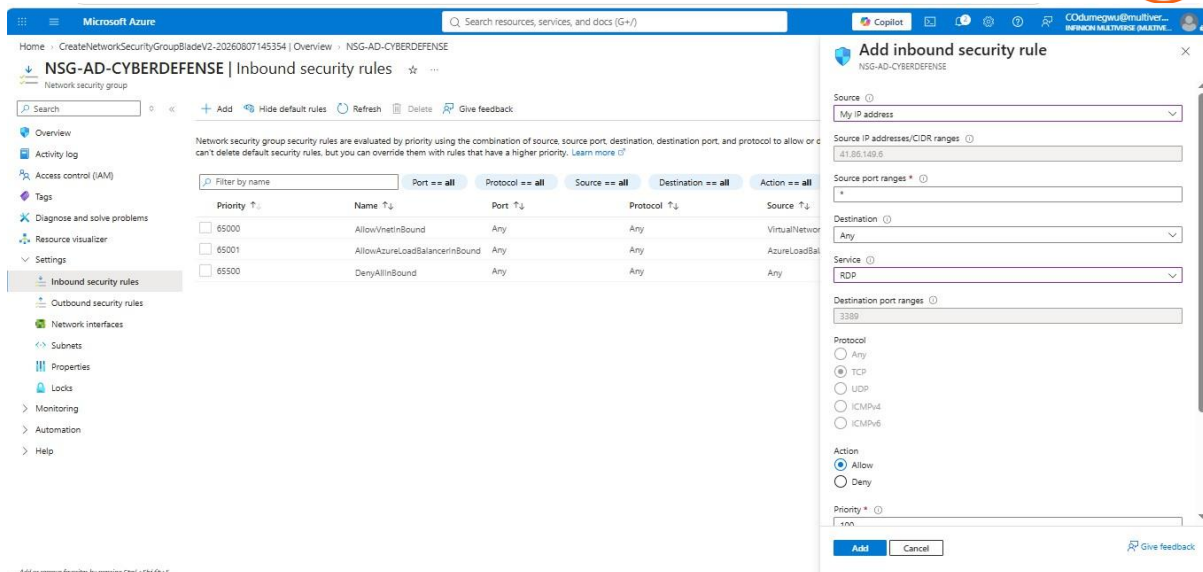


Figure 2.4 — Adding the inbound RDP (TCP 3389) rule, scoped to the source “My IP address” so only the administrator's public IP can connect.

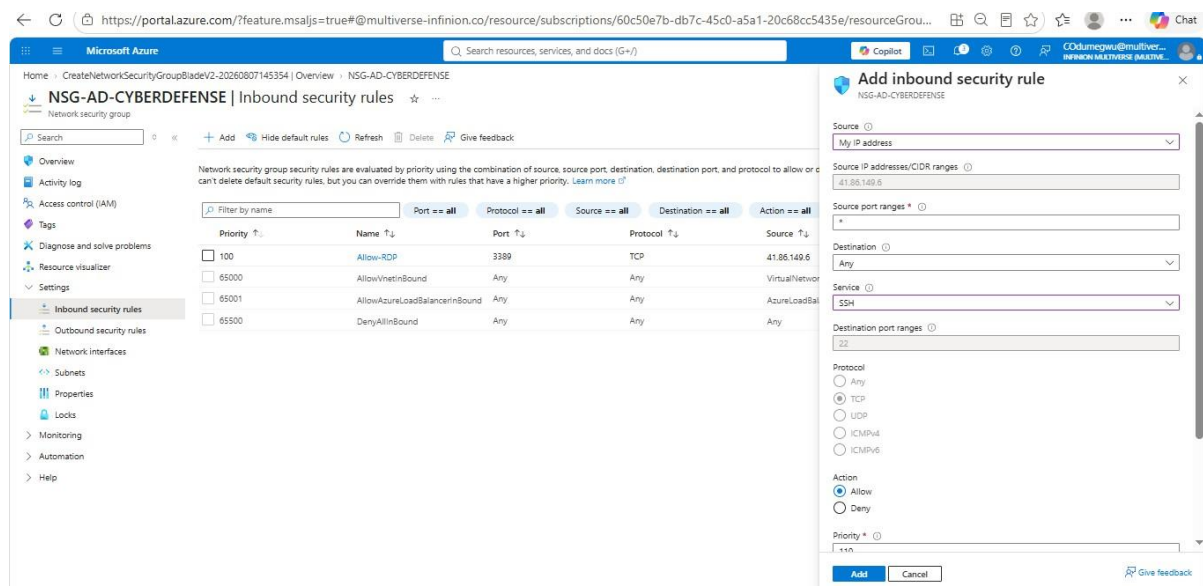


Figure 2.5 — Adding the inbound SSH (TCP 22) rule with the same source-IP restriction, created immediately after the RDP rule (visible above it at priority 100).



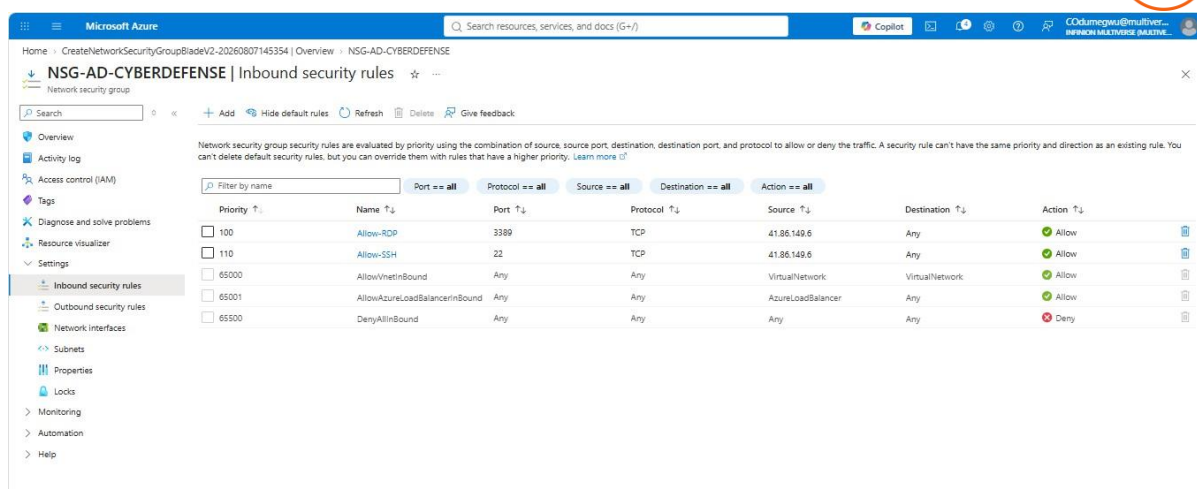


Figure 2.6 — Final inbound rule set on NSG-AD-CYBERDEFENSE: Allow-RDP (priority 100) and Allow-SSH (priority 110), both restricted to the admin's public IP (41.86.149.6), sitting above the default Allow/Deny rules.

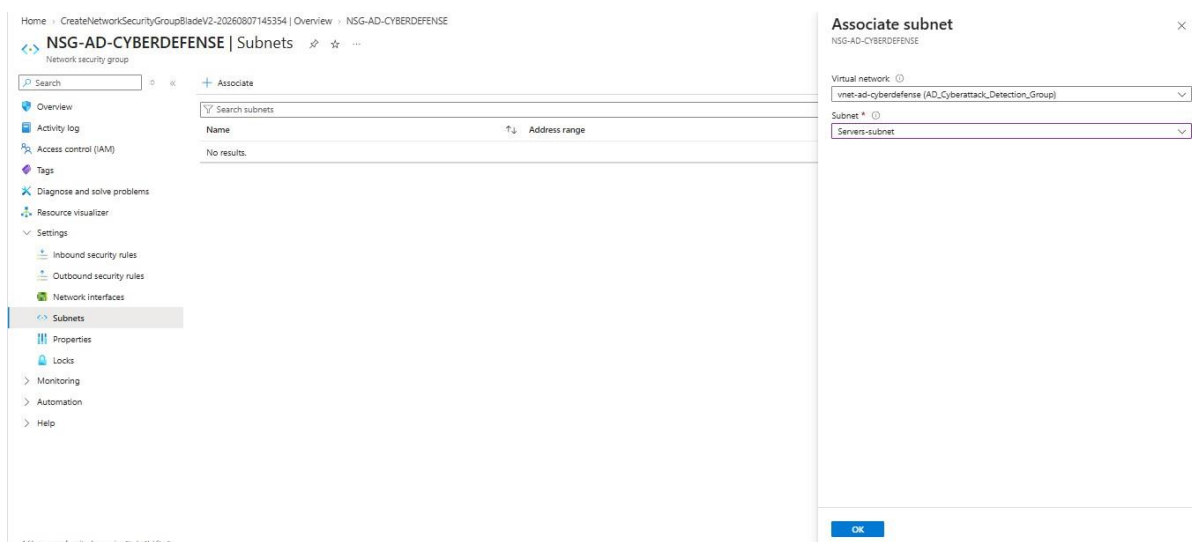


Figure 2.7 — Associating NSG-AD-CYBERDEFENSE with the Servers-subnet inside vnet-ad-cyberdefense.

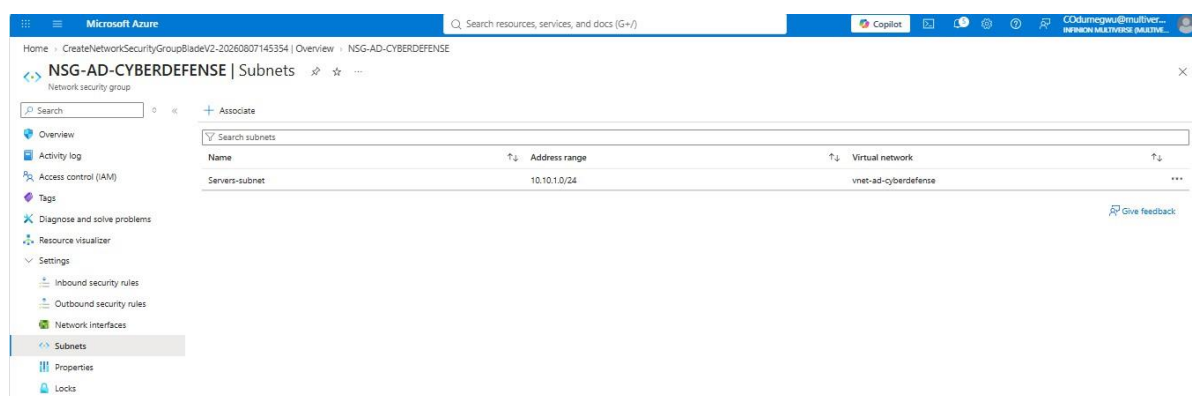


Figure 2.8 — Confirming the association: the Subnets blade of the NSG now lists Servers-subnet (10.10.1.0/24) as linked.

DC01 Machine Deployment

1. Deployed the DC01 virtual machine in Azure using the Windows Server 2022 Datacenter Azure Edition image.



7th floor Ibukun House, Ademola Adetokunbo street, Victoria Island, Lagos, Nigeria



info@infinion.co



infinion.co

2. Connected the VM to the previously created Virtual Network (vnet-ad-cyberdefense) and Subnet (Servers-subnet).
3. Assigned a Public IP address to enable remote administration via RDP.
4. Configured the VM to inherit security controls from the subnet-level Network Security Group (NSG-AD-CYBERDEFENSE).
5. Verified successful VM deployment and provisioning within Azure.
6. Established Remote Desktop Protocol (RDP) connectivity to DC01.
7. Validated successful login using administrative credentials.
8. Configured and verified the static private IP address: 10.10.1.4.
9. Verified the server hostname: DC01.
10. Verified the network configuration using PowerShell (ipconfig).
11. Confirmed communication with the Azure virtual network and gateway.

Outcome: The DC01 server was successfully deployed and prepared for Active Directory installation. The server now has a consistent static IP address and is ready to be promoted to a Domain Controller in the next phase.

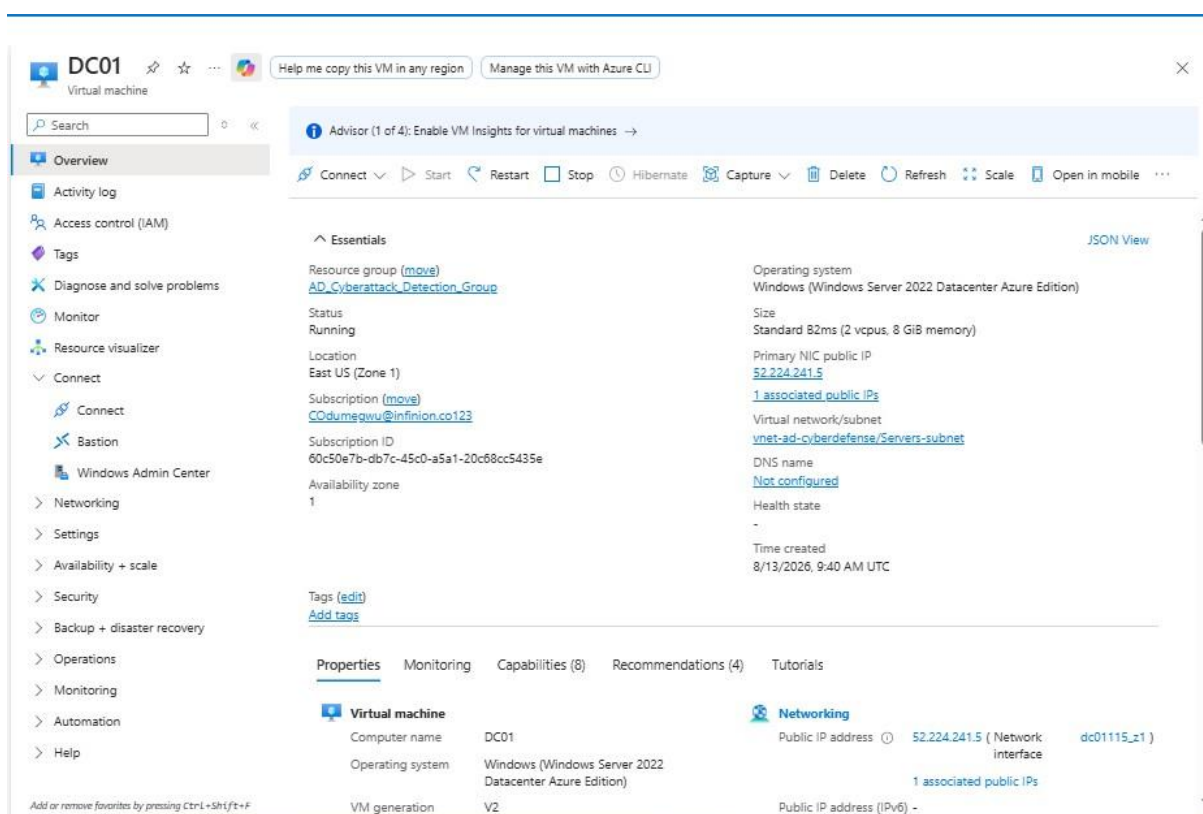


Figure 3.1 — DC01 Overview blade confirming the VM is Running on Windows Server 2022 Datacenter Azure Edition (Standard B2ms), deployed in East US inside vnet-ad-cyberdefense/Servers-subnet with public IP 52.224.241.5.



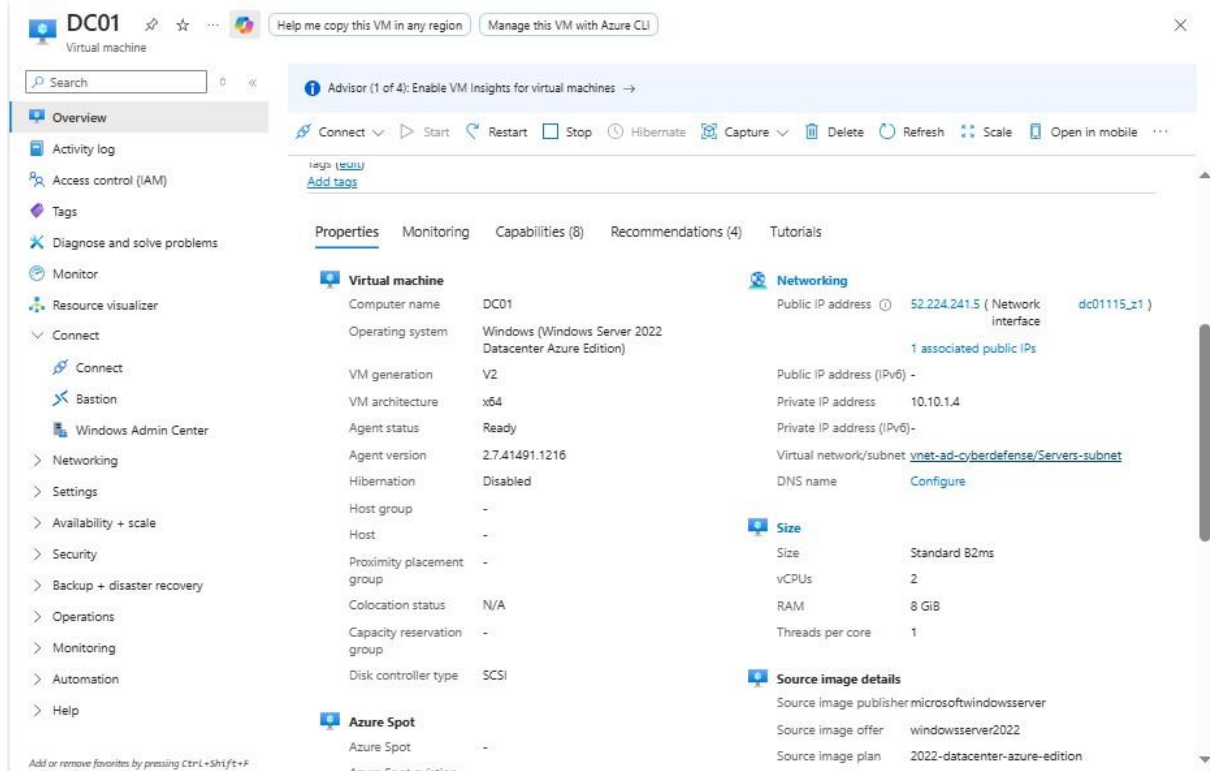


Figure 3.2 — DC01 properties detail confirming the private IP (10.10.1.4), public IP, VM size, and source image (windowsserver2022 / 2022-datacenter-azure-edition).

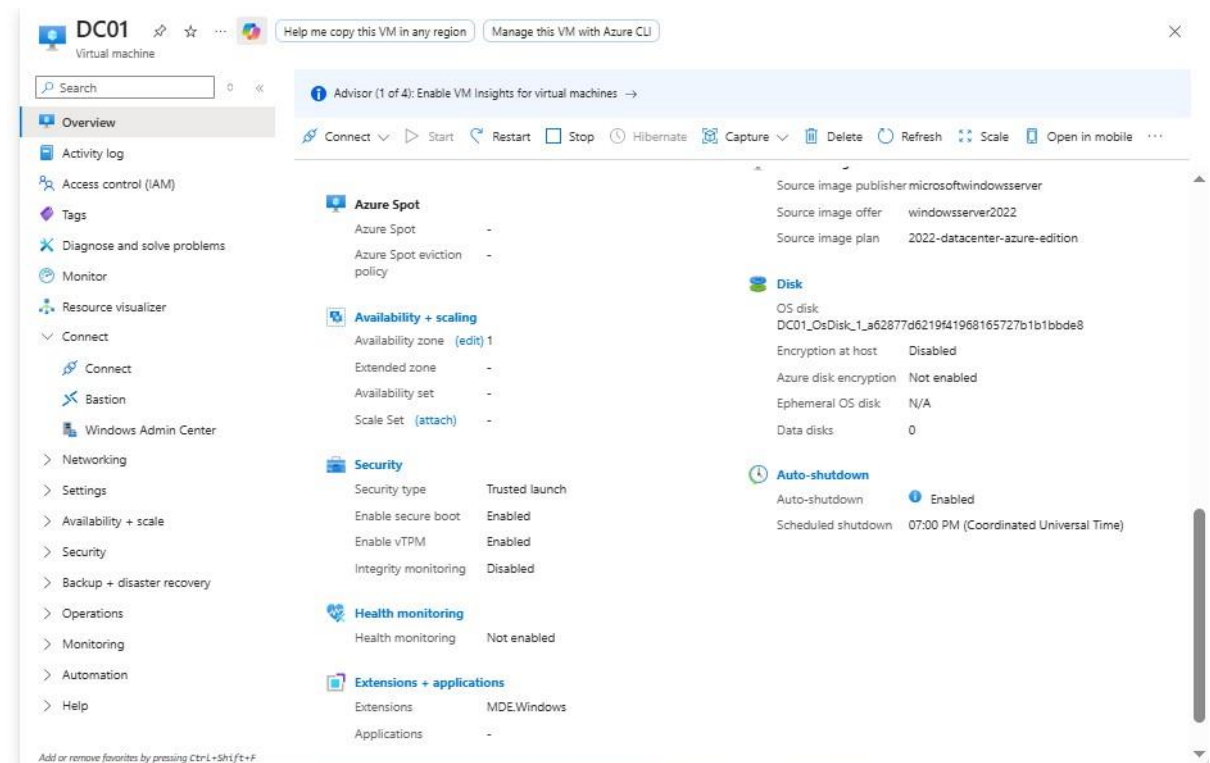


Figure 3.3 — DC01 security and disk configuration: Trusted launch with Secure Boot and vTPM enabled, and the MDE.Windows (Microsoft Defender for Endpoint) extension already present.



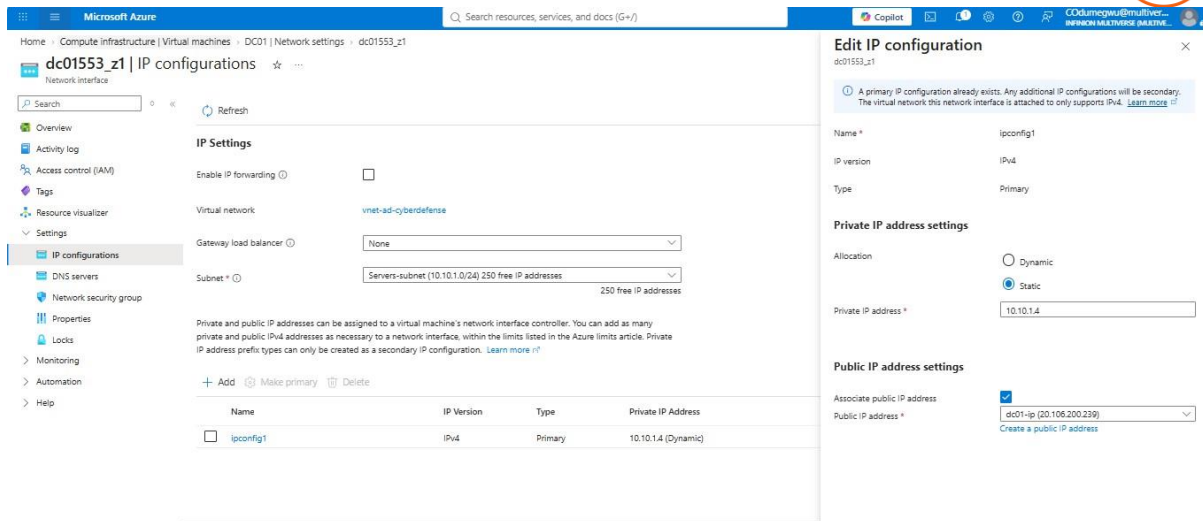


Figure 3.4 — IP configuration blade of the DC01 network interface, showing the private IP set to Static (10.10.1.4) on Servers-subnet, with the dc01-ip public IP associated.

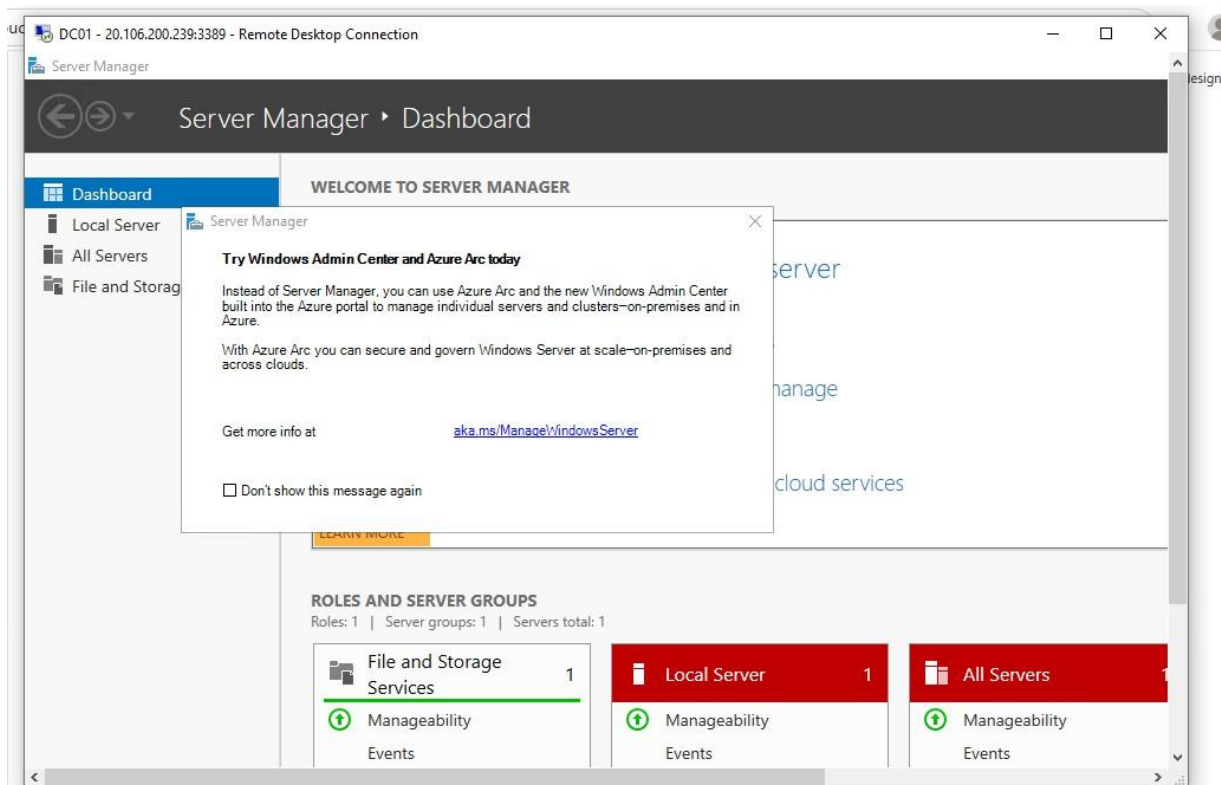


Figure 3.5 — Successful RDP connection to DC01, landing on the Server Manager Dashboard — confirming administrative login worked.

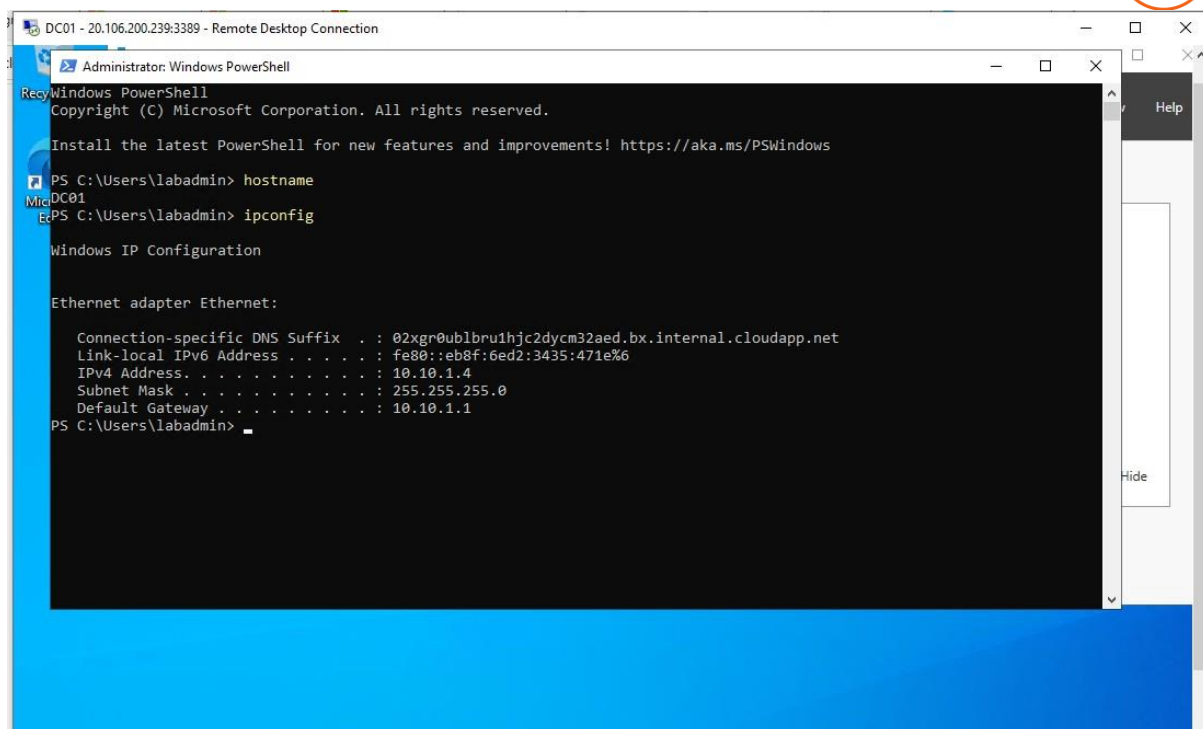


Figure 3.6 — PowerShell session on DC01 running hostname and ipconfig, confirming the hostname (DC01) and the static IPv4 address (10.10.1.4).

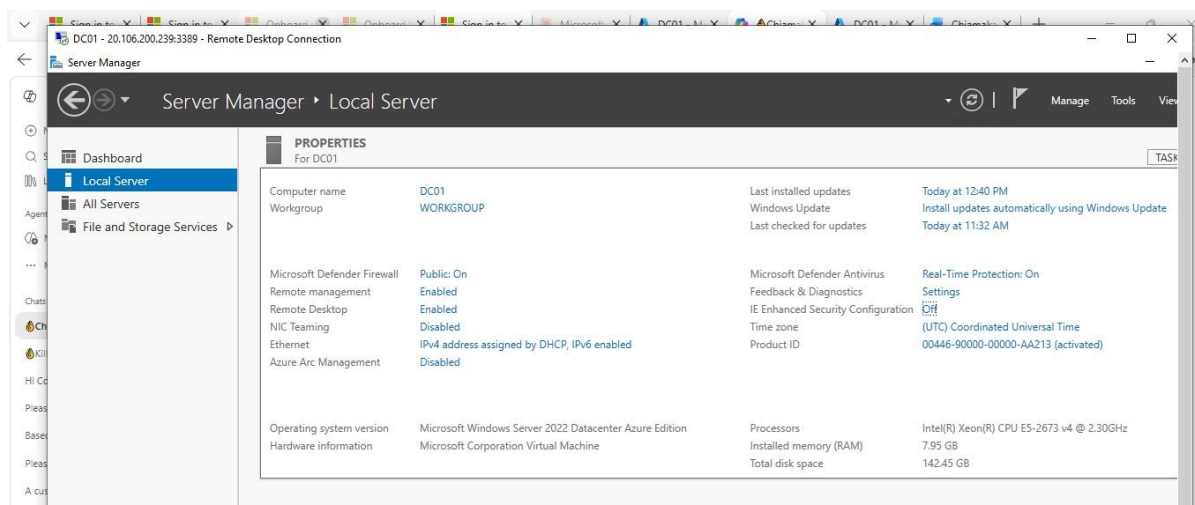


Figure 3.7 — Server Manager → Local Server properties for DC01, verifying computer name, Windows Defender Firewall/Antivirus status, and the OS edition immediately prior to AD DS promotion in Phase 4.

Active Directory Configuration — DC01

1. Installed the Active Directory Domain Services (AD DS) role on DC01.
2. Installed and configured DNS during the AD DS deployment process.
3. Promoted DC01 to a Domain Controller.
4. Created a new Active Directory forest and domain: lab.local.
5. Verified successful domain controller promotion and server restart.
6. Verified Active Directory and DNS services were running.
7. Validated domain information using Active Directory PowerShell commands.



8. Verified DNS name resolution for the lab.local domain.
9. Created the following Organizational Units (OUs): Users account, Servers account, Workstations, Service account, Security groups.
10. Created the following user accounts: john.smith, mary.jones, victor.williams, security.analyst.
11. Created the service account: sqlsvc.
12. Created the security group: IT-Admins.
13. Added the security.analyst account to the IT-Admins security group.
14. Validated all user accounts using PowerShell.
15. Verified Active Directory Users and Computers functionality.

Outcome: A fully functional Active Directory environment was successfully deployed and configured. The environment now contains the organizational structure, user accounts, service accounts, and security groups required to support the attack simulations, threat detection activities, and incident response scenarios that will be implemented in later phases of the project.

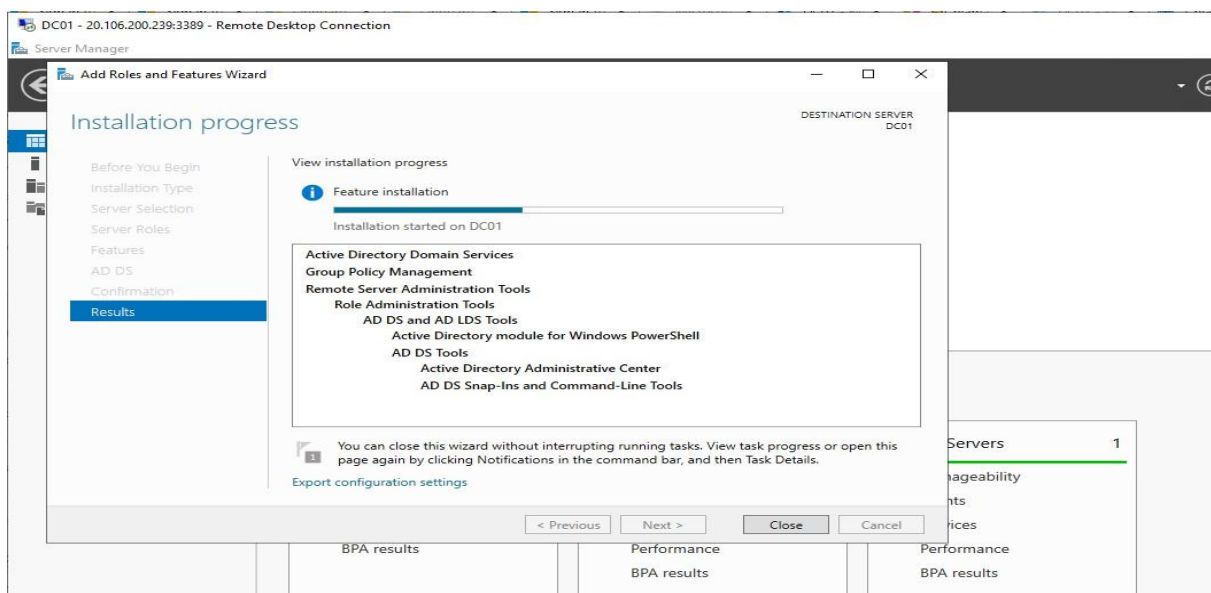


Figure 4.1 — AD DS role installation completing on DC01 (Active Directory Domain Services, Group Policy Management, and the AD DS/AD LDS admin tools), just before promotion to a domain controller.

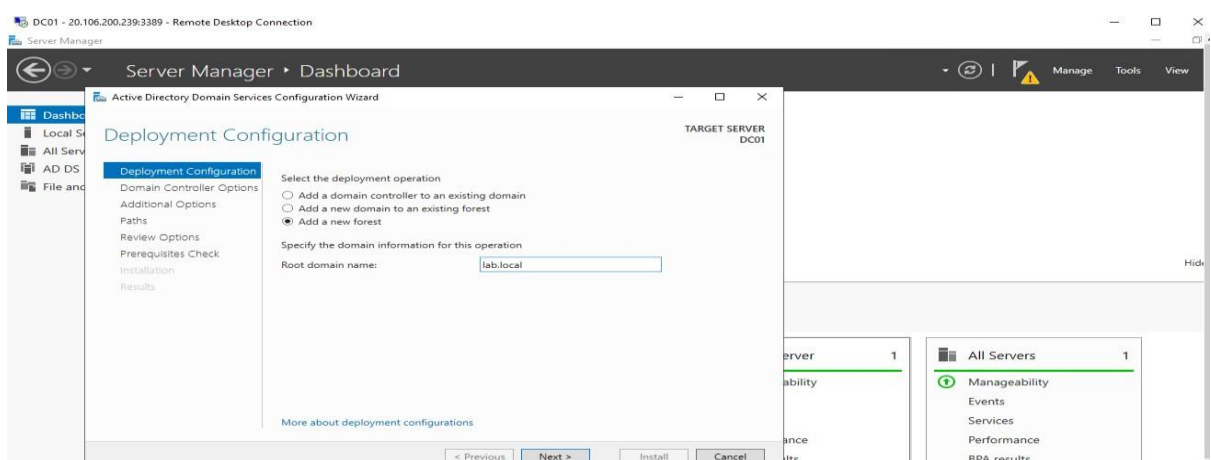


Figure 4.2 — Active Directory Domain Services Configuration Wizard: selecting "Add a new forest" and entering the root domain name lab.local.



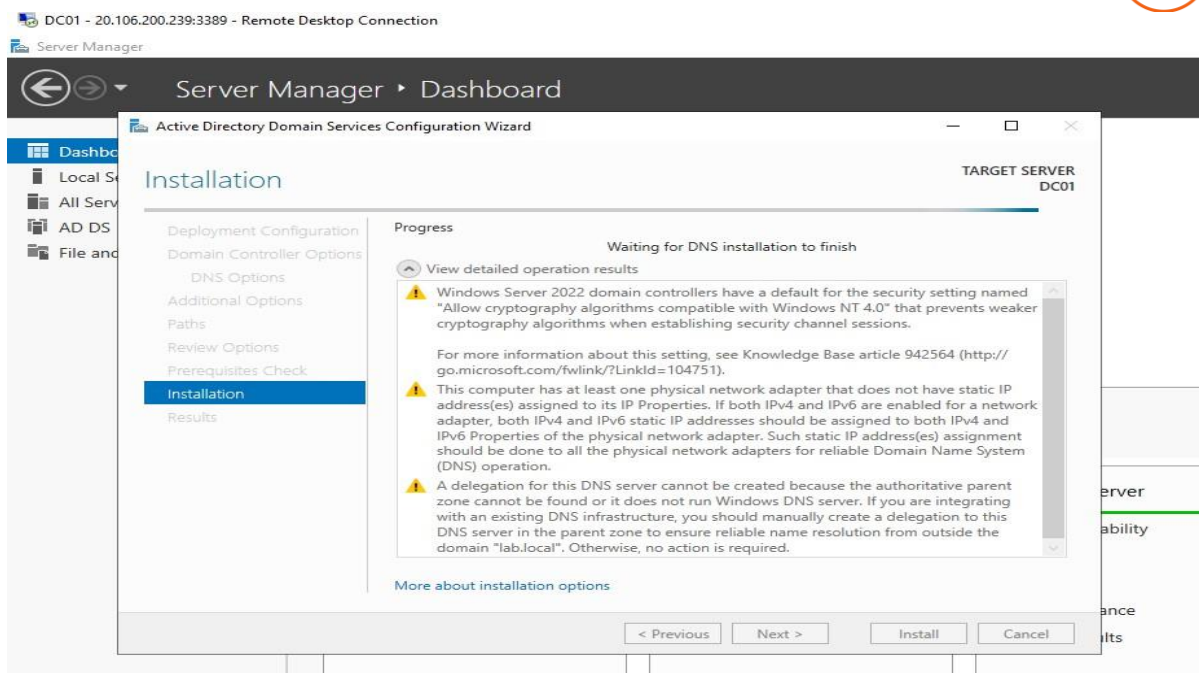


Figure 4.3 — Promotion in progress — the wizard installing DNS alongside AD DS and surfacing the expected default warnings (NT4-compatible crypto, static IP, and DNS delegation) before completing the install.

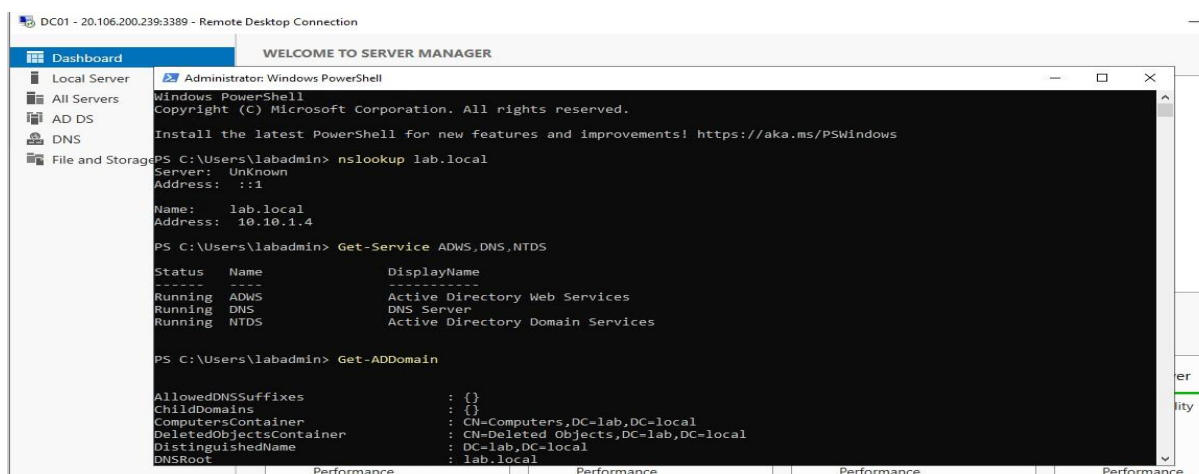


Figure 4.4 — Post-promotion validation on DC01: nslookup lab.local resolving to 10.10.1.4, and Get-Service confirming ADWS, DNS, and NTDS are all Running.

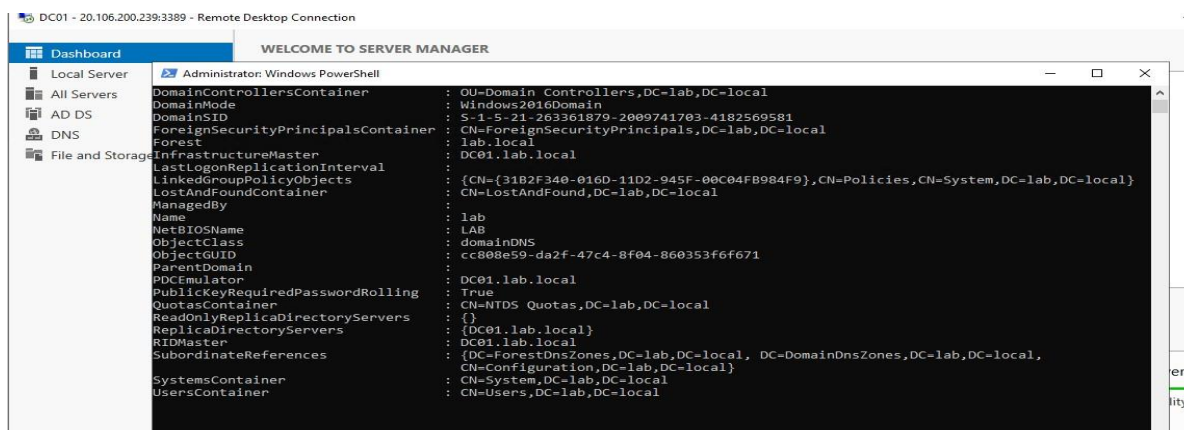


Figure 4.5 — Get-ADDomain output confirming the new forest (lab.local), NetBIOS name LAB, and PDC emulator role held by DC01.lab.local.



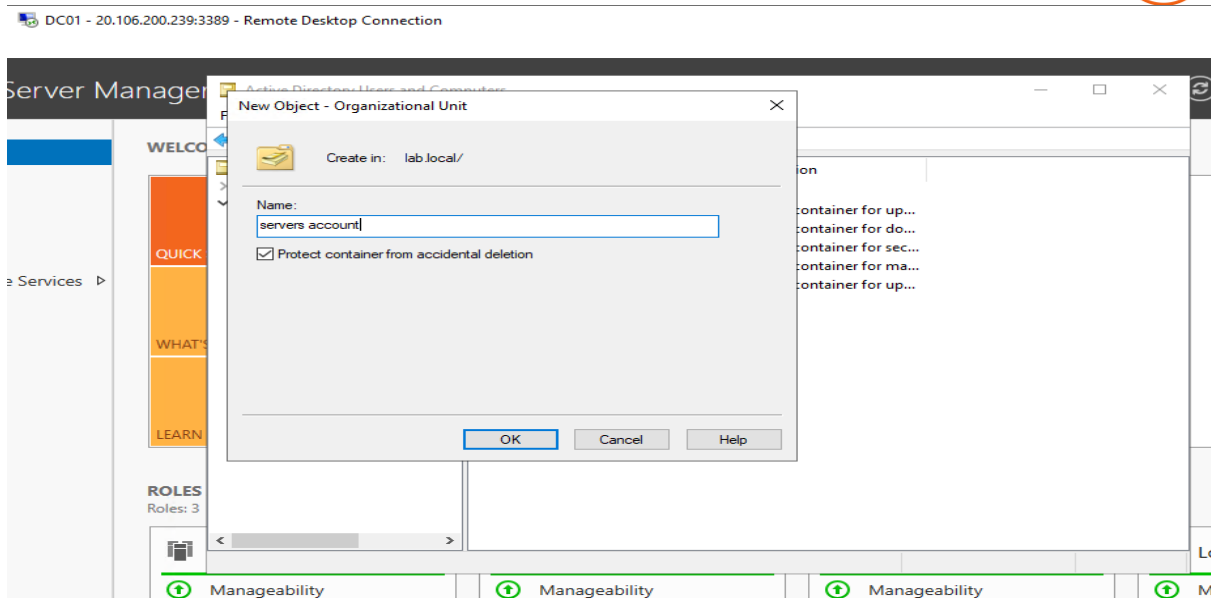


Figure 4.6 — Creating the "servers account" Organizational Unit under lab.local in Active Directory Users and Computers.

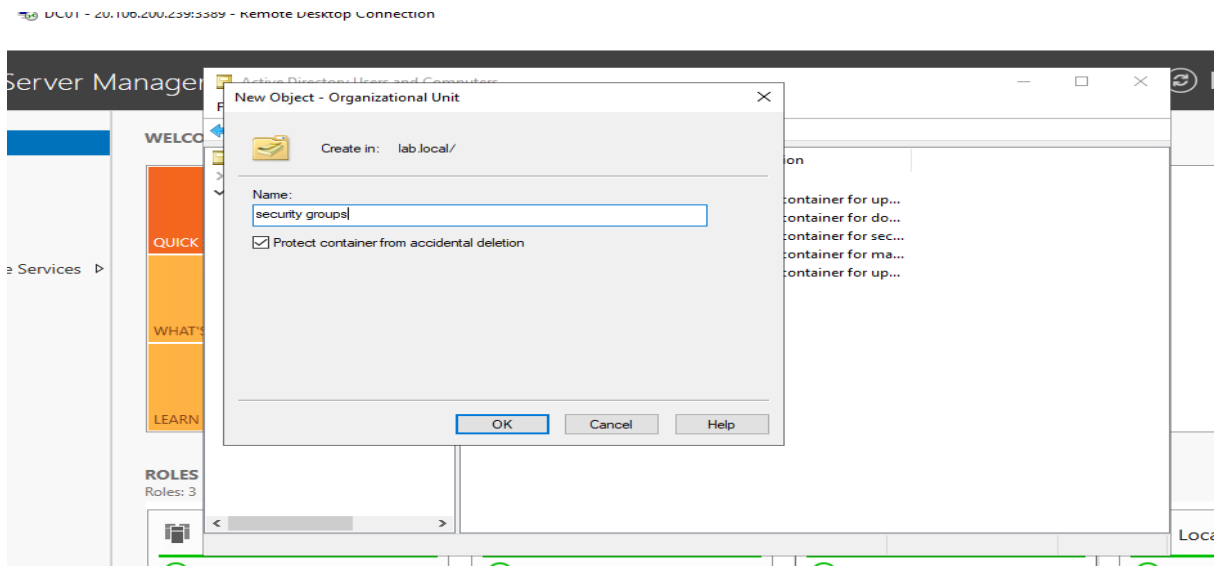


Figure 4.7 — Creating the "security groups" Organizational Unit under lab.local.



DC01 - 20.106.200.239:3389 - Remote Desktop Connection

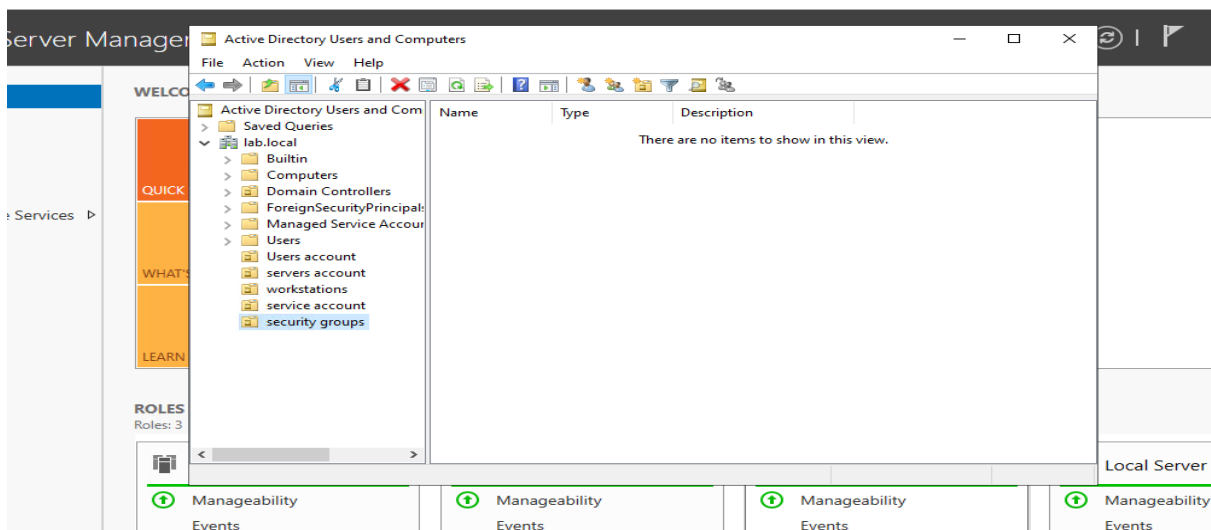


Figure 4.8 — All five planned OUs now present under lab.local: Users account, servers account, workstations, service account, and security groups.

20.106.200.239:3389 - Remote Desktop Connection

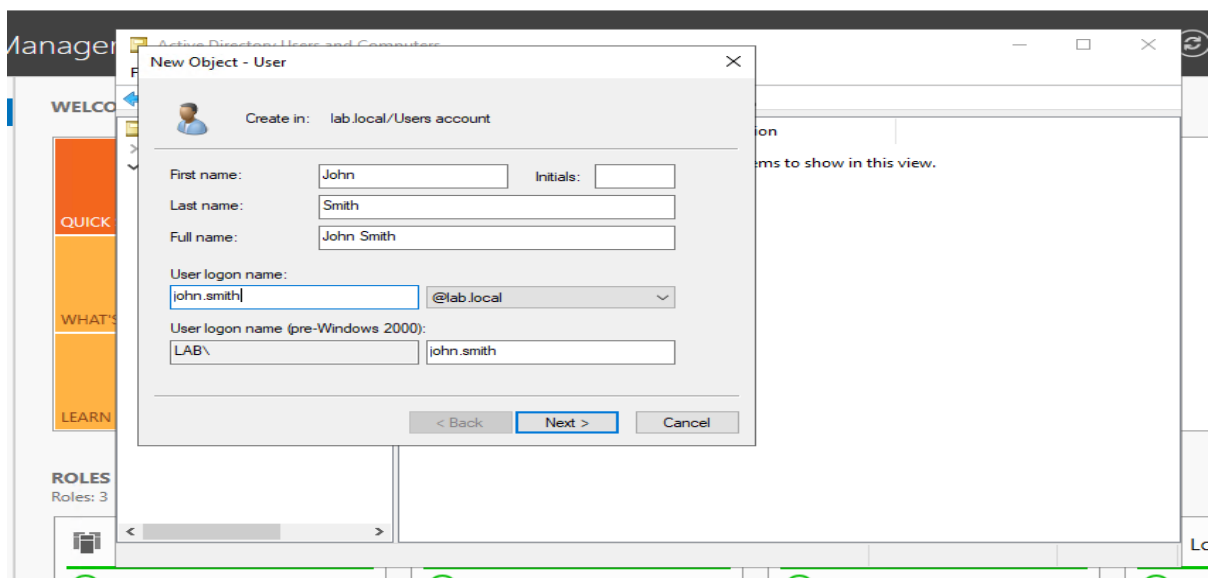


Figure 4.9 — Creating the john.smith user account inside the Users account OU.



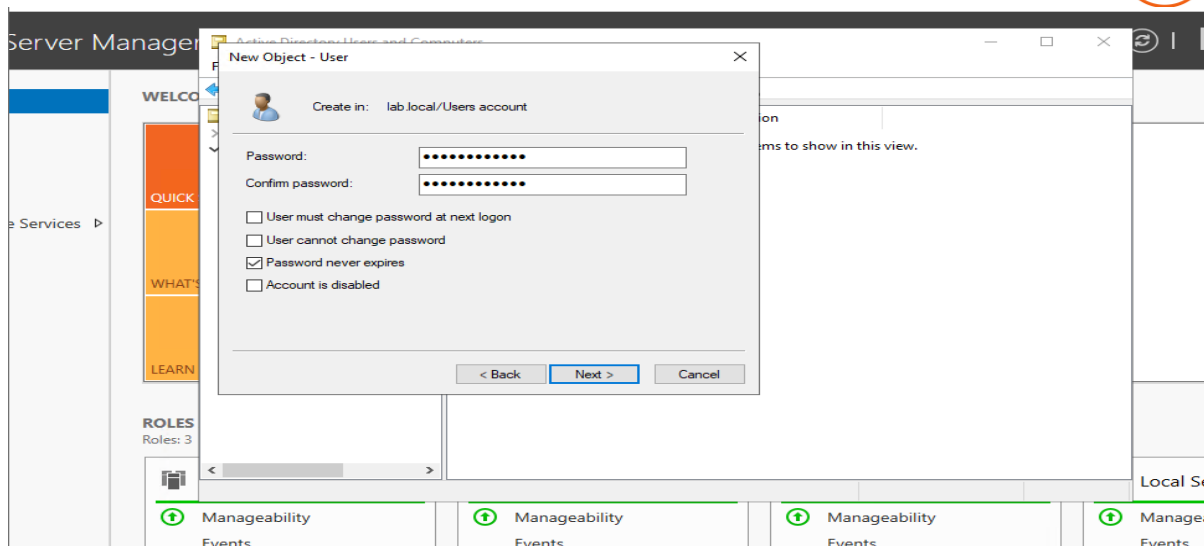


Figure 4.10 — Setting the john.smith password, with "Password never expires" enabled for lab-account stability.

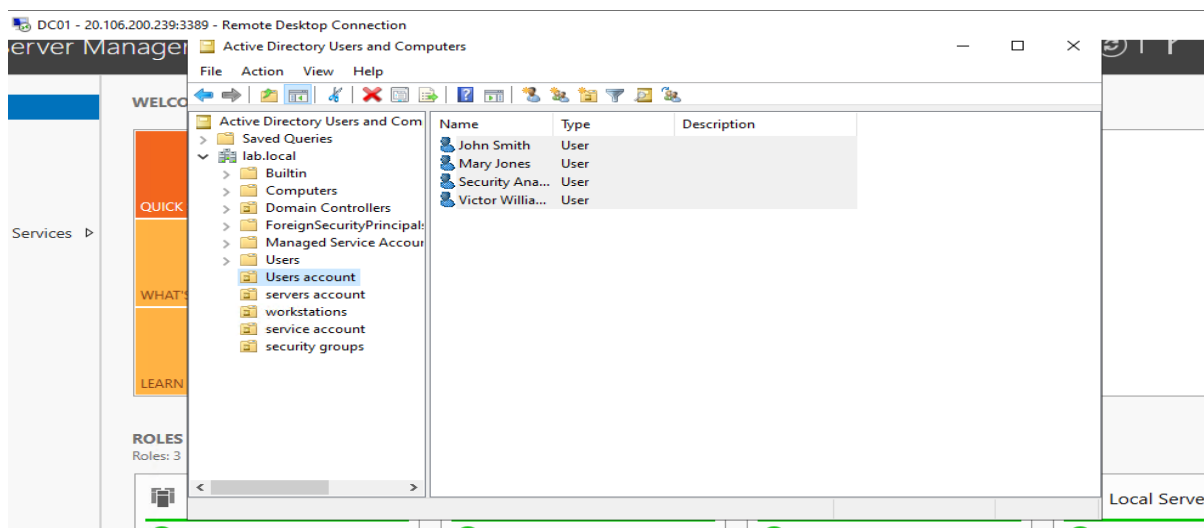


Figure 4.11 — The Users account OU populated with all four user accounts: John Smith, Mary Jones, Security Analyst, and Victor Williams.

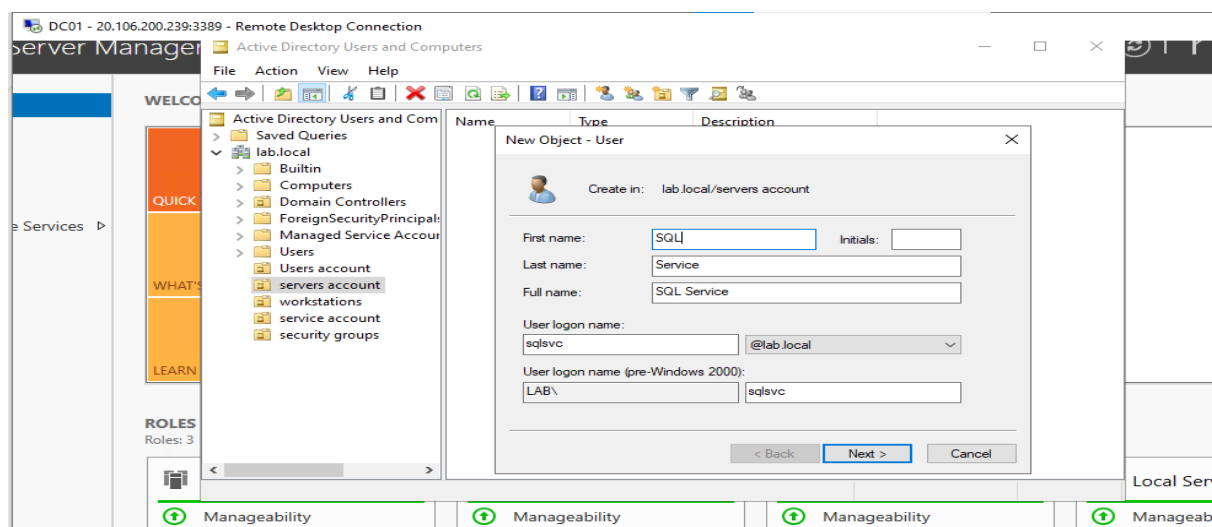


Figure 4.12 — Creating the SQL Service (sqlsvc) account — the account later used as the Kerberoasting target in Phase 9.

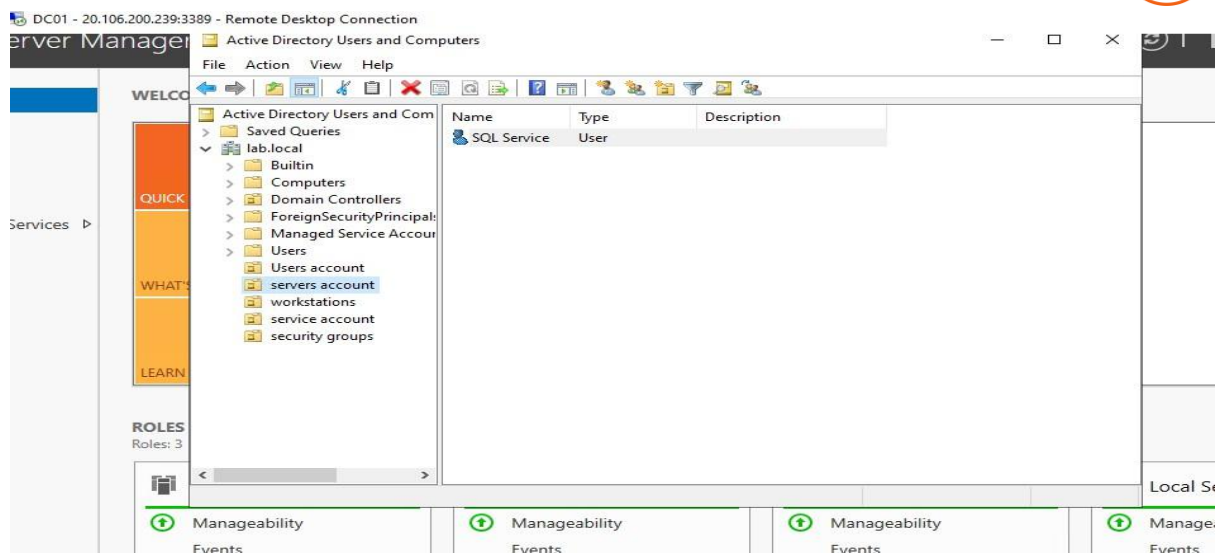


Figure 4.13 — SQL Service account created and visible in the OU listing.

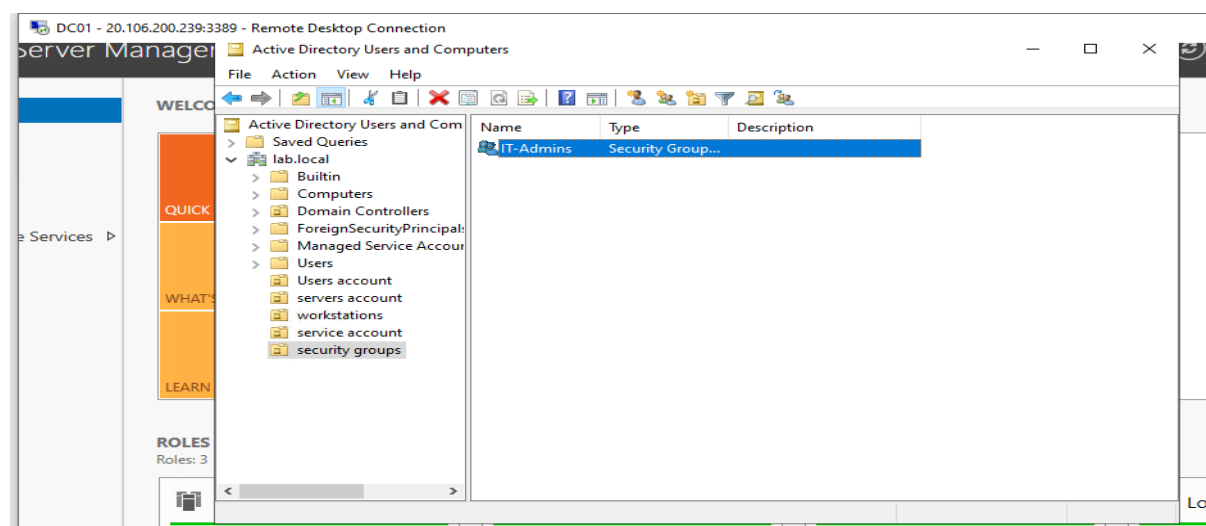


Figure 4.14 — The IT-Admins security group created inside the security groups OU.

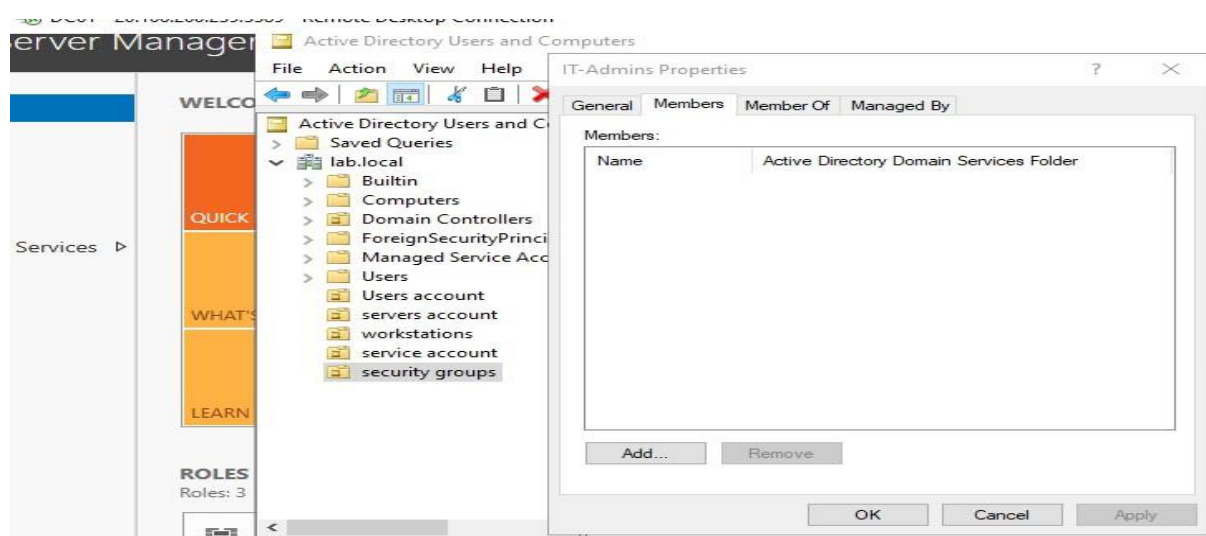


Figure 4.15 — IT-Admins group properties, Members tab, shown empty prior to adding security.analyst.



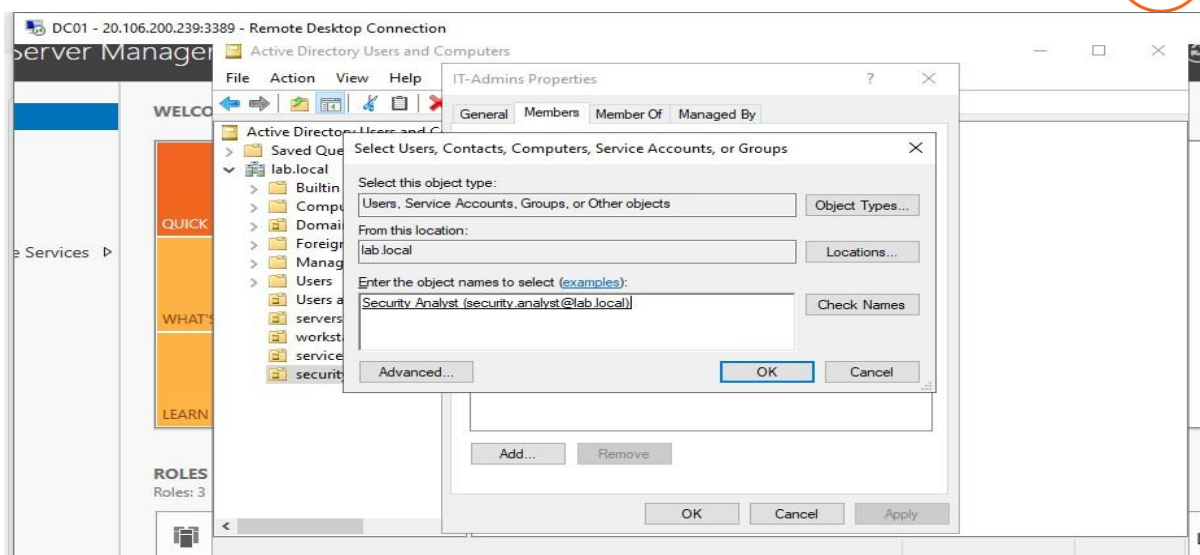


Figure 4.16 — Adding security.analyst to the IT-Admins group via Select Users dialog, with the name resolved through Check Names.

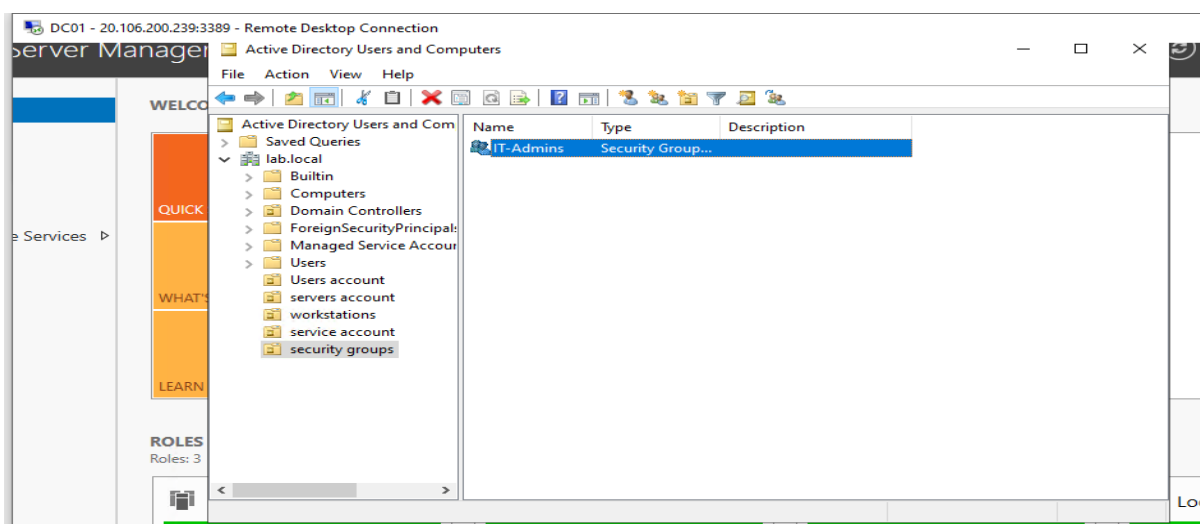


Figure 4.17 — Back at the security groups OU after saving the membership change to IT-Admins.

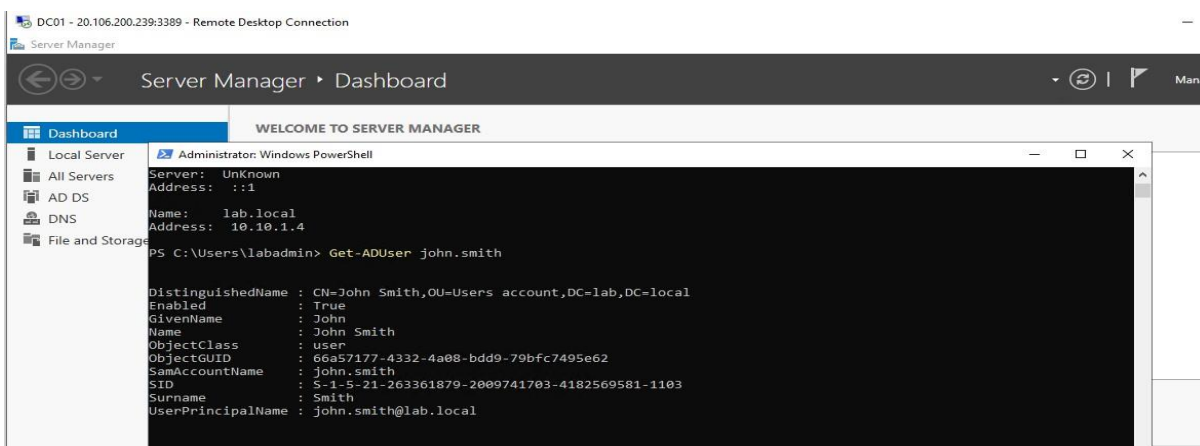


Figure 4.18 — Validating john.smith with Get-ADUser — confirms the account is Enabled and located in the Users account OU.



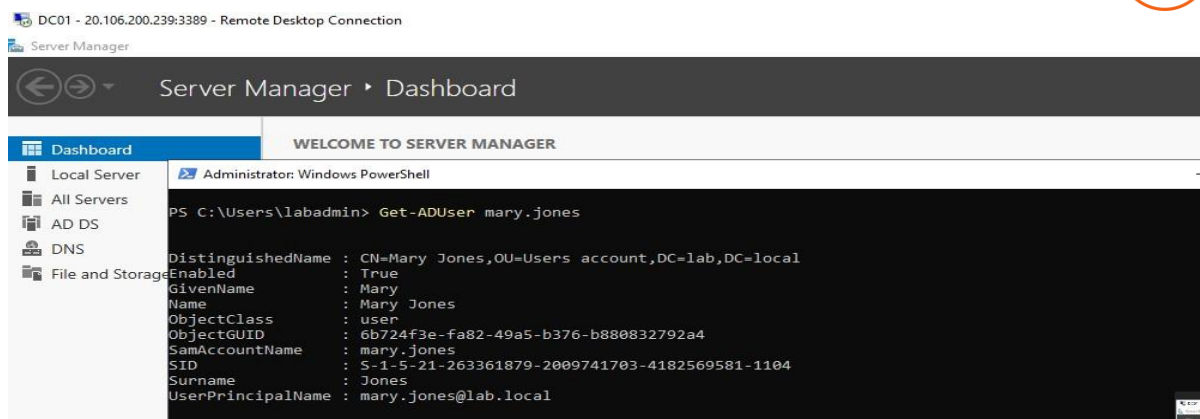


Figure 4.19 — Validating mary.jones with Get-ADUser — account Enabled, correct OU and UPN.

```
PS C:\Users\labamin> setspn -A MSSQLSvc/dc01.lab.local:1433 lab\sqsvc
Checking domain DC=lab,DC=local

Registering ServicePrincipalNames for CN=sqsvc,OU=Service account,DC=lab,DC=local
MSSQLSvc/dc01.lab.local:1433
Updated object
PS C:\Users\labamin>
```

Figure 4.20 — Registering a Service Principal Name on sqlsvc (setspn -A MSSQLSvc/dc01.lab.local:1433 lab\sqsvc), simulating a SQL service account — a realistic Kerberoasting target used later in Phase 9.

What this does: registering the SPN on the sqlsvc account makes it look like the account that runs a SQL Server service on DC01 (port 1433, SQL's default port). This is a very common real-world pattern — SQL service accounts often carry SPNs — and it is exactly the kind of account Kerberoasting targets, since any authenticated domain user can request a Kerberos ticket for a service with an SPN and then attempt to crack the account's password offline from that ticket.

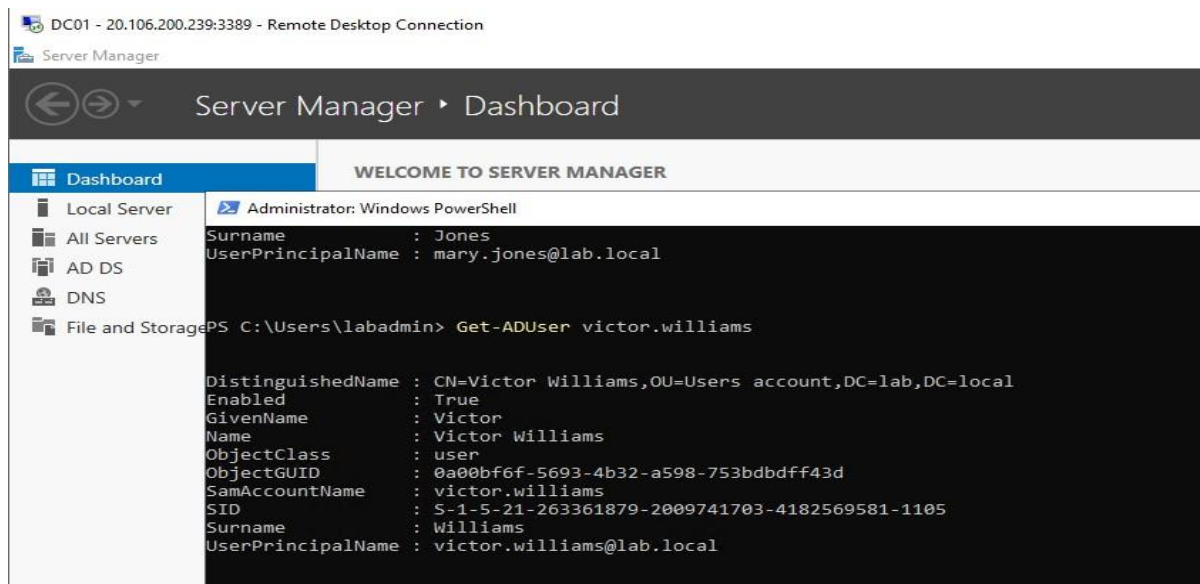


Figure 4.21 — Continued PowerShell validation: Get-ADUser confirming victor.williams is Enabled and correctly placed in the Users account OU.



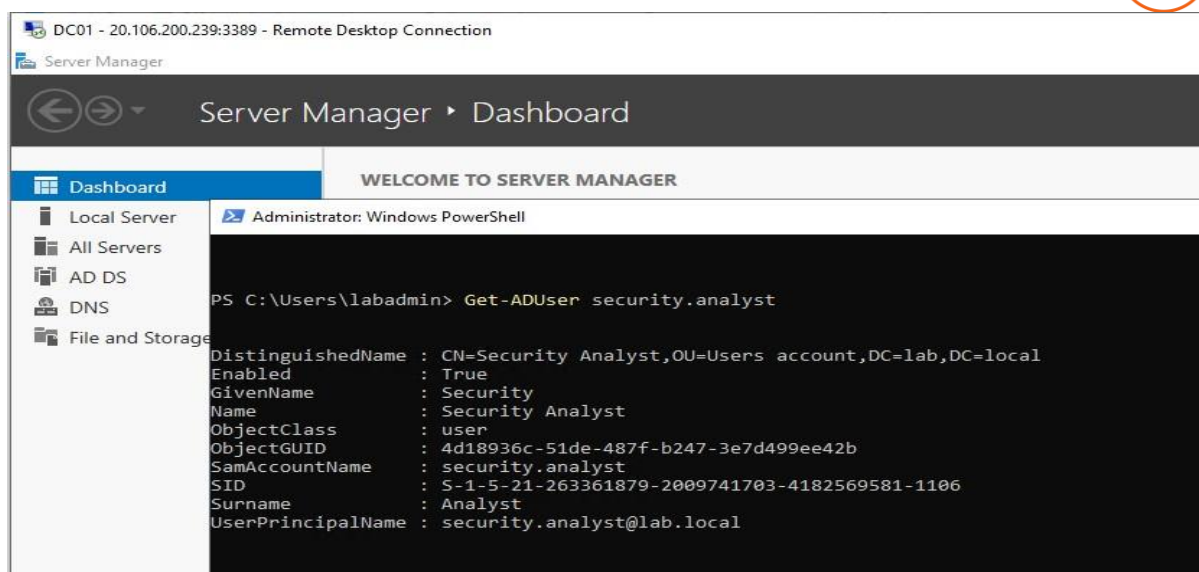


Figure 4.22 — Continued validation: Get-ADUser confirming security.analyst is Enabled and correctly placed in the Users account OU.

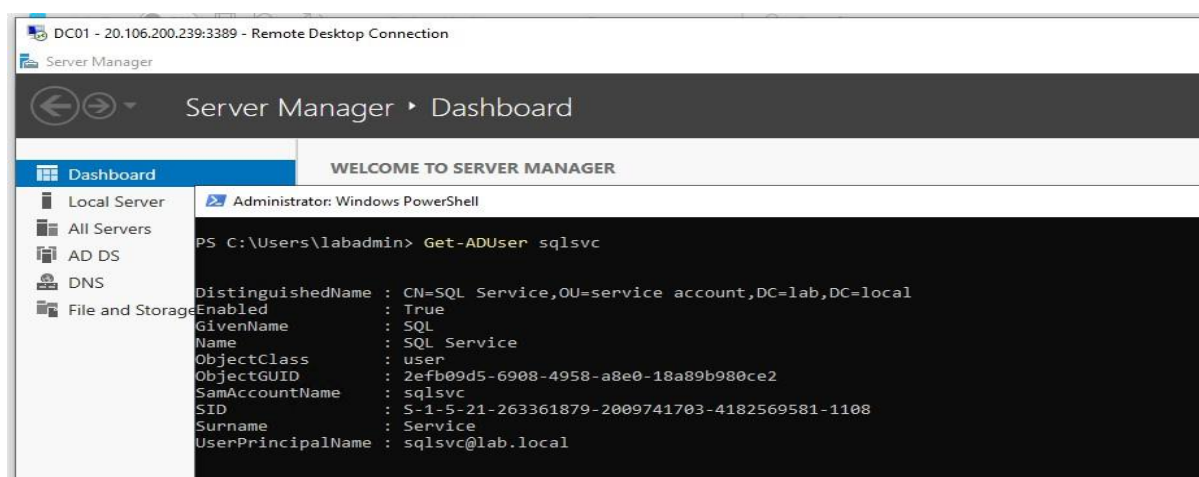


Figure 4.23 — Final validation: Get-ADUser confirming the sqlsvc (SQL Service) account is Enabled and located in the service account OU, ready for its role as the Kerberoasting target.

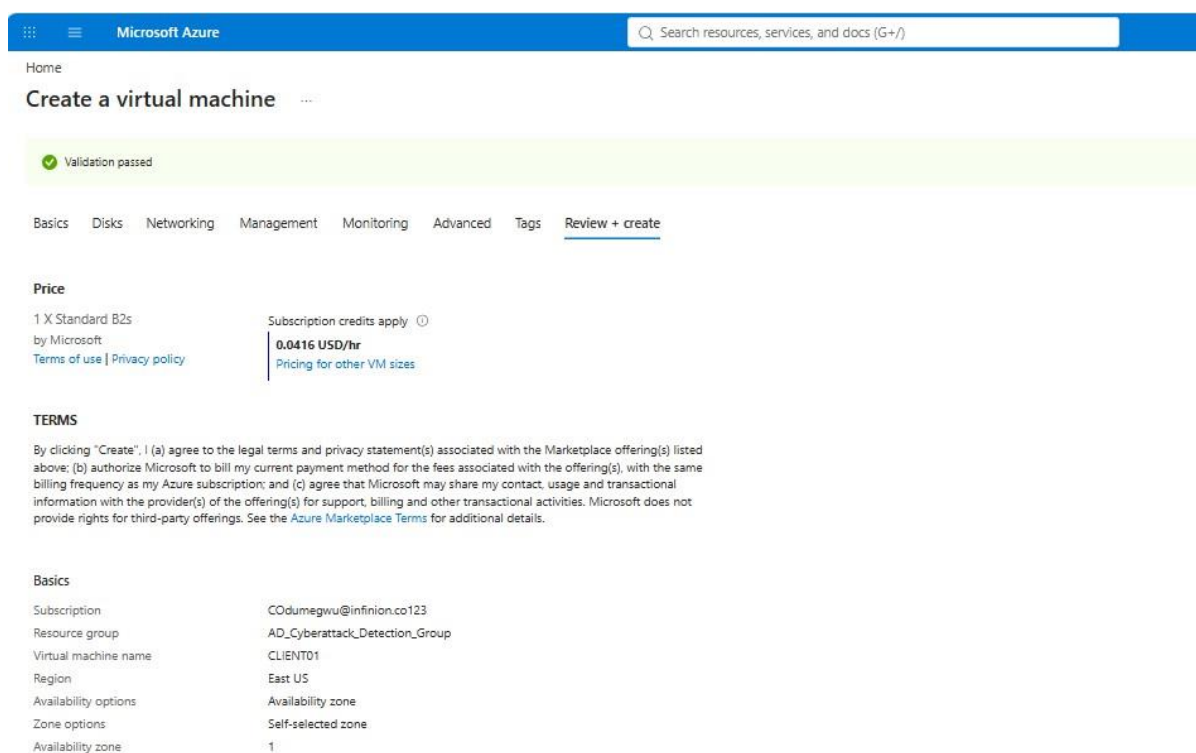
Client Configuration

1. Deployed the CLIENT01 virtual machine using Windows 10 Enterprise 22H2 (x64 Gen 2) on a Standard B2s VM size.
2. Connected CLIENT01 to vnet-ad-cyberdefense / Servers-subnet.
3. Verified successful deployment of CLIENT01 in Azure.
4. Established Remote Desktop (RDP) access to the workstation.
5. Configured DNS settings on CLIENT01 to use the Domain Controller: Preferred DNS Server 10.10.1.4 (DC01).
6. Validated DNS functionality using ipconfig /all, nslookup lab.local, and ping dc01.lab.local.
7. Verified successful communication between CLIENT01 and DC01.
8. Joined CLIENT01 to the Active Directory domain: lab.local.
9. Successfully authenticated the domain join using LAB\labadmin.
10. Received confirmation message: "Welcome to the lab.local domain".



11. Restarted CLIENT01 to apply domain membership changes.
12. Verified domain membership using `systeminfo | findstr /B /C:"Domain"`.
13. Confirmed CLIENT01 is a member of lab.local.
14. Validated Active Directory authentication using `runas /user:LAB\john.smith cmd`.
15. Confirmed successful domain user authentication with `whoami` — result: `lab\john.smith`.

Outcome: CLIENT01 was successfully deployed, configured, and joined to the lab.local Active Directory domain. The workstation can communicate with DC01 and authenticate domain users successfully. The environment now consists of a functioning Domain Controller and a domain-joined workstation, providing the foundation required for Microsoft Defender for Identity integration, Microsoft Sentinel integration, security monitoring, and Active Directory attack simulations.



Microsoft Azure

Search resources, services, and docs (G+)

Home

Create a virtual machine

Validation passed

Basics Disks Networking Management Monitoring Advanced Tags Review + create

Price

1 X Standard B2s
by Microsoft
[Terms of use](#) | [Privacy policy](#)

Subscription credits apply ⓘ
0.0416 USD/hr
[Pricing for other VM sizes](#)

TERMS

By clicking "Create", I (a) agree to the legal terms and privacy statement(s) associated with the Marketplace offering(s) listed above; (b) authorize Microsoft to bill my current payment method for the fees associated with the offering(s), with the same billing frequency as my Azure subscription; and (c) agree that Microsoft may share my contact, usage and transactional information with the provider(s) of the offering(s) for support, billing and other transactional activities. Microsoft does not provide rights for third-party offerings. See the [Azure Marketplace Terms](#) for additional details.

Basics

Subscription	CODumegwu@infinion.co123
Resource group	AD_Cyberattack_Detection_Group
Virtual machine name	CLIENT01
Region	East US
Availability options	Availability zone
Zone options	Self-selected zone
Availability zone	1

Figure 5.1 — Review + create screen for CLIENT01, confirming validation passed with Standard B2s size in the AD_Cyberattack_Detection_Group resource group.



Home

Create a virtual machine

✓ Validation passed

Zone options	Self-selected zone
Availability zone	1
Security type	Trusted launch virtual machines
Enable secure boot	Yes
Enable vTPM	Yes
Integrity monitoring	No
Image	Windows 10 Enterprise, version 22H2 - Gen2
VM architecture	x64
Size	Standard B2s (2 vcpus, 4 GiB memory)
Enable Hibernation	No
Username	labuser
Already have a Windows license?	Yes
License type	Windows Client
Azure Spot	No

Disks

OS disk size	Image default
OS disk type	Premium SSD LRS
Use managed disks	Yes
Delete OS disk with VM	Enabled
Ephemeral OS disk	No

Networking

Virtual network	cont-az-subnetdefaz
-----------------	---------------------

Figure 5.2 — Continued review pane confirming the image (Windows 10 Enterprise, version 22H2 — Gen2), VM size, and Trusted Launch security settings (Secure Boot and vTPM enabled).



Home

Create a virtual machine

Validation passed

Microsoft Defender for Cloud	Standard
System assigned managed identity	Off
Login with Microsoft Entra ID	Off
Auto-shutdown	On
Backup	Disabled
Site Recovery	Disabled
Enable periodic assessment	Off
Enable hotpatch	Off
Patch orchestration options	OS-orchestrated patching: patches will be installed by OS

Monitoring

Alerts	Off
Boot diagnostics	Off
Enable OS guest diagnostics	Off
Enable application health monitoring	Off

Advanced

Extensions	None
VM applications	None
Cloud init	No
User data	No
Disk controller type	SCSI
Proximity placement group	None
Capacity reservation group	None

Figure 5.3 — Remaining review settings: Microsoft Defender for Cloud set to Standard, monitoring/advanced options left at their defaults, and auto-shutdown enabled.

Home

CreateVm-MicrosoftWindowsDesktop.Windows-10-win10-20260810194001 | Overview

Deployment

Search

Delete Cancel Redeploy Download Refresh

Overview

Inputs

Outputs

Template

✓ Your deployment is complete

Deployment name : CreateVm-MicrosoftWindowsDesktop.Windows-10-win10-20260810194001 Start time : 8/10/2026, 8:04:23 PM

Subscription : C0dumegwu@infinion.co123 Correlation ID : 1d573410-da92-4b35-8bfb-dac8226327d2

Resource group : AD_Cyberattack_Detection_Group

Deployment details

Next steps

Set up auto-shutdown Recommended

Monitor VM health, performance, and network dependencies Recommended

Run a script inside the virtual machine Recommended

Go to resource Create another VM Scale out your VM

Cost management

Get notified to stay within your budget and prevent unexpected charges on your bill.

Set up cost alerts >

Figure 5.4 — Deployment confirmation — “Your deployment is complete” for the CLIENT01 virtual machine.



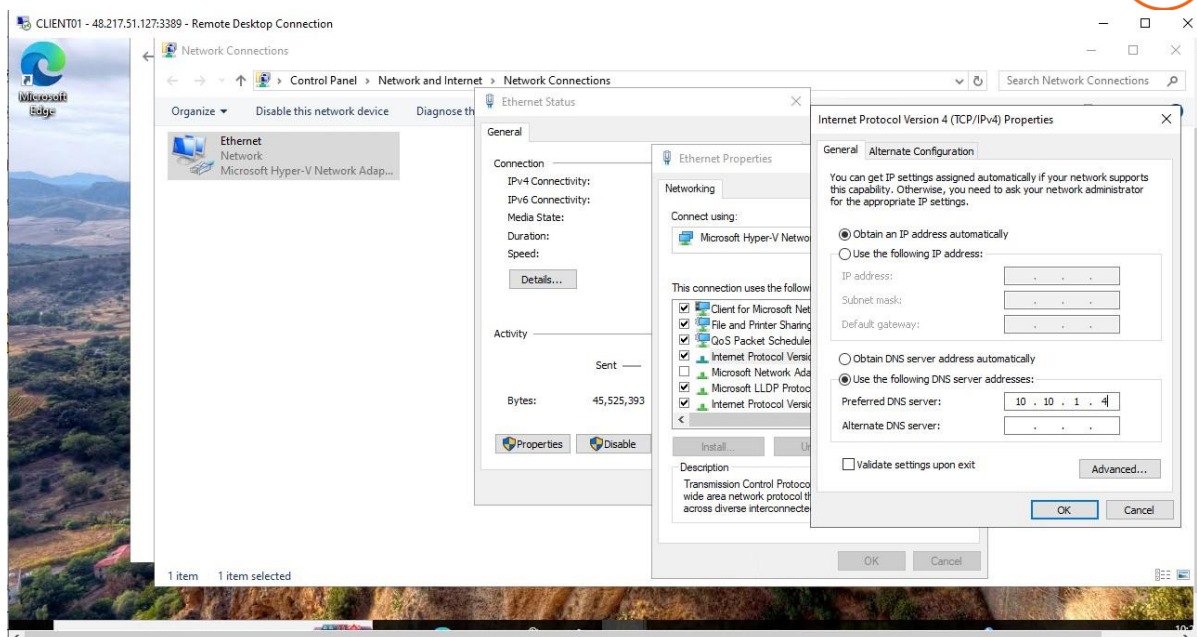


Figure 5.5 — Configuring CLIENT01's IPv4 properties, setting the Preferred DNS server to 10.10.1.4 (DC01) so the workstation can resolve the lab.local domain.

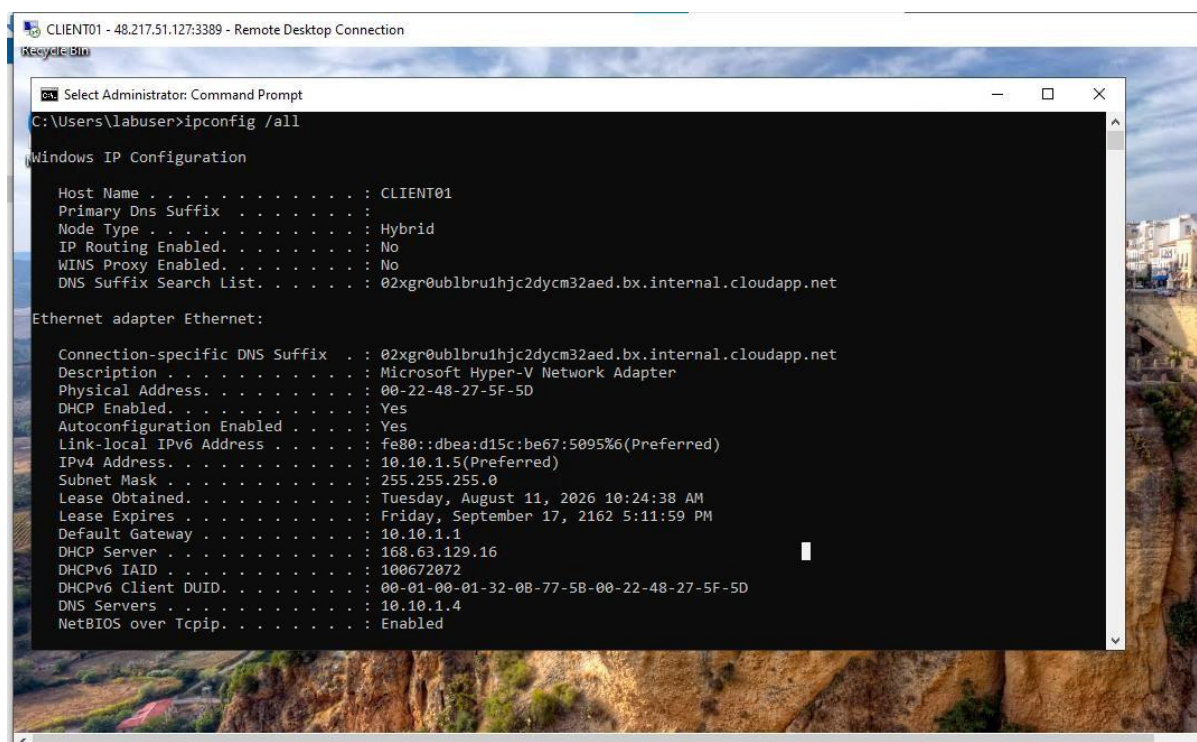


Figure 5.6 — ipconfig /all on CLIENT01, confirming the hostname (CLIENT01), assigned IP (10.10.1.5), and DNS Servers correctly pointing to 10.10.1.4.



```

NetBIOS over Tcpip. . . . . : Enabled

C:\Users\labuser>nslookup lab.local
Server: dc01.internal.cloudapp.net
Address: 10.10.1.4

Name:    lab.local
Address: 10.10.1.4

C:\Users\labuser>ping dc01.lab.local

Pinging dc01.lab.local [10.10.1.4] with 32 bytes of data:
Reply from 10.10.1.4: bytes=32 time=1ms TTL=128
Reply from 10.10.1.4: bytes=32 time=1ms TTL=128
Reply from 10.10.1.4: bytes=32 time=1ms TTL=128
Reply from 10.10.1.4: bytes=32 time=1ms TTL=128

Ping statistics for 10.10.1.4:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 1ms, Maximum = 1ms, Average = 1ms

C:\Users\labuser>

```

Figure 5.7 — nslookup lab.local resolving via DC01, and ping dc01.lab.local succeeding with 0% packet loss — confirming DNS resolution and network connectivity to the domain controller.

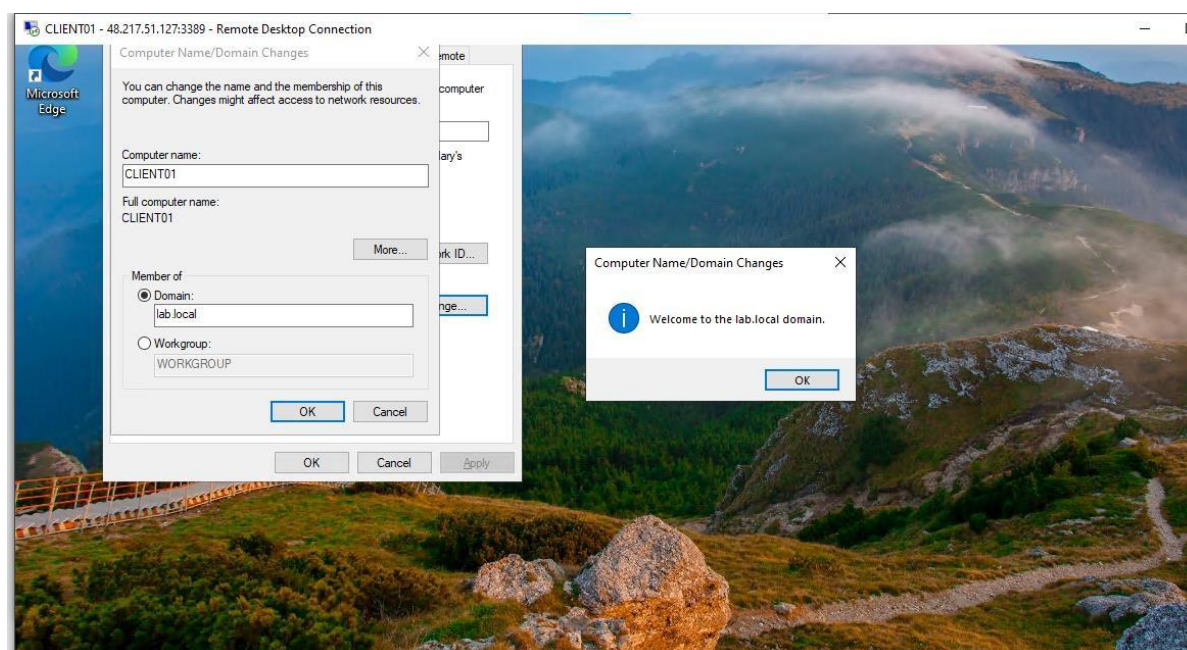


Figure 5.8 — Joining CLIENT01 to the lab.local domain — the “Welcome to the lab.local domain” confirmation message following authentication as LAB\labadmin.



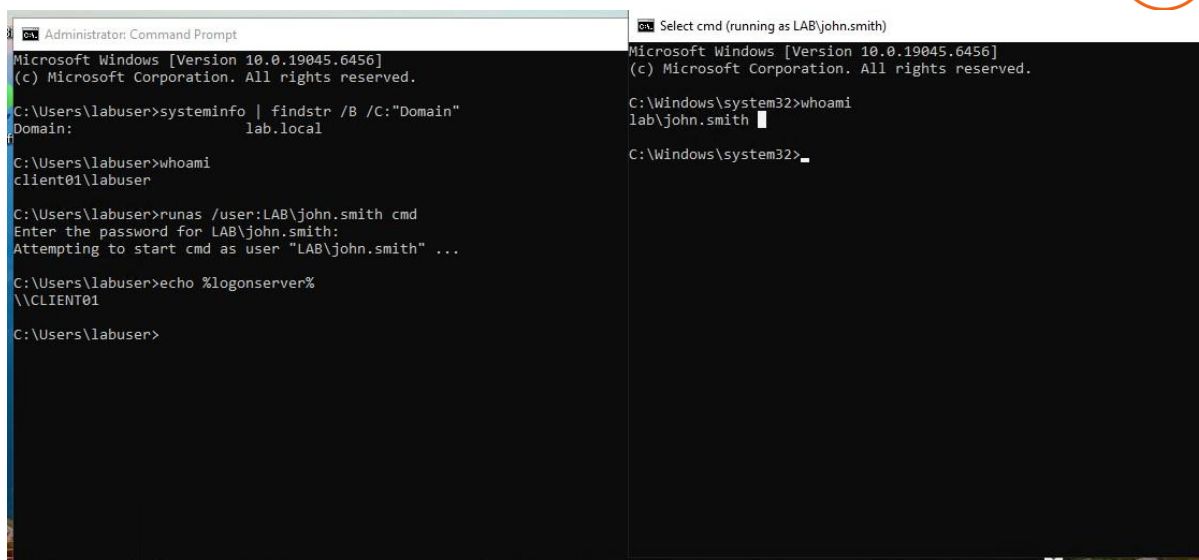


Figure 5.9 — Post-restart validation: systeminfo confirms Domain: lab.local; runas /user:LAB\john.smith cmd opens a shell as the domain user, and whoami in that shell returns lab\john.smith, confirming successful domain-user authentication.

Kali (Attack Machine) Configuration

1. Lessons Applied From Prior Incident

Before deployment, the VM size lesson from the DC01 rebuild (Dsv3 quota failure in East US) was applied upfront — Standard_B2ms was selected directly, with quota availability confirmed in East US before starting deployment, avoiding a repeat of the earlier failure.

2. VM Deployment — KALI01

KALI01 was deployed via the Azure Portal into the existing lab infrastructure:

1. Resource Group: AD_Cyberattack_Detection_Group
2. Region: East US
3. Size: Standard_B2ms
4. Image: Official Kali Linux Marketplace image (publisher: Kali Linux, first-party) — Kali 2026.2, x64, Gen 2
5. Authentication: SSH public key authentication configured (selected over password authentication for stronger security posture and consistency with the lab's existing hardened design)
6. Disk: Standard SSD

3. Networking Configuration

KALI01 was placed on the existing subnet and configured to match DC01/CLIENT01's established pattern:

1. Subnet: Servers-subnet (10.10.1.0/24), on vnet-ad-cyberdefense
2. Static private IP: 10.10.1.6 — set manually via NIC → IP configurations after deployment
3. Public IP: Static, Standard SKU
4. NIC-level NSG: Set to None, so the VM inherits security rules from the subnet-level NSG-AD-CYBERDEFENSE rather than maintaining a separate NIC-level NSG



5. Public inbound ports: None opened by default — access instead gated through Just-in-Time (JIT) VM Access

4. Just-in-Time (JIT) VM Access

JIT was configured via the VM's Connect blade (Configure JIT + Request access), rather than manually through Defender for Cloud's Workload Protections blade, which had not yet indexed the newly created VM:

1. Port 22 (SSH) policy created, scoped to the current public IP
2. Access requested and granted for a defined time window
3. This ensures SSH access is only briefly open on request, consistent with the JIT approach already used on DC01

5. Connectivity Verification

A successful SSH connection was established from a local Windows machine to KALI01 using the downloaded private key (`ssh -i "KALI01-key.pem" kaliuser@<public-ip>`). This confirmed JIT access, NSG inheritance, and SSH key authentication were all functioning correctly end-to-end.

6. Attack Tooling Installation

The system was first updated, then the following AD attack toolkit was installed and individually verified as functional: Nmap (7.99) for network/service reconnaissance; Impacket — suite of scripts including GetUserSPNs and secretsdump (v0.14.0.dev0) for Kerberoasting, credential dumping, and DCSync support; Responder (3.2.2.0) for LLMNR/NBT-NS hash capture (supports Pass-the-Hash); the BloodHound Python collector (bloodhound-python, verified functional) for AD relationship/attack-path enumeration; ldapsearch/OpenLDAP utilities (2.6.10) for manual LDAP-based AD enumeration; smbclient (4.24.3-Debian) for SMB share enumeration; and NetExec / nxc (1.5.1) for password spraying, Pass-the-Hash, and lateral movement testing.

During verification, package naming differences were identified and resolved — Impacket scripts on this Kali release are installed as `impacket-<toolname>` (e.g. `impacket-GetUserSPNs`, `impacket-secretsdump`) rather than the older `toolname.py` convention, and this was confirmed via `dpkg -L` package inspection.

Outcome: KALI01 is fully deployed, network-isolated per the lab's security design (static IP, NSG-none/inherit, JIT-gated access), and equipped with a verified, functioning attack toolkit mapped against the specific attacks defined in the project scope (Password Spraying, Kerberoasting, Pass-the-Hash, DCSync, Lateral Movement). This satisfies Phase 6's expected outcome: "Kali Linux machine ready for attack simulations."



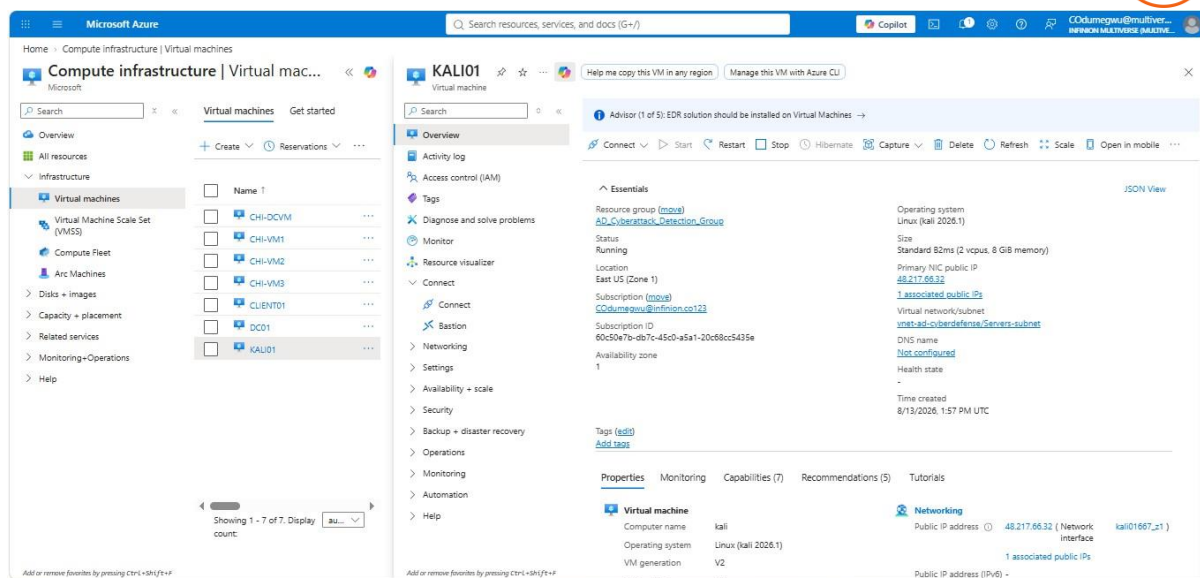


Figure 6.1 — KALI01 VM overview: Running on Linux (kali 2026.1), Standard B2ms, sharing vnet-ad-cyberdefense/Servers-subnet with DC01 and CLIENT01.

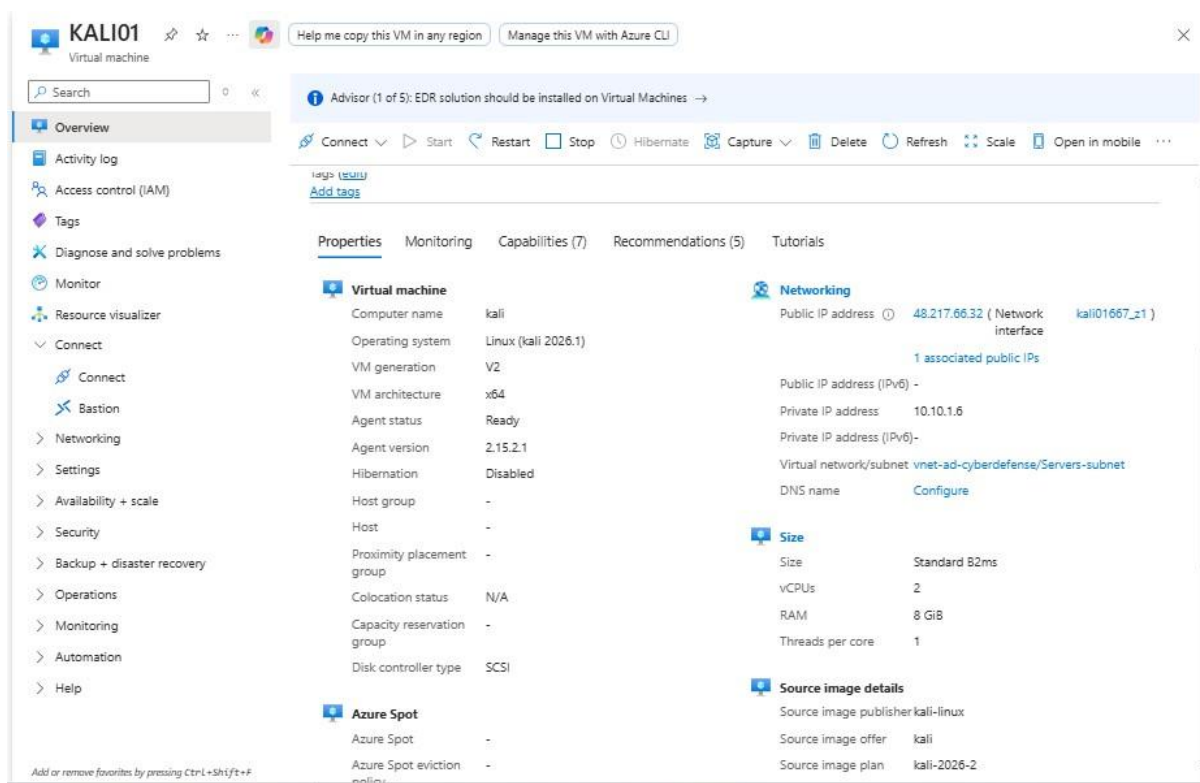


Figure 6.2 — KALI01 properties confirming the static private IP (10.10.1.6), VM size, and source image details (publisher kali-linux, plan kali-2026-2).



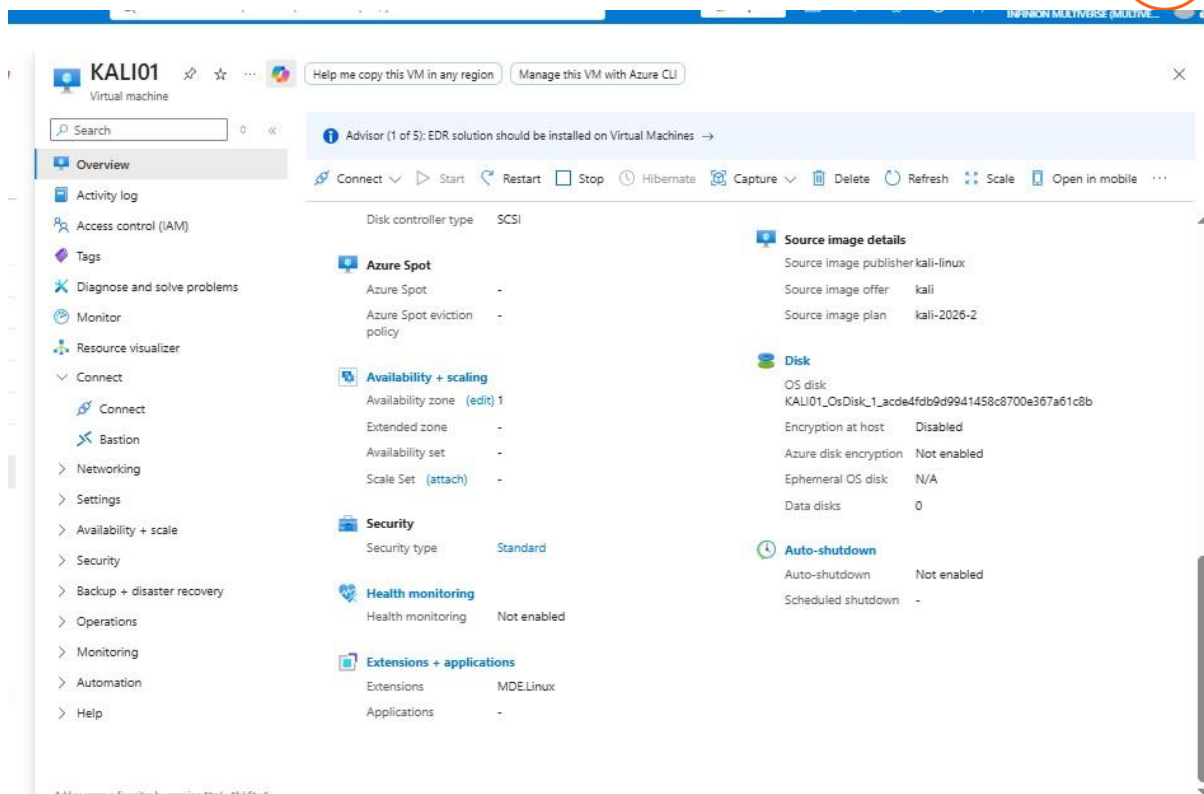


Figure 6.3 — Disk and security configuration for KALI01: Standard security type, OS disk details, and auto-shutdown not enabled.



```

kaliuser@kali: ~
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Windows\system32> Test-NetConnection 48.217.66.32 -Port 22

ComputerName      : 48.217.66.32
RemoteAddress     : 48.217.66.32
RemotePort        : 22
InterfaceAlias    : Wi-Fi
SourceAddress     : 192.168.88.108
TcpTestSucceeded  : True

PS C:\Windows\system32> ssh -i "C:\Users\chiamaka\Downloads\KALI01-key.pem" kaliuser@48.217.66.32
The authenticity of host '48.217.66.32 (48.217.66.32)' can't be established.
ED25519 key fingerprint is SHA256:ZZd1gcU+u0ApeYuF/JiRuMg//1bCzRkoF17RwdMCfsA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '48.217.66.32' (ED25519) to the list of known hosts.
linux kali 6.18.12+kali-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
(Message from kali developers)

This is a minimal installation of Kali Linux, you likely
want to install supplementary tools. Learn how:
@ https://www.kali.org/docs/troubleshooting/common-minimum-setup/

This is a cloud installation of Kali Linux. Learn more about
the specificities of the various cloud images:
@ https://www.kali.org/docs/troubleshooting/common-cloud-setup/

(Run: "touch ~/.hushlogin" to hide this message)
kaliuser@kali: ~$

```

Figure 6.4 — From a local Windows machine: Test-NetConnection confirms port 22 is reachable (JIT window open), followed by a successful SSH login to KALI01 using the downloaded private key.

```

kaliuser@kali: ~$ ping -c 4 10.10.1.4
PING 10.10.1.4 (10.10.1.4) 56(84) bytes of data:
64 bytes from 10.10.1.4: icmp_seq=1 ttl=128 time=2.91 ms
64 bytes from 10.10.1.4: icmp_seq=2 ttl=128 time=2.24 ms
64 bytes from 10.10.1.4: icmp_seq=3 ttl=128 time=1.46 ms
64 bytes from 10.10.1.4: icmp_seq=4 ttl=128 time=1.65 ms

--- 10.10.1.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.458/2.064/2.910/0.567 ms
kaliuser@kali: ~$

```

Figure 6.5 — From KALI01, pinging DC01's private IP (10.10.1.4) — 0% packet loss, confirming reachability to the domain controller.



```

[~] (kali@kali:~)~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 7c:1e:52:0a:ec:f1 brd ff:ff:ff:ff:ff:ff
    inet 10.10.1.6/24 metric 100 brd 10.10.1.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::7c1e:520a:ecf1:1/64 scope link proto kernel_l1
        valid_lft forever preferred_lft forever

[~] (kali@kali:~)~$ ip route
default via 10.10.1.1 dev eth0 proto dhcp src 10.10.1.6 metric 100
10.10.1.0/24 dev eth0 proto kernel scope link src 10.10.1.6 metric 100
10.10.1.1 dev eth0 proto dhcp scope link src 10.10.1.6 metric 100
168.63.129.16 via 10.10.1.1 dev eth0 proto dhcp src 10.10.1.6 metric 100
169.254.169.254 via 10.10.1.1 dev eth0 proto dhcp src 10.10.1.6 metric 100

[~] (kali@kali:~)~$ cat /etc/resolv.conf
# This is /run/systemd/resolved/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search .

```

Figure 6.6 — Verifying KALI01's own network configuration: ip addr confirms 10.10.1.6 on eth0, ip route shows the default route, and /etc/resolv.conf is inspected for DNS settings.

```

[~] (kali@kali:~)~$ resolvectl status
Global
    DNSStubResolver: +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
    ResolvConfMode: stub

Link 2 (eth0)
    CurrentScope: DNS LLMNR/IPv4 LLMNR/IPv6
    Protocol: +DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
    CurrentDNSDomain: 168.63.129.16
    DNSDomain: 168.63.129.16
    DNSStubResolver: yes

```

Figure 6.7 — resolvectl status confirming DNS resolver configuration on eth0 (Azure's default DNS server at this point).



To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

```
kaliadmin@KALI01:~$ hostname
KALI01
kaliadmin@KALI01:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 60:45:bd:ef:c8:bc brd ff:ff:ff:ff:ff:ff
    inet 10.10.1.6/24 metric 100 brd 10.10.1.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::6245:bdfc:feef:c8bc/64 scope link
        valid_lft forever preferred_lft forever
kaliadmin@KALI01:~$
```

Figure 6.8 — A fresh SSH session as kaliadmin, confirming hostname KALI01 and the static IP 10.10.1.6 via ip addr.

```
kaliadmin@KALI01:~$ ping -c 4 10.10.1.5
PING 10.10.1.5 (10.10.1.5) 56(84) bytes of data.
64 bytes from 10.10.1.5: icmp_seq=1 ttl=128 time=1.64 ms
64 bytes from 10.10.1.5: icmp_seq=2 ttl=128 time=1.65 ms
64 bytes from 10.10.1.5: icmp_seq=3 ttl=128 time=1.19 ms
64 bytes from 10.10.1.5: icmp_seq=4 ttl=128 time=1.37 ms

--- 10.10.1.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.191/1.460/1.645/0.191 ms
kaliadmin@KALI01:~$
```

Figure 6.9 — Pinging CLIENT01's private IP (10.10.1.5) from KALI01 — 0% packet loss, confirming reachability to the workstation as well.

```
rtt min/avg/max/mdev = 1.191/1.460/1.645/0.191 ms
kaliadmin@KALI01:~$ sudo apt update
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:6 http://azure.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:8 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:9 http://azure.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:10 http://azure.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:11 http://azure.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:12 http://azure.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:13 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1187 kB]
Get:14 http://azure.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [281 kB]
Get:15 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]
Get:16 http://azure.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1682 kB]
Get:17 http://azure.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [335 kB]
Get:18 http://azure.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]
Get:19 http://azure.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [34.9 kB]
Get:20 http://azure.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [1416 kB]
Get:21 http://azure.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [321 kB]
Get:22 http://azure.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [45.4 kB]
Get:23 http://azure.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [12.3 kB]
```

Figure 6.10 — Running sudo apt update on KALI01, the first step before installing the attack toolkit.



```

B]
Get:52 http://azure.archive.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata
468 B]
Fetched 36.4 MB in 5s (6763 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
10 packages can be upgraded. Run 'apt list --upgradable' to see them.

```

Figure 6.11 — apt update completing — package lists refreshed, 10 packages flagged as upgradable.

```

kaliadmin@KALI01: $ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
  libnss-systemd libpam-systemd libsystemd-shared libsystemd0 libudev1 systemd systemd-dev
  systemd-resolved systemd-sysv udev
10 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
10 standard LTS security updates
Need to get 8847 kB of archives.
After this operation, 17.4 kB of additional disk space will be used.
Get:1 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 libnss-systemd amd64 255
.4-1ubuntu8.17 [159 kB]
Get:2 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-dev all 255.4-1u
buntu8.17 [107 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-resolved amd64 2
55.4-1ubuntu8.17 [297 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 libsystemd-shared amd64
255.4-1ubuntu8.17 [2077 kB]
Get:5 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 libsystemd0 amd64 255.4-

```

Figure 6.12 — sudo apt upgrade -y applying the available system updates.

```

update-initramfs: Generating /boot/initrd.img-6.17.0-1022-azure
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...
systemctl restart multipathd.service packagekit.service polkit.service rsyslog.service ssh.se
rvice udisks2.service walinuxagent.service

Service restarts being deferred:
systemctl restart ModemManager.service
/etc/needrestart/restart.d/dbus.service
systemctl restart networkd-dispatcher.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service

No containers need to be restarted.

User sessions running outdated binaries:
kaliadmin @ session #2: apt[2146], sshd[1616]
kaliadmin @ user manager service: systemd[1621]

```

Figure 6.13 — Upgrade completing — initramfs rebuilt and background services restarted as part of the update.



```

kaliadmin@KALI01:~$ sudo apt install -y \
> nmap \
> git \
> python3-pip \
> netcat-openbsd
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
netcat-openbsd is already the newest version (1.226-1ubuntu2).
netcat-openbsd set to manually installed.
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu build-essential bzip2 cpp cpp-13
  cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu dpkg-dev fakeroot fontconfig-config
  fonts-dejavu-core fonts-dejavu-mono g++ g++-13 g++-13-x86-64-linux-gnu
  g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu
  javascript-common libalgorithm-diff-perl libalgorithm-diff-xs-perl libalgorithm-merge-perl
  libao3 libasan8 libatomic1 libbinutils libblas3 libc-dev-bin libc-devtools libc6-dev
  libcc1-0 libcrypt-dev libctf-nobfd0 libctf0 libdeflate0 libdpkg-perl libexpat1-dev
  libfakeroot libfile-fcntllock-perl libfontconfig1 libgcc-13-dev libgd3 libgomp1

```

Figure 6.14 — Installing the base toolset: nmap, git, python3-pip, and netcat-openbsd.

```

Setting up libc-devtools (2.39-0ubuntu8.8) ...
Processing triggers for libc-bin (2.39-0ubuntu8.8) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...

Service restarts being deferred:
  /etc/needrestart/restart.d/dbus.service
systemctl restart networkd-dispatcher.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service

No containers need to be restarted.

User sessions running outdated binaries:
kaliadmin @ session #2: sshd[1616]
kaliadmin @ user manager service: systemd[1621]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
kaliadmin@KALI01:~$

```

Figure 6.15 — Base toolset installation completing cleanly.



```
kaliadmin@KALI01:~$ nmap -Pn 10.10.1.4
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-11 14:23 UTC
Nmap scan report for dc01.internal.cloudapp.net (10.10.1.4)
Host is up (0.0020s latency).
Not shown: 988 filtered tcp ports (no-response)
PORT      STATE SERVICE
53/tcp    open  domain
88/tcp    open  kerberos-sec
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
389/tcp   open  ldap
445/tcp   open  microsoft-ds
464/tcp   open  kpasswd5
593/tcp   open  http-rpc-epmap
636/tcp   open  ldapssl
3268/tcp  open  globalcatLDAP
3269/tcp  open  globalcatLDAPssl
3389/tcp  open  ms-wbt-server

Nmap done: 1 IP address (1 host up) scanned in 4.24 seconds
kaliadmin@KALI01:~$
```

Figure 6.16 — Verifying Nmap by scanning DC01 (10.10.1.4) — correctly fingerprints it as a domain controller (Kerberos, LDAP, Global Catalog ports all open).

```
kaliadmin@KALI01:~$ sudo apt install -y \
> python3-pip \
> git \
> nmap \
> netcat-openbsd \
> ldap-utils \
> smbclient
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-pip is already the newest version (24.0+dfsg-1ubuntu1.3).
git is already the newest version (1:2.43.0-1ubuntu7.3).
nmap is already the newest version (7.94+git20230807.3be01efb1+dfsg-3build2).
netcat-openbsd is already the newest version (1.226-1ubuntu2).
The following additional packages will be installed:
  libavahi-client3 libavahi-common-data libavahi-common3 libcups2t64 libldb2 libsmbclient0
  libtdb1 libtevent0t64 python3-gpg python3-ldb python3-markdown python3-samba
  python3-talloc python3-tdb samba-common samba-common-bin samba-dsdb-modules samba-libs
Suggested packages:
  libsasl2-modules-gssapi-mit | libsasl2-modules-gssapi-heimdal cups-common
  python-markdown-doc heimdal-clients python3-dnspython
The following NEW packages will be installed:
  ldap-utils libavahi-client3 libavahi-common-data libavahi-common3 libcups2t64 libldb2
  libsmbclient0 libtdb1 libtevent0t64 python3-gpg python3-ldb python3-markdown python3-samba
  python3-talloc python3-tdb samba-common samba-common-bin samba-dsdb-modules samba-libs
  smbclient
0 upgraded, 20 newly installed, 0 to remove and 0 not upgraded.
```

Figure 6.17 — Installing the next set of tools: ldap-utils and smbclient, for manual LDAP enumeration and SMB access.



```

Setting up smbclient (2:4.19.5+dfsg-4ubuntu9.7) ...
Setting up samba-common-bin (2:4.19.5+dfsg-4ubuntu9.7) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.8) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...

Service restarts being deferred:
/etc/needrestart/restart.d/dbus.service
systemctl restart networkd-dispatcher.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service

No containers need to be restarted.

User sessions running outdated binaries:
kaliadmin @ session #2: sshd[1616]
kaliadmin @ user manager service: systemd[1621]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
kaliadmin@KALI01: ~$

```

Figure 6.18 — smbclient / samba-common-bin installation completing successfully.

```

No VM guests are running outdated hypervisor (qemu) binaries on this host.
kaliadmin@KALI01: ~$ python3 -m pip install impacket --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Collecting impacket
  Downloading impacket-0.13.1-py3-none-any.whl.metadata (6.0 kB)
Requirement already satisfied: pyasn1>=0.2.3 in /usr/lib/python3/dist-packages (from impacket) (0.4.8)
Requirement already satisfied: pyasn1_modules in /usr/lib/python3/dist-packages (from impacket) (0.2.8)
Collecting pycryptodomex (from impacket)
  Downloading pycryptodomex-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.4 kB)
Requirement already satisfied: pyOpenSSL in /usr/lib/python3/dist-packages (from impacket) (23.2.0)
Requirement already satisfied: six in /usr/lib/python3/dist-packages (from impacket) (1.16.0)
Collecting ldap3!=2.5.0,!2.5.2,!2.6,>=2.5 (from impacket)
  Downloading ldap3-2.9.1-py2.py3-none-any.whl.metadata (5.4 kB)
Collecting ldapdomaindump>=0.9.0 (from impacket)
  Downloading ldapdomaindump-0.10.0-py3-none-any.whl.metadata (512 bytes)
Collecting flask>=1.0 (from impacket)
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting charset_normalizer (from impacket)
  Downloading charset_normalizer-3.4.9-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (41 kB)
Collecting blinker>=1.0.0 (from flask>=1.0->impacket)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.5 kB)
41.7/41.7 kB 1.9 MB/s eta 0:00:00

```

Figure 6.19 — Installing Impacket via pip (--break-system-packages), pulling in its dependencies (ldap3, ldapdomaindump, etc.).



```

Downloading impacket-0.13.1-py3-none-any.whl (1.8 MB)
 1.8/1.8 MB 61.7 MB/s eta 0:00:00
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
103.4/103.4 kB 6.9 MB/s eta 0:00:00
Downloading ldap3-2.9.1-py2.py3-none-any.whl (432 kB)
432.2/432.2 kB 26.6 MB/s eta 0:00:00
Downloading ldapdomaindump-0.10.0-py3-none-any.whl (19 kB)
Downloading charset_normalizer-3.4.9-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.m
nylinux_2_28_x86_64.whl (224 kB)
224.3/224.3 kB 14.3 MB/s eta 0:00:00
Downloading pycryptodomex-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.3
MB)
2.3/2.3 MB 83.4 MB/s eta 0:00:00
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
226.5/226.5 kB 14.7 MB/s eta 0:00:00
Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)
331.1/331.1 kB 15.2 MB/s eta 0:00:00
Installing collected packages: werkzeug, pycryptodomex, ldap3, itsdangerous, dnspython, charse
t_normalizer, blinker, ldapdomaindump, flask, impacket
WARNING: The script normalizer is installed in '/home/kaliadmin/.local/bin' which is not on
PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-
warn-script-location.
WARNING: The scripts ldapdomaindump, ldd2bloodhound and ldd2pretty are installed in '/home/k
aliadmin/.local/bin' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-
warn-script-location.
WARNING: The script flask is installed in '/home/kaliadmin/.local/bin' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-
warn-script-location.
Successfully installed blinker-1.9.0 charset_normalizer-3.4.9 dnspython-2.8.0 flask-3.1.3 impa
cket-0.13.1 itsdangerous-2.2.0 ldap3-2.9.1 ldapdomaindump-0.10.0 pycryptodomex-3.23.0 werkzeug
-3.1.8

```

Figure 6.20 — Impacket pip installation finishing successfully.

```

kaliadmin@KALI01:~$ impacket-GetNPUsers -h
impacket-GetNPUsers: command not found
kaliadmin@KALI01:~$ ls ~/.local/bin | grep -i impacket
kaliadmin@KALI01:~$ ls ~/.local/bin
CheckLDAPStatus.py    dpapidump.py        mimikatz.py          rpcdump.py
DumpNTLMInfo.py       esentutl.py          mqtt_check.py        rpcmap.py
Get-GPPPassword.py    exchanger.py         mssqlclient.py       sambaPipe.py
GetADComputers.py     filetime.py          mssqlinstance.py     samedit.py
GetADUsers.py         findDelegation.py    net.py               samrdump.py
GetLAPSPassword.py    flask                netview.py           secretsdump.py
GetNPUsers.py         getArch.py           normalizer.py         services.py
GetUserSPNs.py        getPac.py            ntfs-read.py         smbclient.py
__pycache__           getST.py             ntlmrelayx.py        smbexec.py
addcomputer.py        getTGT.py            ownededit.py          smbserver.py
atexec.py             goldenPac.py          ping.py              sniff.py
attrib.py             karmaSMB.py           ping6.py              sniffer.py
badsuccessor.py       keylistattack.py     psexec.py            split.py
changepasswd.py       kintercept.py        raiseChild.py        ticketConverter.py
checkMSSQLStatus.py   ldapdomaindump       rbcd.py              ticketer.py
dacledit.py           ldd2bloodhound       rdp_check.py         tstool.py
dcomexec.py           ldd2pretty           reg.py               wmiexec.py
describeTicket.py     lookupsid.py         registry-read.py     wmipersist.py
dpapi.py              machine_role.py       regsecrets.py        wmiquery.py
kaliadmin@KALI01:~$
kaliadmin@KALI01:~$ /home/kaliadmin/.local/bin
-bash: /home/kaliadmin/.local/bin: Is a directory
kaliadmin@KALI01:~$

```

Figure 6.21 — Discovering a naming/PATH issue: impacket-GetNPUsers is not found, and ls ~/.local/bin shows the pip-installed scripts use the plain toolname.py convention instead.



```

Last login: Tue Aug 11 13:57:48 2026 from 41.86.149.6
kaliadmin@KALI01: $ echo 'export PATH=$PATH:$HOME/.local/bin' >> ~/.bashrc
kaliadmin@KALI01: $ source ~/.bashrc
kaliadmin@KALI01: $ echo $PATH
/home/kaliadmin/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/home/kaliadmin/.local/bin
kaliadmin@KALI01: $ GetNPUsers.py -h
Impacket v0.13.1 - Copyright Fortra, LLC and its affiliated companies

usage: GetNPUsers.py [-h] [-request] [-outputfile OUTPUTFILE] [-format {hashcat,john}]
                  [-usersfile USERSFILE] [-ts] [-debug] [-hashes LMHASH:NTHASH]
                  [-no-pass] [-k] [-aesKey hex key] [-dc-ip ip address]
                  target

Queries target domain for users with 'Do not require Kerberos preauthentication' set and
export their TGTs for cracking

positional arguments:
  target                [[domain/]username[:password]]

options:
  -h, --help            show this help message and exit
  -request              Requests TGT for users and output them in JtR/hashcat format
                        (default False)
  -outputfile OUTPUTFILE
                        Output filename to write ciphers in JtR/hashcat format
  -format {hashcat,john}
                        format to save the AS_REQ of users without pre-authentication.
                        Default is hashcat
  -usersfile USERSFILE  File with user per line to test
  -ts                  Adds timestamp to every logging output
  -debug               Turn DEBUG output ON

authentication:
  -hashes LMHASH:NTHASH
                        NTLM hashes, format is LMHASH:NTHASH
  -no-pass              don't ask for password (useful for -k)
  -k                   Use Kerberos authentication. Grabs credentials from ccache file
                        (KRB5CCNAME) based on target parameters. If valid credentials cannot
                        be found, it will use the ones specified in the command line
  -aesKey hex key      AES key to use for Kerberos Authentication (128 or 256 bits)

connection:
  -dc-ip ip address     IP Address of the domain controller. If omitted it use the domain
                        part (FQDN) specified in the target parameter

```

Figure 6.22 — Adding ~/.local/bin to PATH and re-sourcing .bashrc; GetNPUsers.py -h now runs directly, confirming Impacket v0.13.1 is functional via this route.



```

(kaliuser@ kali)~$
$ impacket-GetUserSPNs --help
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

usage: GetUserSPNs.py [-h] [-target-domain TARGET_DOMAIN] [-no-preauth NO_PREAUTH] [-stealth] [-machine-only]
                    [-usersfile USERSFILE] [-request] [-request-user username] [-request-machine machinename] [-save]
                    [-outputfile OUTPUTFILE] [-ts] [-debug] [-hashes LMHASH:NTHASH] [-no-pass] [-k] [-aesKey hex key]
                    [-dc-ip ip address] [-dc-host hostname]
                    target

Queries target domain for SPNs that are running under a user account

positional arguments:
  target                domain[/username[:password]]

options:
  -h, --help            show this help message and exit
  -target-domain TARGET_DOMAIN
                        Domain to query/request if different than the domain of the user. Allows for Kerberoasting across
                        trusts.
  -no-preauth NO_PREAUTH
                        account that does not require preauth, to obtain Service Ticket through the AS
  -stealth              Removes the (servicePrincipalName=*) filter from the LDAP query for added stealth. May cause huge
                        memory consumption / errors on large domains.
  -machine-only         Queries for machine accounts only, by adjusting 'objectCategory=person' to 'objectCategory=computer'.
                        Active Directory may limit results to 1,000 objects by default. LDAP paging is required to retrieve
                        more.
  -usersfile USERSFILE  File with user per line to test
  -request              Requests TGS for users and output them in JtR/hashcat format (default False)
  -request-user username
                        Requests TGS for the SPN associated to the user specified (just the username, no domain needed)
  -request-machine machinename
                        Requests TGS for the SPN associated to the machine specified. Example: 'workstation01$'
  -save                 Saves TGS requested to disk. Format is <username>.ccache. Auto selects -request
  -outputfile OUTPUTFILE
                        Output filename to write ciphers in JtR/hashcat format. Auto selects -request
  -ts                  Adds timestamp to every logging output.
  -debug               Turn DEBUG output ON

authentication:
  -hashes LMHASH:NTHASH
                        NTLM hashes, format is LMHASH:NTHASH
  -no-pass              don't ask for password (useful for -k)
  -k                   Use Kerberos authentication. Grabs credentials from ccache file (KRB5CCNAME) based on target

```

Figure 6.23 — A separate, system-wide install exposes the `impacket-<toolname>` convention instead: `impacket-GetUserSPNs --help` confirms Impacket v0.14.0.dev0, matching the version used later in Phase 9.

```

more.
  -usersfile USERSFILE  File with user per line to test
  -request              Requests TGS for users and output them in JtR/hashcat format (default False)
  -request-user username
                        Requests TGS for the SPN associated to the user specified (just the username, no domain needed)
  -request-machine machinename
                        Requests TGS for the SPN associated to the machine specified. Example: 'workstation01$'
  -save                 Saves TGS requested to disk. Format is <username>.ccache. Auto selects -request
  -outputfile OUTPUTFILE
                        Output filename to write ciphers in JtR/hashcat format. Auto selects -request
  -ts                  Adds timestamp to every logging output.
  -debug               Turn DEBUG output ON

authentication:
  -hashes LMHASH:NTHASH
                        NTLM hashes, format is LMHASH:NTHASH
  -no-pass              don't ask for password (useful for -k)
  -k                   Use Kerberos authentication. Grabs credentials from ccache file (KRB5CCNAME) based on target
                        parameters. If valid credentials cannot be found, it will use the ones specified in the command line
  -aesKey hex key      AES key to use for Kerberos Authentication (128 or 256 bits)

connection:
  -dc-ip ip address     IP Address of the domain controller. If omitted it use the domain part (FQDN) specified in the target
                        parameter. Ignored if -target-domain is specified.
  -dc-host hostname     Hostname of the domain controller to use. If omitted, the domain part (FQDN) specified in the account
                        parameter will be used

(kaliuser@ kali)~$
$ which impacket-secretsdump
/usr/bin/impacket-secretsdump

(kaliuser@ kali)~$
$

```

Figure 6.24 — Confirming `impacket-secretsdump` resolves to `/usr/bin/impacket-secretsdump` — the `impacket-*` naming convention used on this Kali release, resolving the earlier naming discrepancy via `dpkg -L` inspection.



```
(kaliuser@ kali)-[~]  
$ responder --version
```

Responder 3.2.2.0

```
[~](kaliuser@ kali)~$ python --help
usage: bloodhound-python [-h] [-c COLLECTIONMETHOD] [-d DOMAIN] [-v] [-u USERNAME] [-p PASSWORD] [-k] [--hashes HASHES]
                        [--no-pass] [--aesKey hex key] [--auth-method {auto,ntlm,kerberos}] [--ns NAMESERVER] [--dns-tcp]
                        [--dns-timeout DNS_TIMEOUT] [-dc HOST] [-gc HOST] [-w WORKERS] [--exclude-dcs] [--disable-pooling]
                        [--disable-autogc] [--zip] [--computerfile COMPUTERFILE] [--cachefile CACHEFILE]
                        [--ldap-channel-binding] [--use-ldaps] [-op PREFIX_NAME]

Python based ingestor for BloodHound LEGACY
For help or reporting issues, visit https://github.com/dirkjanm/BloodHound.py

options:
  -h, --help            show this help message and exit
  -c, --collectionmethod COLLECTIONMETHOD
                        Which information to collect. Supported: Group, LocalAdmin, Session, Trusts, Default (all previous),
                        DCOMOnly (no computer connections), DCOM, RDP,PSRemote, LoggedOn, Container, ObjectProps, ACL, All (all
                        except LoggedOn). You can specify more than one by separating them with a comma. (default: Default)
  -d, --domain DOMAIN   Domain to query.
  -v                    Enable verbose output

authentication options:
Specify one or more authentication options.
By default Kerberos authentication is used and NTLM is used as fallback.
Kerberos tickets are automatically requested if a password or hashes are specified.

-u, --username USERNAME Username. Format: username[@domain]; If the domain is unspecified, the current domain is used.
-p, --password PASSWORD Password
-k, --kerberos          Use kerberos ccache file
--hashes HASHES        LM:NLTm hashes
--no-pass              don't ask for password (useful for -k)
--aesKey hex key       AES key to use for Kerberos Authentication (128 or 256 bits)
--auth-method {auto,ntlm,kerberos}
                        Authentication methods. Force Kerberos or NTLM only or use auto for Kerberos with NTLM fallback

collection options:
-n, --nameserver NAMESERVER Alternative name server to use for queries
--dns-tcp               Use TCP instead of UDP for DNS queries
--dns-timeout DNS_TIMEOUT DNS query timeout in seconds (default: 3)
-dc, --domain-controller HOST Override which DC to query (hostname)
-gc, --global-catalog HOST Override which GC to query (hostname)
```

```

-aesKey hex key      AES key to use for Kerberos Authentication (128 or 256 bits)
--auth-method {auto,ntlm,kerberos}
                        Authentication methods. Force Kerberos or NTLM only or use auto for Kerberos with NTLM fallback

collection options:
  -ns, --nameserver NAMESERVER
                        Alternative name server to use for queries
  --dns-tcp            Use TCP instead of UDP for DNS queries
  --dns-timeout DNS_TIMEOUT
                        DNS query timeout in seconds (default: 3)
  -dc, --domain-controller HOST
                        Override which DC to query (hostname)
  -gc, --global-catalog HOST
                        Override which GC to query (hostname)
  -w, --workers WORKERS
                        Number of workers for computer enumeration (default: 10)
  --exclude-dcs        Skip DCs during computer enumeration
  --disable-pooling     Don't use subprocesses for ACL parsing (only for debugging purposes)
  --disable-autogc      Don't automatically select a Global Catalog (use only if it gives errors)
  --zip                Compress the JSON output files into a zip archive
  --computerfile COMPUTERFILE
                        File containing computer FQDNs to use as allowlist for any computer based methods
  --cachefile CACHEFILE
                        Cache file (experimental)
  --ldap-channel-binding
                        Use LDAP Channel Binding (will force ldaps protocol to be used)
  --use-ldaps          Use LDAP over TLS on port 636 by default
  -op, --outputprefix PREFIX_NAME
                        String to prepend to output file names

```

Figure 6.27 — Continued BloodHound help output, confirming full functionality.

```

(kaliuser@ kali)-[~]
$ ldapsearch -V
ldapsearch: @(#) $OpenLDAP: ldapsearch 2.6.10+dfsg-1+b2 (Apr 29 2026 10:11:22) $
        Debian OpenLDAP Maintainers <pkg-openldap-devel@lists.alioth.debian.org>
        (LDAP library: OpenLDAP 20610)
ldap_sasl_interactive_bind: Can't contact LDAP server (-1)

```

Figure 6.28 — Verifying ldapsearch -V — confirms OpenLDAP 2.6.10 for manual LDAP-based AD enumeration.

```

dquote bquote>
dquote bquote>
dquote bquote>

(kaliuser@ kali)-[~]
$ smbclient --version
Version 4.24.3-Debian-4.24.3+dfsg-1

(kaliuser@ kali)-[~]
$

```

Figure 6.29 — Verifying smbclient --version — confirms 4.24.3-Debian, completing the toolkit verification pass.

Microsoft Security Integration

1. Prerequisite Verification — Defender for Endpoint (MDE) Coverage

Before deploying the Defender for Identity (MDI) sensor, it was confirmed that MDI sensor v3.x requires MDE to already be active on the target domain controller, as the sensor now runs as an extension of the MDE/SENSE service rather than as a standalone installer.

1. DC01 was found to already be auto-onboarded to MDE. This was investigated and confirmed to be expected behaviour: the lab's Azure subscription has the Defender for Servers plan enabled (the same plan powering JIT VM Access), which automatically deploys the MDE agent to any qualifying server OS VM in the subscription — no manual onboarding action was required for DC01.
2. CLIENT01, being a workstation OS (Windows 10), was not covered by this automatic server-tier provisioning and required manual onboarding.



3. KALI01 was deliberately excluded from MDE onboarding, as it serves as the attacker machine and is intentionally kept outside the monitored/defended perimeter — consistent with realistic red team lab design, and outside Microsoft's supported Linux distribution list for MDE in any case.

2. Manual MDE Onboarding — CLIENT01

CLIENT01 was onboarded to Microsoft Defender for Endpoint using the local script deployment method:

1. Onboarding package downloaded from Microsoft Defender portal → Settings → Endpoints → Onboarding
2. Script transferred to and executed locally on CLIENT01 via an elevated Command Prompt
3. Onboarding was confirmed successful in Assets → Devices, with CLIENT01 (client01.lab.local) showing Onboarding status: Onboarded and Sensor health state: Active

3. Defender for Identity Sensor Deployment — DC01

With MDE confirmed active on DC01, the modern MDI sensor v3.x was activated directly from the portal (no installer download or manual access key required, reflecting Microsoft's updated deployment model):

1. Microsoft Defender portal → Settings → Identities → Activation
2. DC01 selected from the list of eligible servers and activated
3. Sensor health was monitored post-activation; initial state showed Onboarding / Unreachable (expected immediately after activation)
4. After a short propagation period, sensor status stabilized to Running / Up to date / Healthy, confirming successful MDI deployment on the domain controller

4. Dedicated Sentinel Workspace Creation

A dedicated Log Analytics workspace and Sentinel instance were created specifically for this capstone project, isolated from other unrelated project workspaces:

1. Log Analytics workspace created within AD_Cyberattack_Detection_Group, region East US
2. Microsoft Sentinel enabled on top of that workspace
3. Workspace named to reflect the lab's purpose (Cyberattack-Detection-Log-Analytics-workspace)

5. Defender XDR ↔ Sentinel Integration

The new Sentinel workspace was connected to Microsoft Defender XDR using the current unified security operations model (Microsoft's now-standard approach, replacing the legacy standalone data connector method):

1. Microsoft Defender portal → Settings → Microsoft Sentinel → Connect a workspace
2. The lab's Sentinel workspace selected and connected as the Primary workspace
3. Connection confirmed via workspace status showing Connected / Primary

During validation, it was noted that the Defender Incidents view is tenant-wide by default and surfaces alerts from all connected workspaces (including unrelated prior project work) — this distinction was documented to ensure lab-specific validation used direct KQL queries against the dedicated workspace rather than the shared incidents queue.

6. Log Ingestion Validation



Initial validation queries against the newly created workspace returned no results, which was investigated and correctly attributed to the workspace being newly connected with no historical backfill (each Sentinel workspace only ingests data generated after its connection is active — it does not inherit history from other workspaces).

To confirm ingestion, controlled activity was generated:

1. An RDP session was initiated into DC01 via Just-in-Time (JIT) VM Access, with basic login/logout activity performed
2. After an approximately 10–15 minute propagation window, KQL queries were re-run against the lab's dedicated Sentinel workspace
3. Log ingestion was successfully confirmed, with relevant identity and device telemetry appearing in the workspace as expected

Outcome: The Microsoft security integration for this lab is fully operational: DC01 is monitored by an active, healthy MDI sensor; both DC01 and CLIENT01 are onboarded to MDE; the dedicated Sentinel workspace is connected as the primary workspace in Defender XDR's unified security operations model; and log ingestion from the lab environment has been validated end-to-end. This satisfies Phase 7's expected outcomes: MDI, Sentinel, and Defender XDR fully integrated and functional.

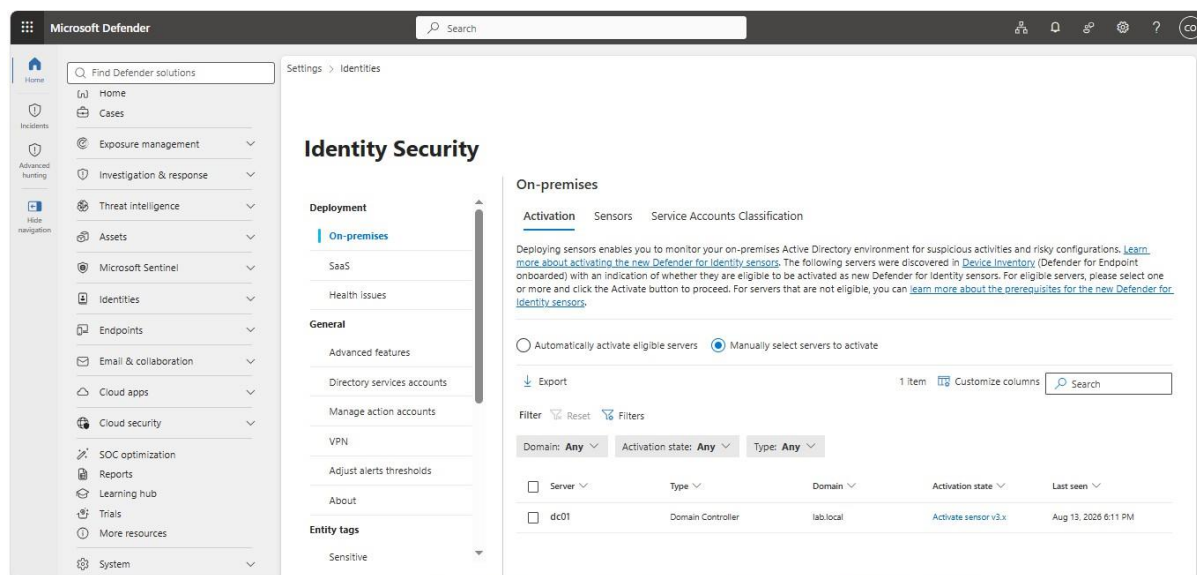


Figure 7.1 — Microsoft Defender portal → Settings → Identities → Activation, showing DC01 discovered as an eligible domain controller (via its MDE onboarding) ready to activate the modern MDI sensor v3.x.



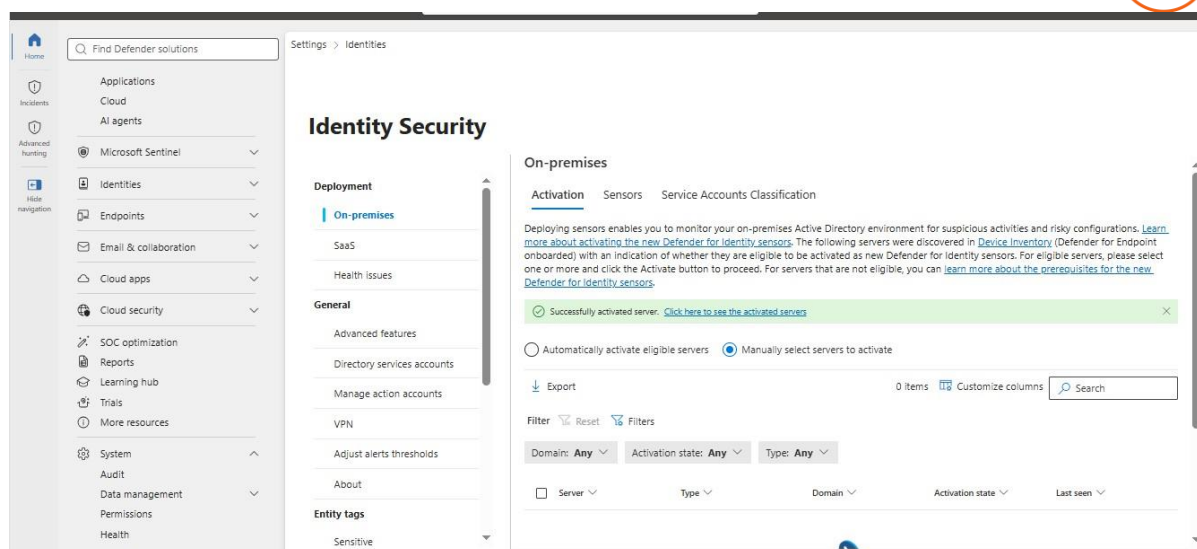


Figure 7.2 — “Successfully activated server” confirmation immediately after activating the MDI sensor on DC01.

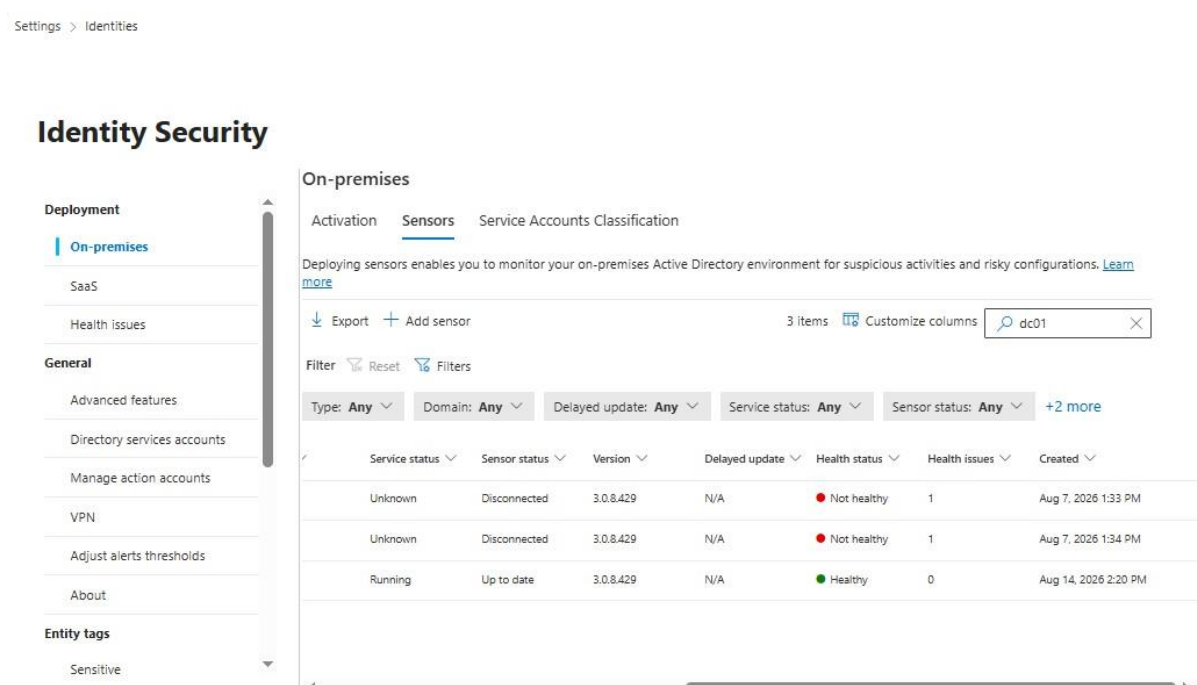


Figure 7.3 — Sensors tab showing DC01's sensor history: two earlier disconnected/unhealthy entries from a prior rebuild, and the current sensor already Running / Up to date / Healthy.

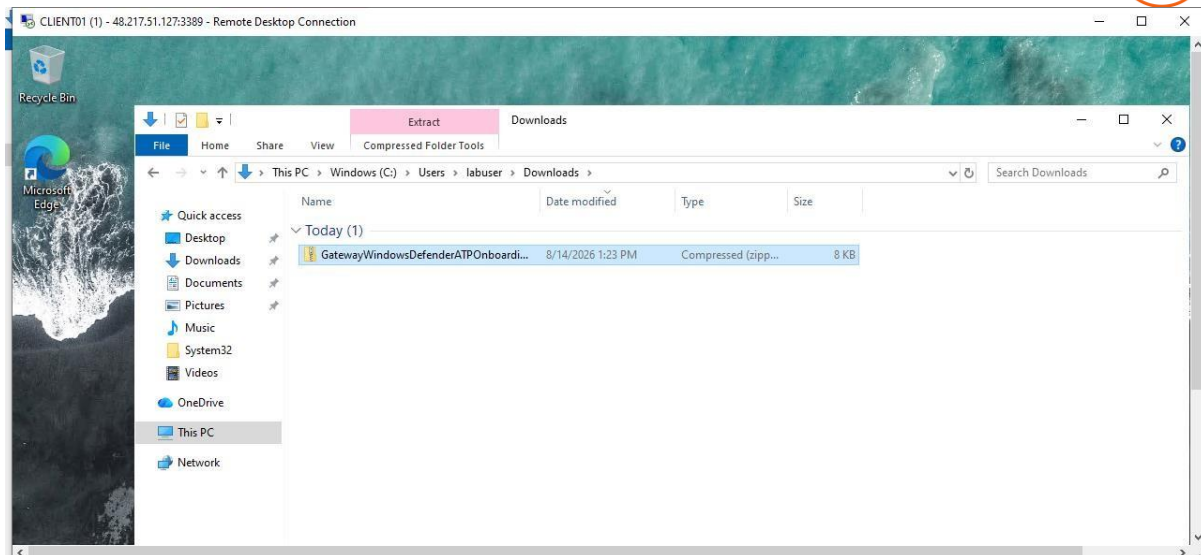


Figure 7.4 — The MDE onboarding package downloaded from the Defender portal, sitting in the Downloads folder on CLIENT01.

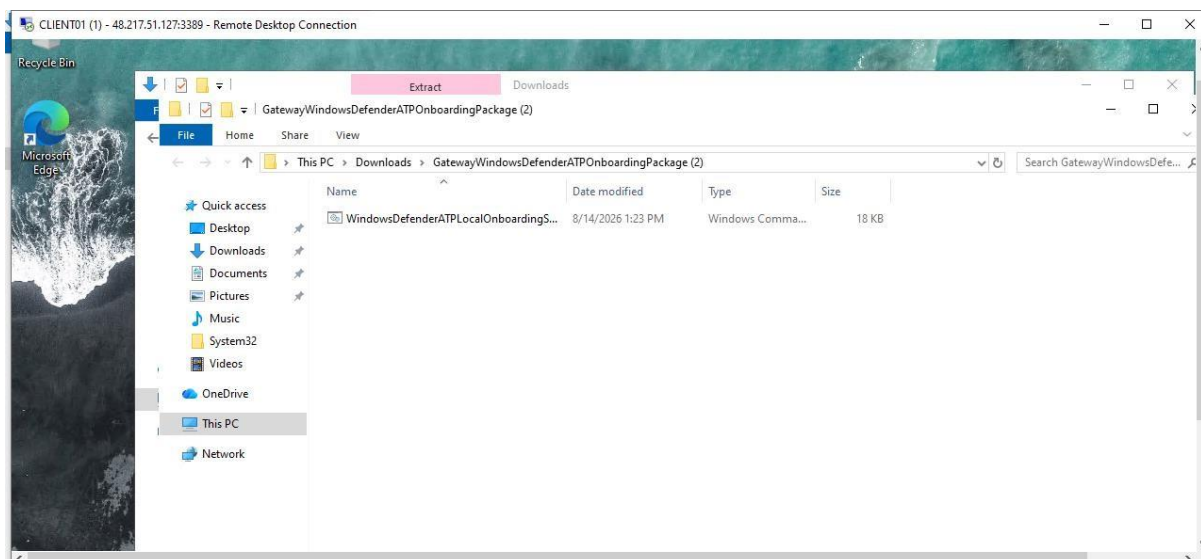


Figure 7.5 — The onboarding package extracted, revealing WindowsDefenderATPLocalOnboardingScript.cmd.



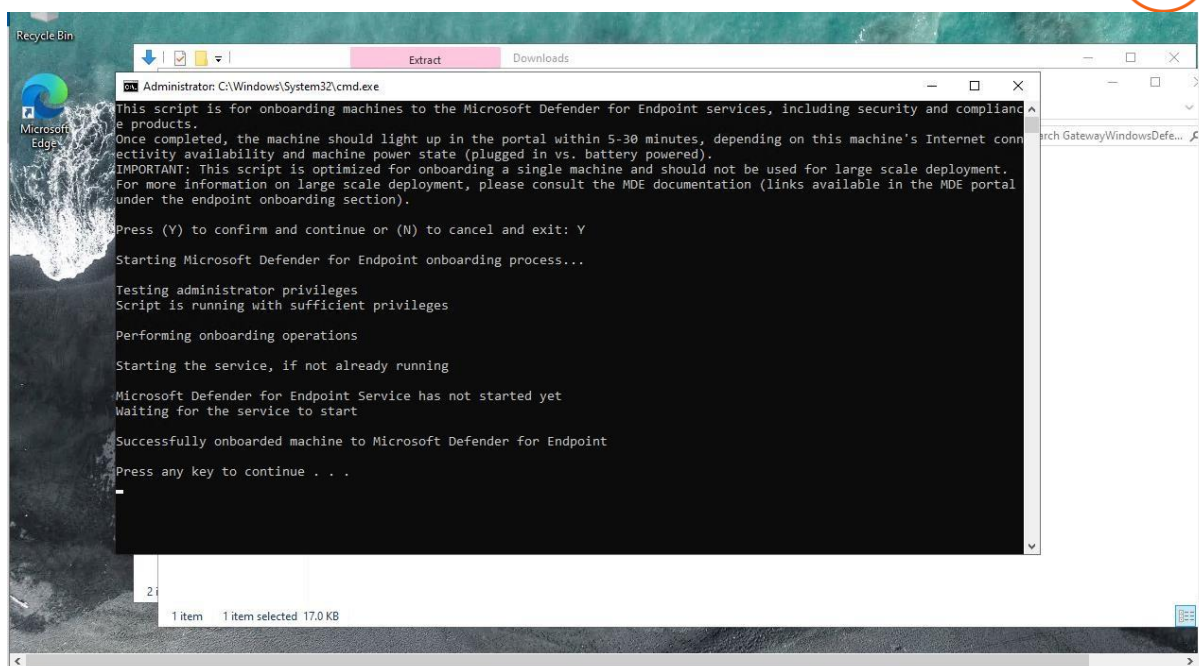


Figure 7.6 — Running the onboarding script in an elevated Command Prompt — “Successfully onboarded machine to Microsoft Defender for Endpoint.”

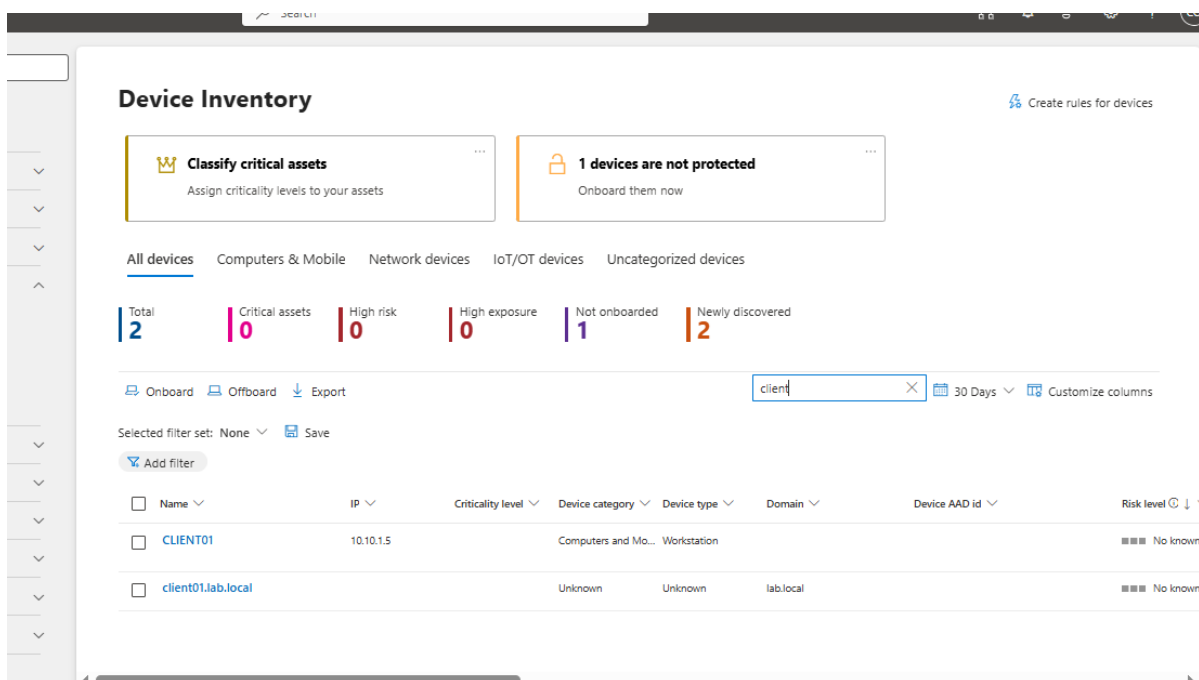


Figure 7.7 — Assets → Devices, filtered to “client” — CLIENT01 and client01.lab.local both discovered in Device Inventory following onboarding.



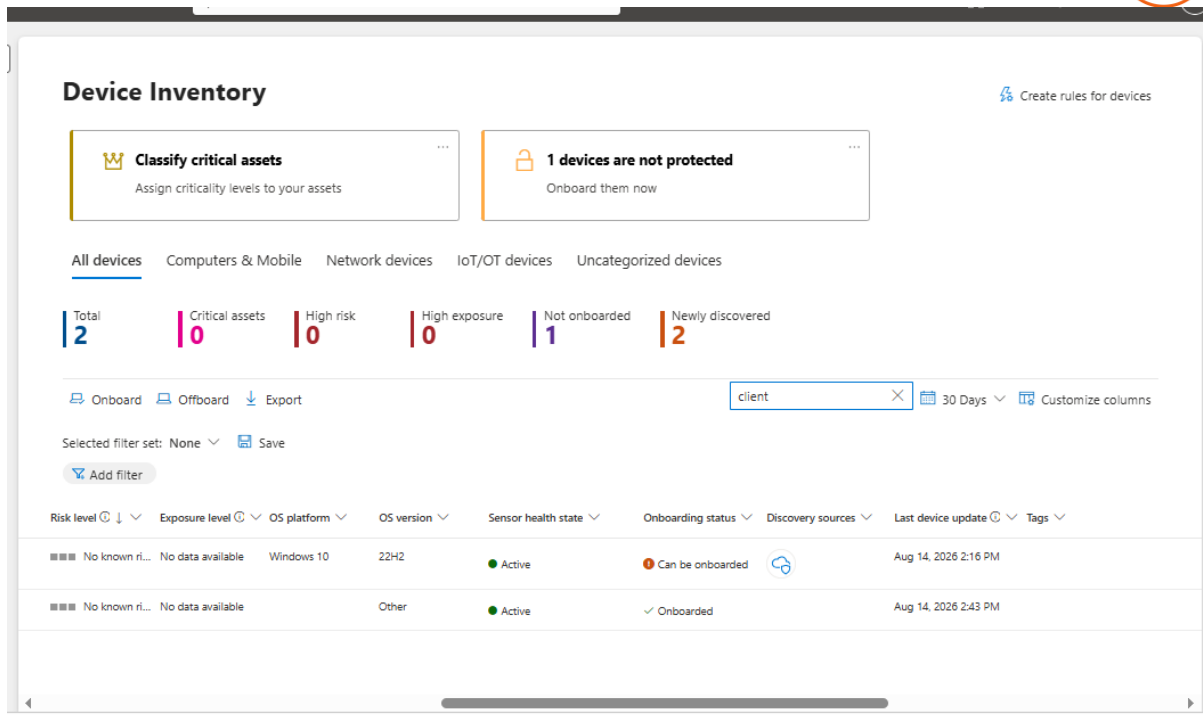


Figure 7.8 — Scrolled view of the same list: the client01.lab.local record shows Sensor health state: Active and Onboarding status: Onboarded, confirming the onboarding succeeded.

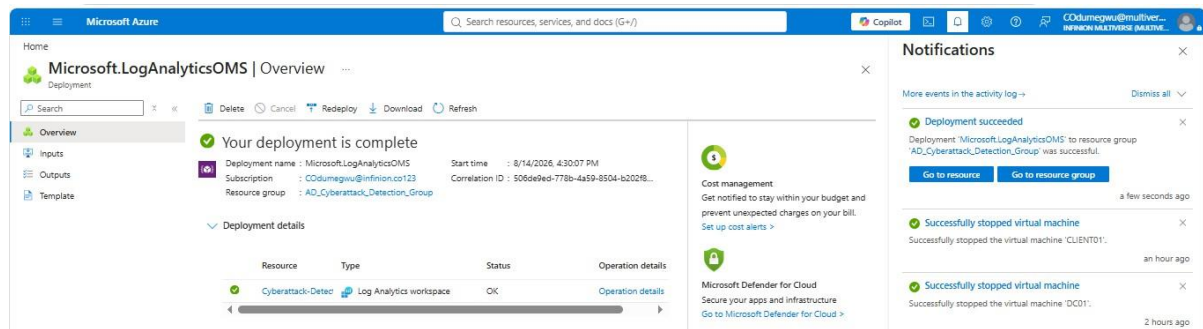


Figure 7.9 — Deployment confirmation for the new Log Analytics workspace in AD_Cyberattack_Detection_Group — the foundation for the dedicated Sentinel instance.

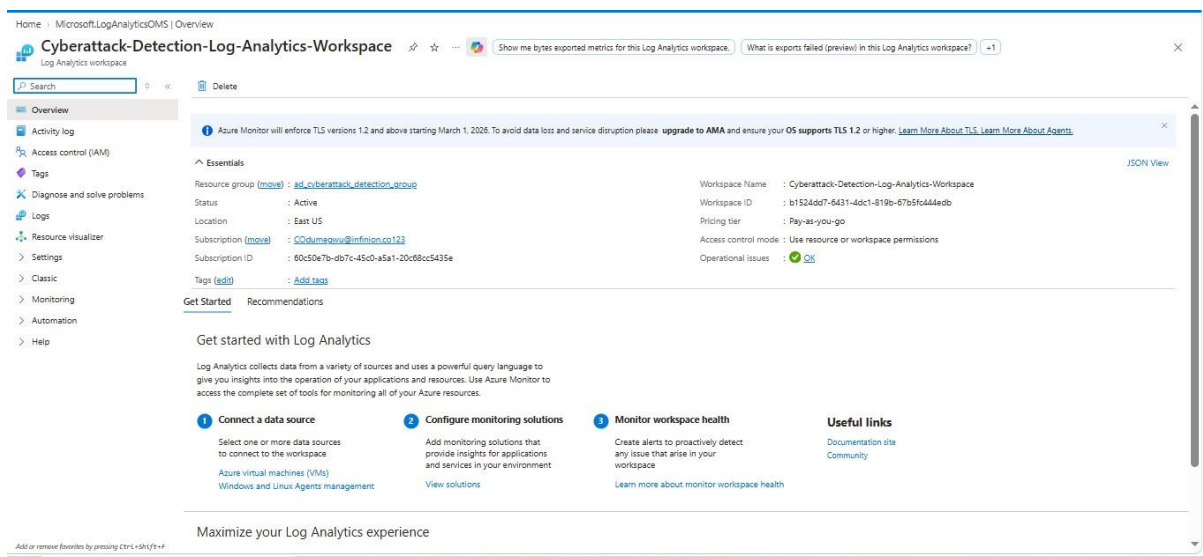


Figure 7.10 — The Cyberattack-Detection-Log-Analytics-Workspace overview — Active status, East US, isolated inside AD_Cyberattack_Detection_Group.

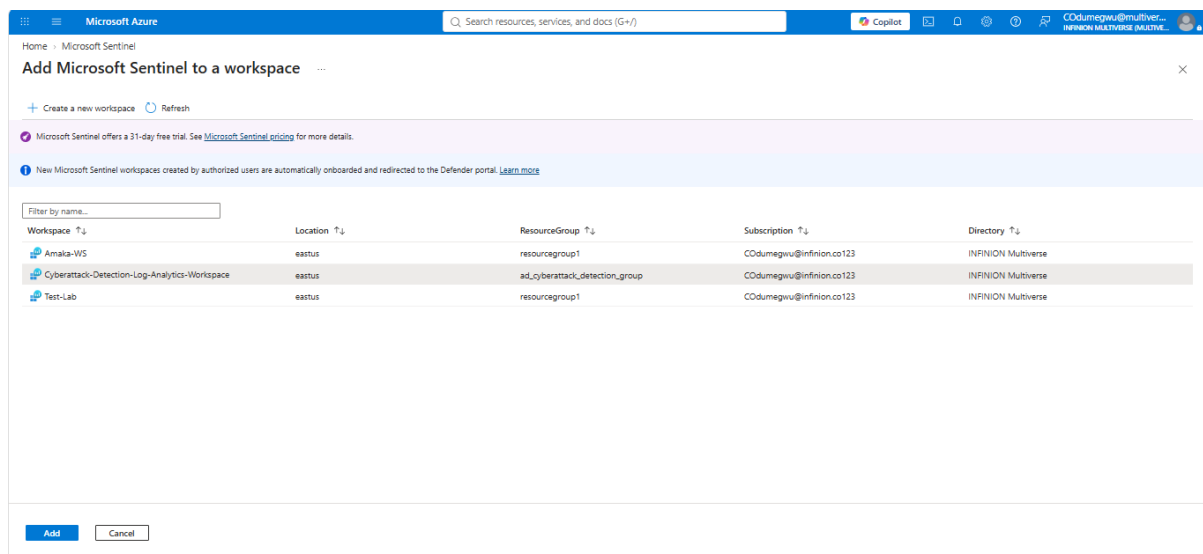


Figure 7.11 — “Add Microsoft Sentinel to a workspace” — selecting the lab's dedicated workspace, distinct from other unrelated workspaces (Amaka-WS, Test-Lab).

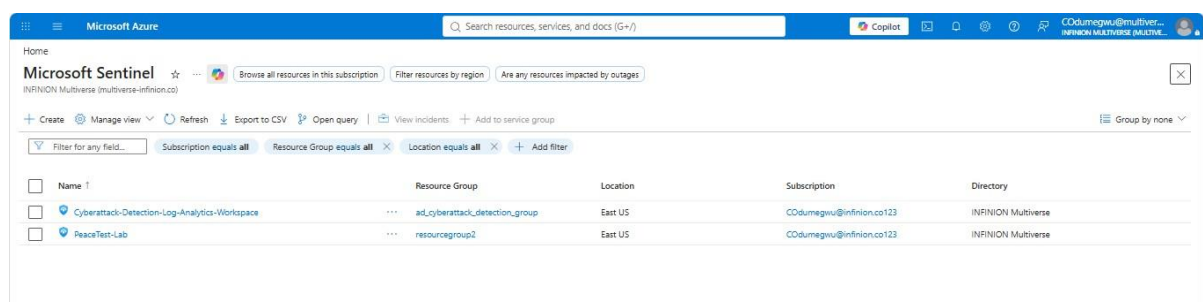


Figure 7.12 — Microsoft Sentinel resource list confirming the workspace is now enabled as its own Sentinel instance.

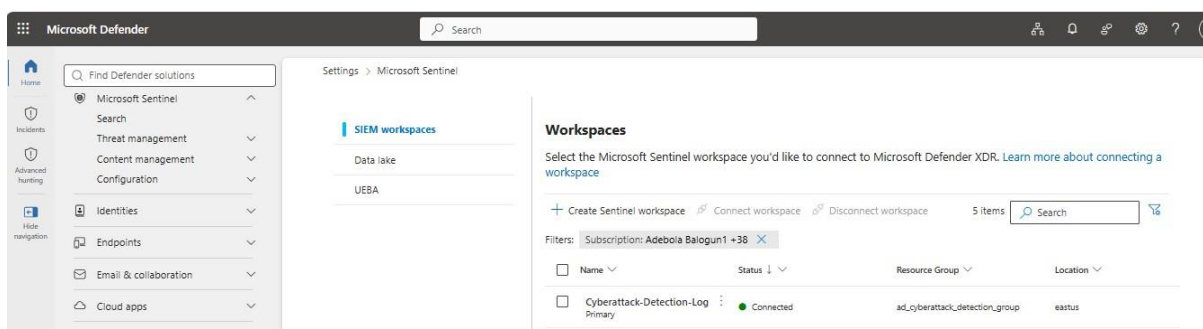


Figure 7.13 — Defender portal → Settings → Microsoft Sentinel → Workspaces: the lab's workspace shows Status: Connected and is set as the Primary workspace in Defender XDR's unified security operations model.

Playbook Development

1. Playbook Scope Planning

Before building, the playbook scope was deliberately planned to align with the later phases of the project (Containment, Eradication, Recovery), rather than building a single generic alerting playbook. Three playbooks were scoped:

1. Notify-SOC — baseline alerting on any incident.
2. Isolate-Machine — automated containment for high-severity incidents.
3. Disable-Account — automated eradication of compromised credentials for high-severity incidents.

This mapping was intentional so that Phase 8's playbooks would later serve as direct evidence for Phase 11 (Containment) and Phase 12 (Eradication), with Recovery (Phase 13) deliberately left as a manual, human-reviewed step rather than fully automated.

2. Playbook 1 — Notify-SOC-ADLab

A Logic App was built from scratch with:

1. Trigger: Microsoft Sentinel incident.
2. Action: Send an email (V2) via Office 365 Outlook, dynamically populated with Incident Title, Severity, Status, and Description from the triggering incident.

The playbook was granted Microsoft Sentinel Automation Contributor permissions and wired to a dedicated automation rule (Trigger-Notify-SOC), configured to fire on every incident regardless of severity, ensuring baseline visibility into all detections.

3. Playbook 2 — Isolate-Machine-ADLab

Rather than building this from scratch, an existing, previously self-built playbook (Isolate-endpoint---MDE---Incident-Triggered) was identified as reusable, having originally been built independently on personal Azure resources before being separately replicated onto a client's environment during a client engagement — confirming no client-owned resources were being reused.

The playbook's workflow (dynamically pulling affected hosts from incident entities, cross-referencing against Defender for Endpoint's device list, and conditionally isolating matched machines) was cloned into the lab's resource group via Logic App code view export/import. This required:

1. Enabling System-Assigned Managed Identity on the new Logic App (the original used identity-based authentication, which cannot be transferred via JSON copy).
2. Re-authenticating the Microsoft Sentinel and Defender for Endpoint connectors fresh, scoped to the lab's own subscription context.
3. Granting the new playbook Sentinel automation permissions.

The rebuilt playbook retained its full logic: Get Hosts → cross-reference MDE machine list → Condition → Isolate Machine → Add comment to incident (audit trail).

4. Playbook 3 — Disable-Account-ADLab (with supporting infrastructure)

This playbook required additional infrastructure, since the lab's Active Directory (lab.local) is a standalone AD DS environment not synced to Microsoft Entra ID — meaning cloud-native identity connectors (e.g. Microsoft Entra ID connector) cannot reach or act on its user accounts.

To bridge this gap, the following was built:

1. Azure Automation Account (aa-ad-cyberdefense-lab) created in the lab's resource group.
2. Hybrid Runbook Worker registered on DC01 (extension-based), allowing cloud-triggered automation jobs to execute locally against the on-prem-style AD.
3. PowerShell Runbook (Disable-CompromisedAccount_Ad_Lab) authored to accept a username parameter and execute Disable-ADAccount against it.



4. The runbook was tested end-to-end using a disposable test account (TestDisableUser), confirming successful execution on the Hybrid Worker and correct account disablement, verified via both job output and direct PowerShell query.

With the infrastructure validated, an existing self-built playbook pattern (Disable_user_testrun) was used as a structural reference — specifically its method of extracting the compromised account's identity from incident entities (Entities - Get Accounts → Initialize variables, concatenating account name and UPN suffix into a usable identity string). Rather than reusing this playbook's original HTTP-based Microsoft Graph API call (unsuitable for on-prem AD), the Disable-Account-ADLab playbook was rebuilt fresh in the Designer with:

1. Microsoft Sentinel incident trigger.
2. Entities - Get Accounts.
3. Initialize variable (TargetUserId, built from account Name + UPN Suffix).
4. Azure Automation — Create job, targeting the Hybrid Worker Group and the tested runbook, passing TargetUserId as the runbook's username parameter, configured to wait for job completion.
5. Add comment to incident (audit trail).

5. Automation Rule Wiring

Two automation rules were configured in Microsoft Sentinel to orchestrate the three playbooks:

1. Trigger-Notify-SOC — no severity condition, runs Notify-SOC-ADLab on every incident.
2. Contain-Eradicate-Rule — condition: Severity equals High, runs Isolate-Machine-ADLab followed sequentially by Disable-Account-ADLab, reflecting a realistic contain-then-eradicate response order.

Outcome: All three playbooks are built, individually validated, and correctly wired into Sentinel's incident automation pipeline. The design deliberately separates low-impact automated notification (runs on all incidents) from higher-impact automated response actions (gated behind severity), and includes a working solution for automating actions against on-premises-style Active Directory from cloud-based Sentinel incidents — a nontrivial architectural gap that was identified, explained, and resolved during the build. This satisfies Phase 8's expected outcome: playbooks developed and integrated with incident response workflows.

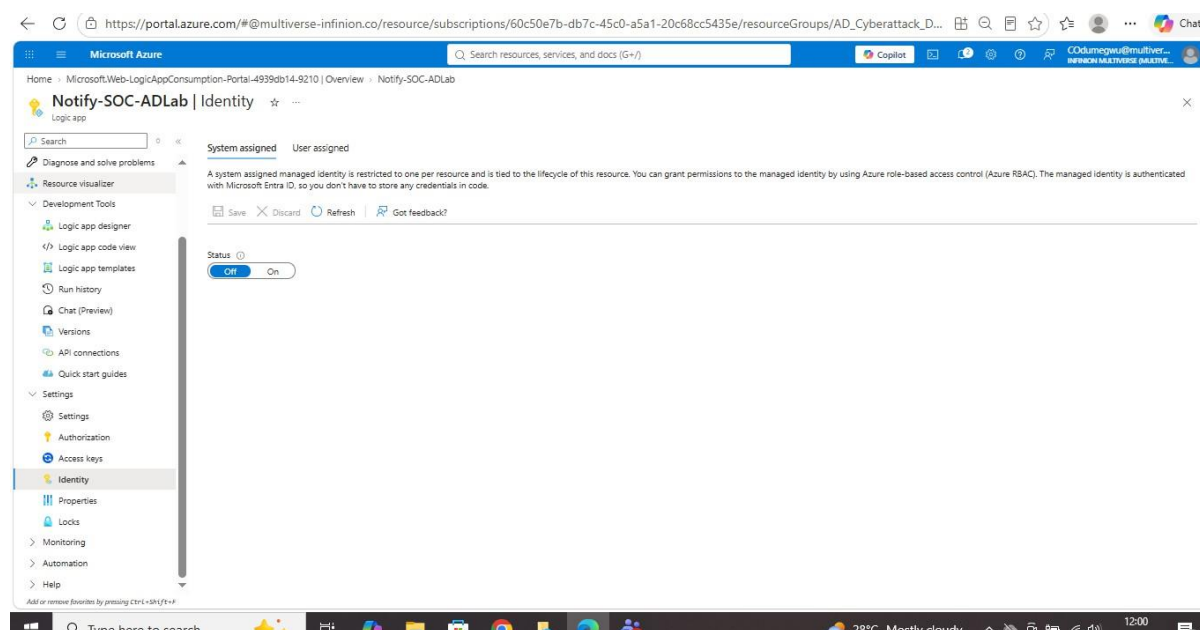


Figure 8.1 — Notify-SOC-ADLab Logic App, Identity blade — system-assigned managed identity currently Off, prior to activation.

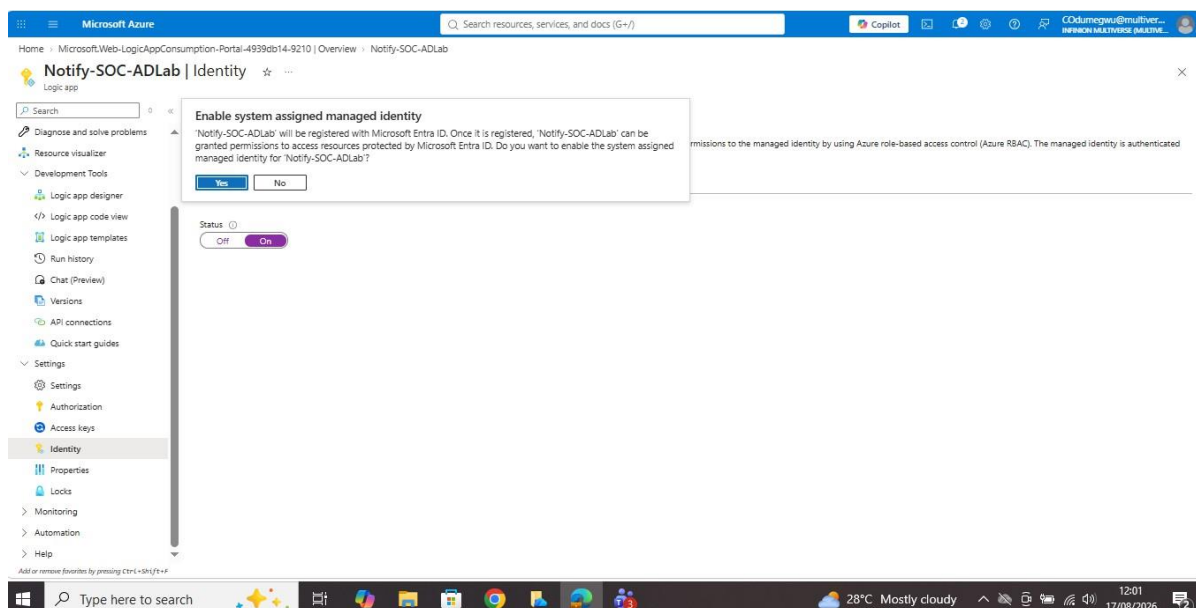


Figure 8.2 — Confirming the prompt to enable system-assigned managed identity for Notify-SOC-ADLab.

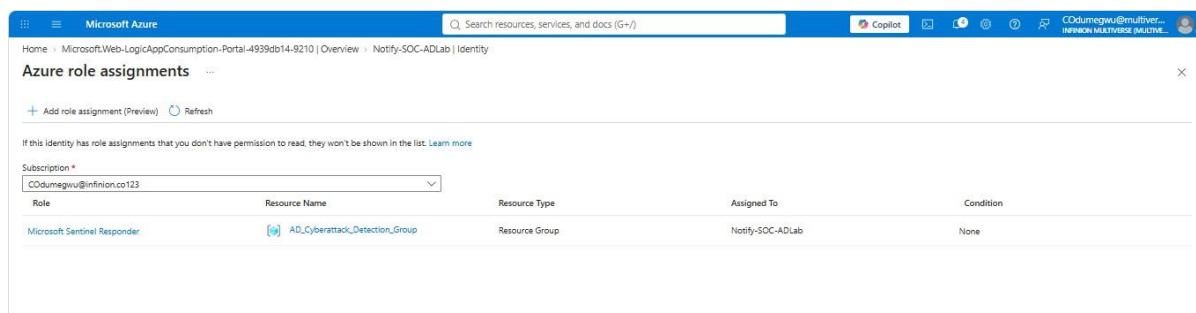


Figure 8.3 — Azure role assignments confirming Notify-SOC-ADLab's managed identity holds the Microsoft Sentinel Responder role on AD_Cyberattack_Detection_Group.



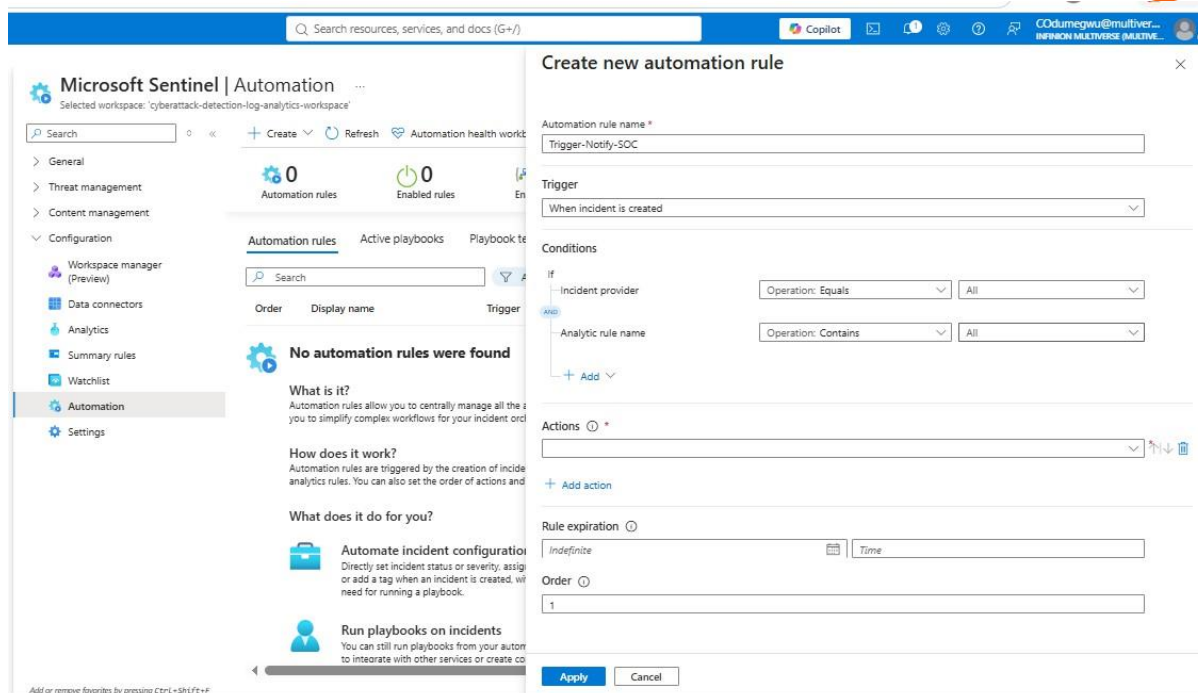


Figure 8.4 — Creating the Trigger-Notify-SOC automation rule in Microsoft Sentinel, trigger set to “When incident is created”.

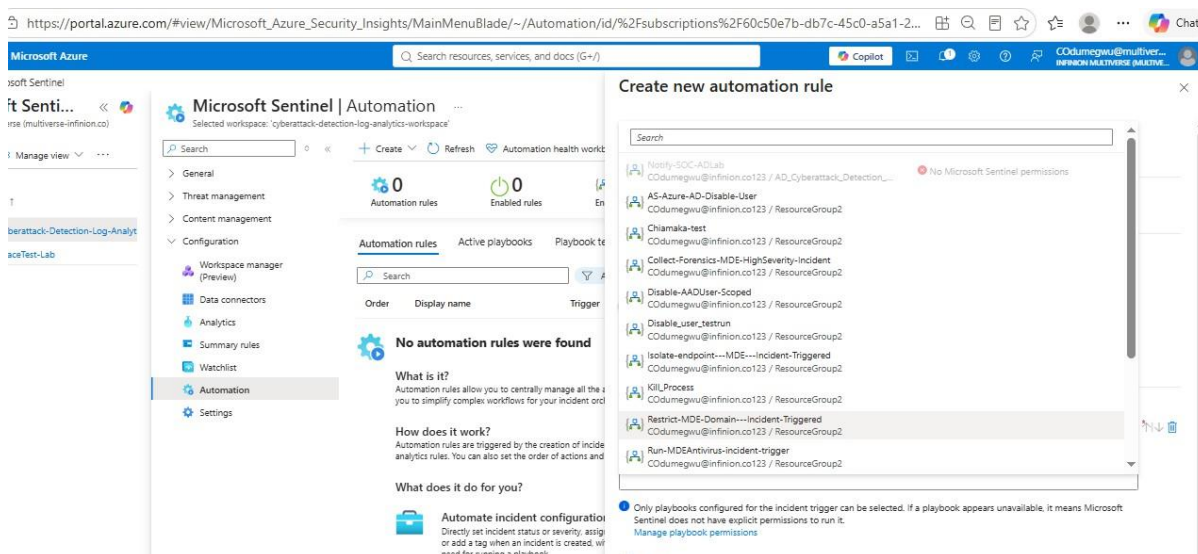


Figure 8.5 — The playbook selector initially flagging Notify-SOC-ADLab with “No Microsoft Sentinel permissions”, alongside other pre-existing playbooks from earlier work.



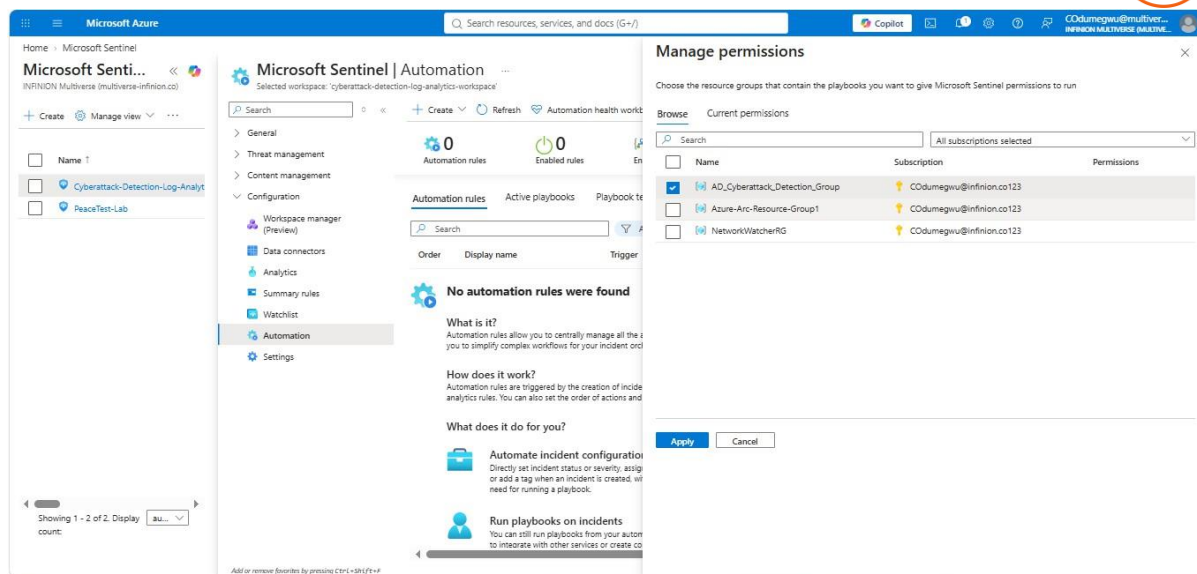


Figure 8.6 — Resolving the permissions gap via "Manage permissions", explicitly granting Sentinel access to run playbooks from AD_Cyberattack_Detection_Group.

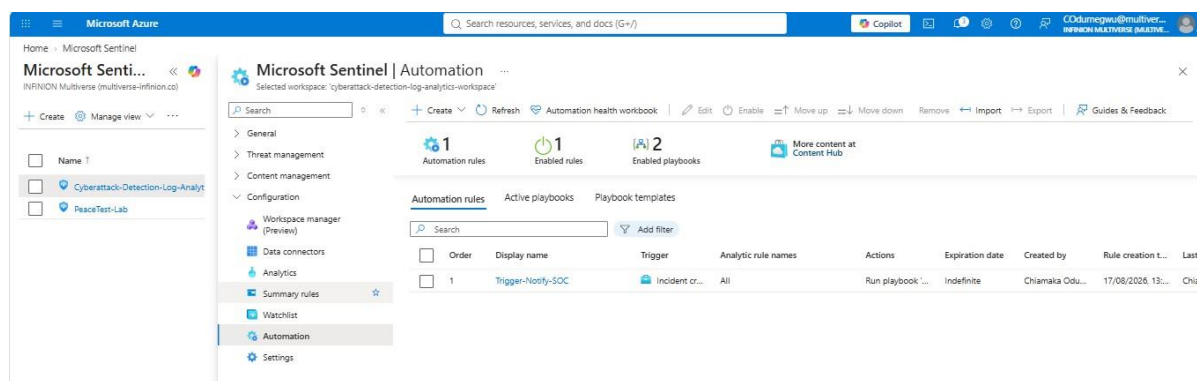


Figure 8.7 — Trigger-Notify-SOC automation rule live: 1 automation rule, 1 enabled rule, 2 enabled playbooks now shown in the workspace.

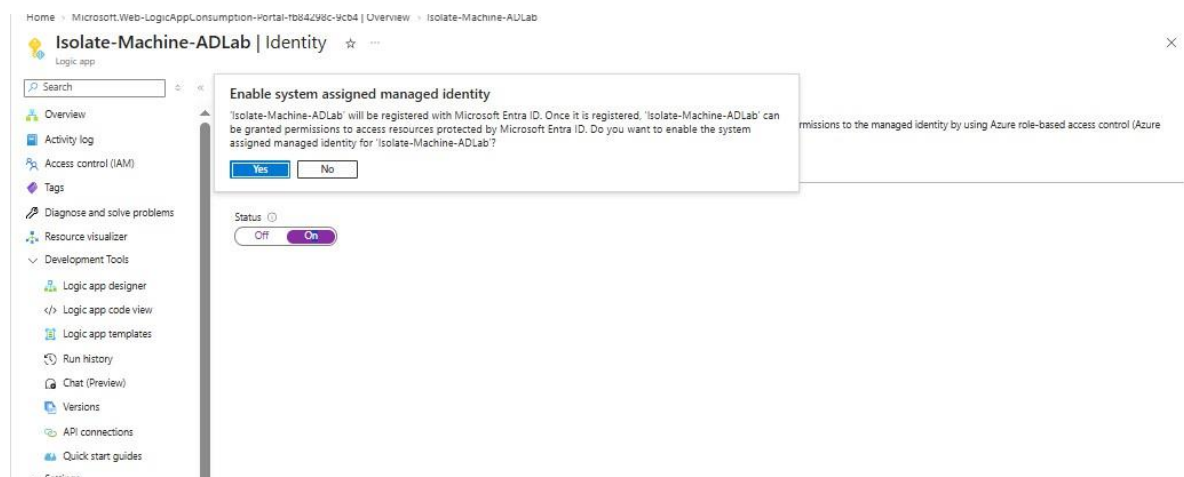


Figure 8.8 — Enabling system-assigned managed identity on the cloned Isolate-Machine-ADLab playbook.



Microsoft Azure

Home > Microsoft.Web-LogicAppConsumption-Portal-fb84298c-9c64 | Overview > Isolate-Machine-ADLab | Identity

Azure role assignments

+ Add role assignment (Preview) Refresh

If this identity has role assignments that you don't have permission to read, they won't be shown in the list. [Learn more](#)

Subscription *
CODumegwu@infinion.co123

Role	Resource Name	Resource Type	Assigned To	Condition
Microsoft Sentinel Automation Contributor	AD_Cyberattack_Detection_Group	Resource Group	Isolate-Machine-ADLab	None

Figure 8.9 — Azure role assignments confirming Isolate-Machine-ADLab's managed identity holds the Microsoft Sentinel Automation Contributor role.

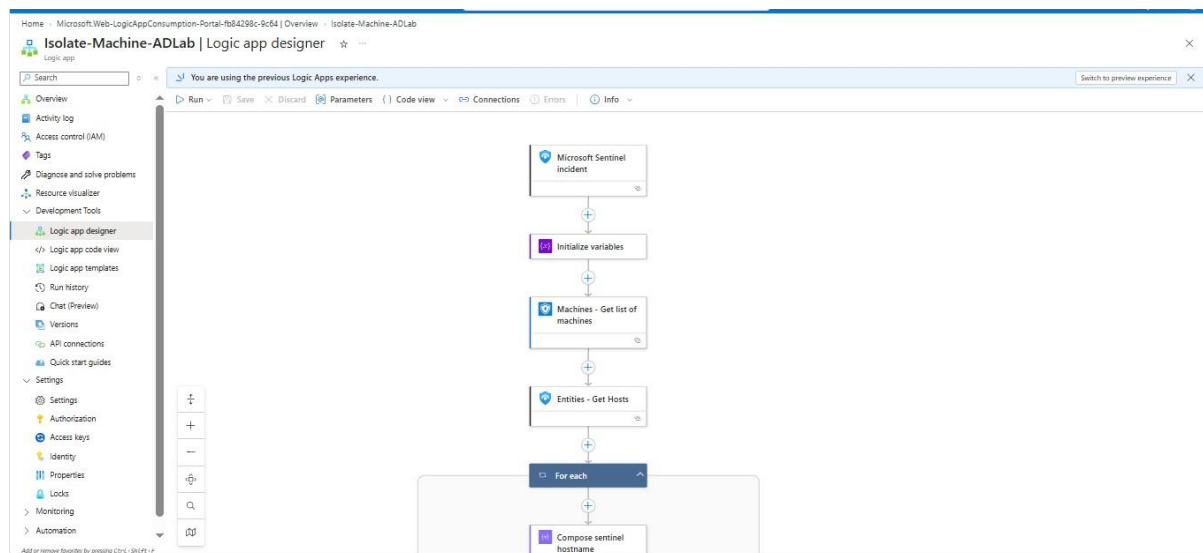


Figure 8.10 — The rebuilt Isolate-Machine-ADLab workflow in Logic App Designer: Sentinel incident trigger, Initialize variables, Machines - Get list of machines, Entities - Get Hosts, and the start of a For each loop.

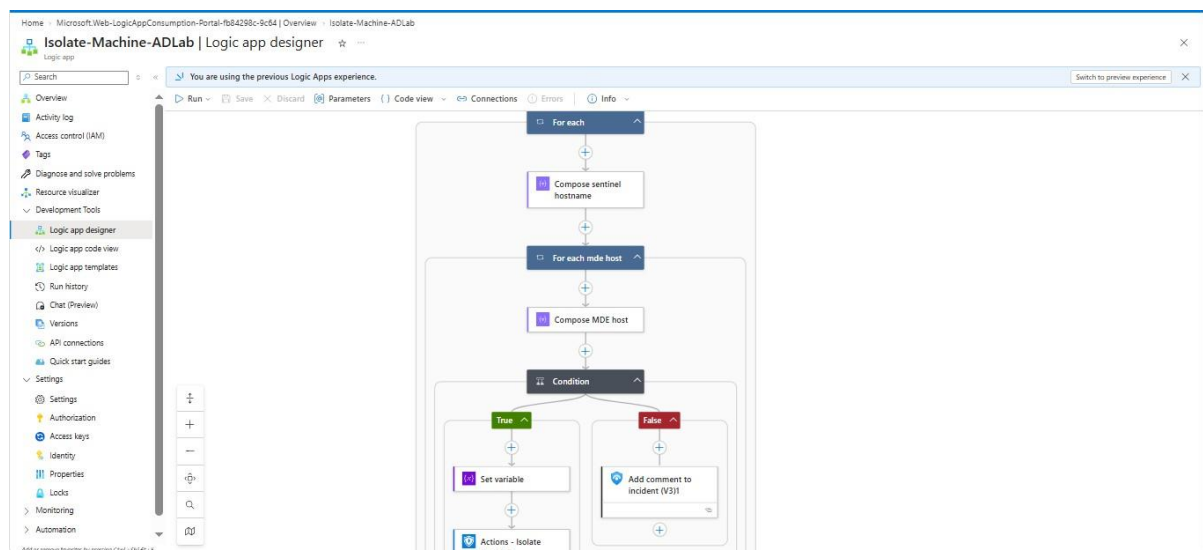


Figure 8.11 — Continuation of the workflow: a nested loop cross-referencing the MDE host list, feeding into a Condition that branches to isolation or comment-only.

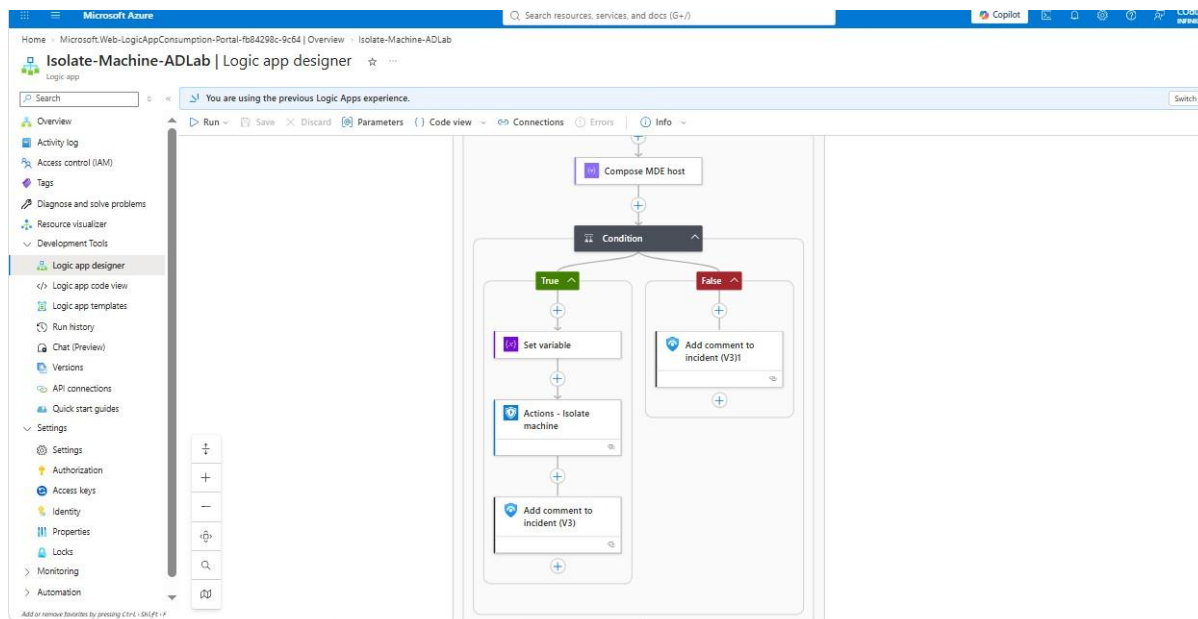


Figure 8.12 — Full view of the Condition branches: the True path sets a variable, runs Actions - Isolate machine, then adds a comment; the False path only adds a comment to the incident.

Figure 8.13 — Creating the Contain-Eradicate-Rule automation rule: condition Severity equals High, first action set to run playbook Isolate-Machine-ADLab.



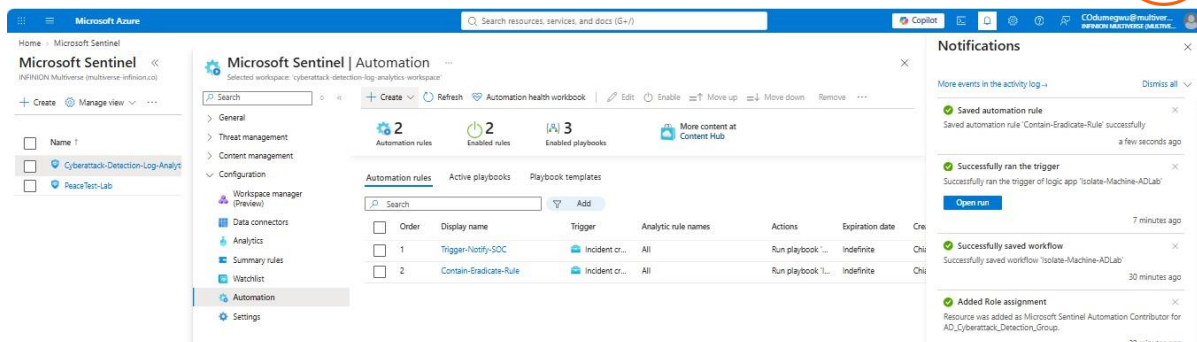


Figure 8.14 — Contain-Eradicate-Rule saved successfully — 2 automation rules, 2 enabled rules, 3 enabled playbooks, confirmed via the notifications panel.

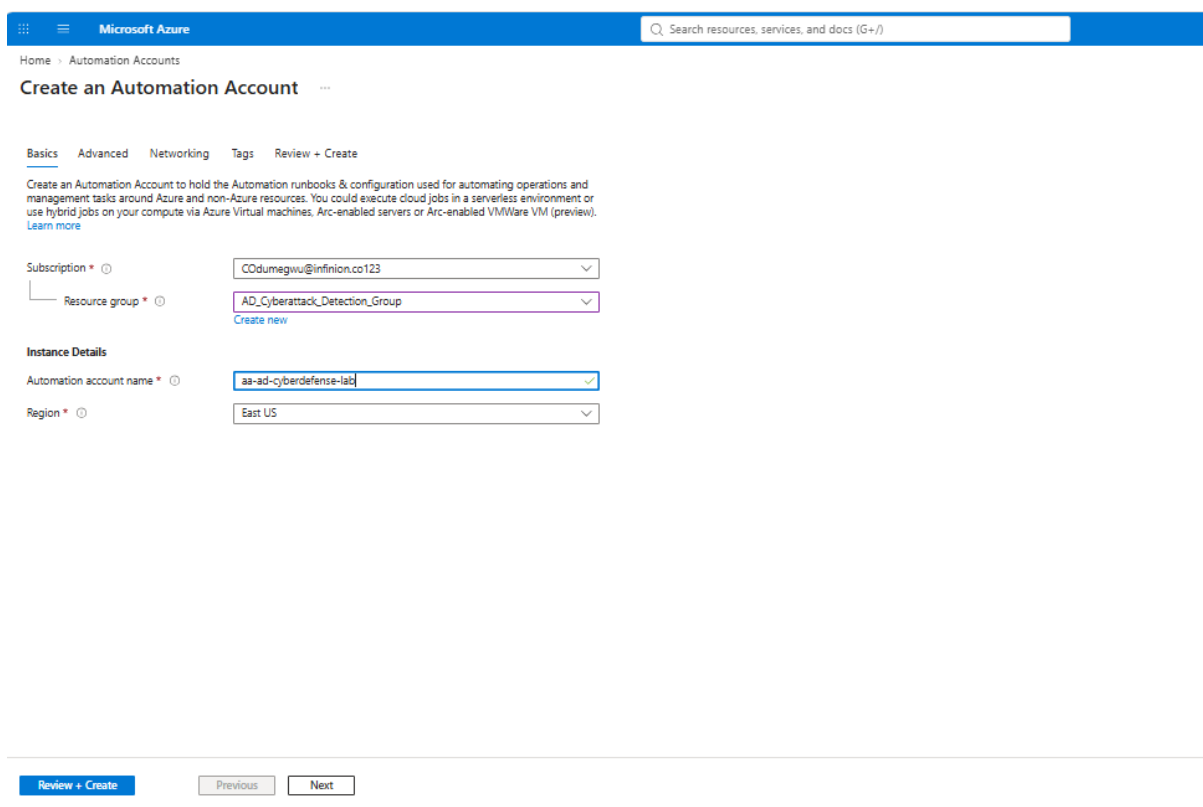


Figure 8.15 — Creating the Azure Automation Account (aa-ad-cyberdefense-lab) in East US, inside AD_Cyberattack_Detection_Group.



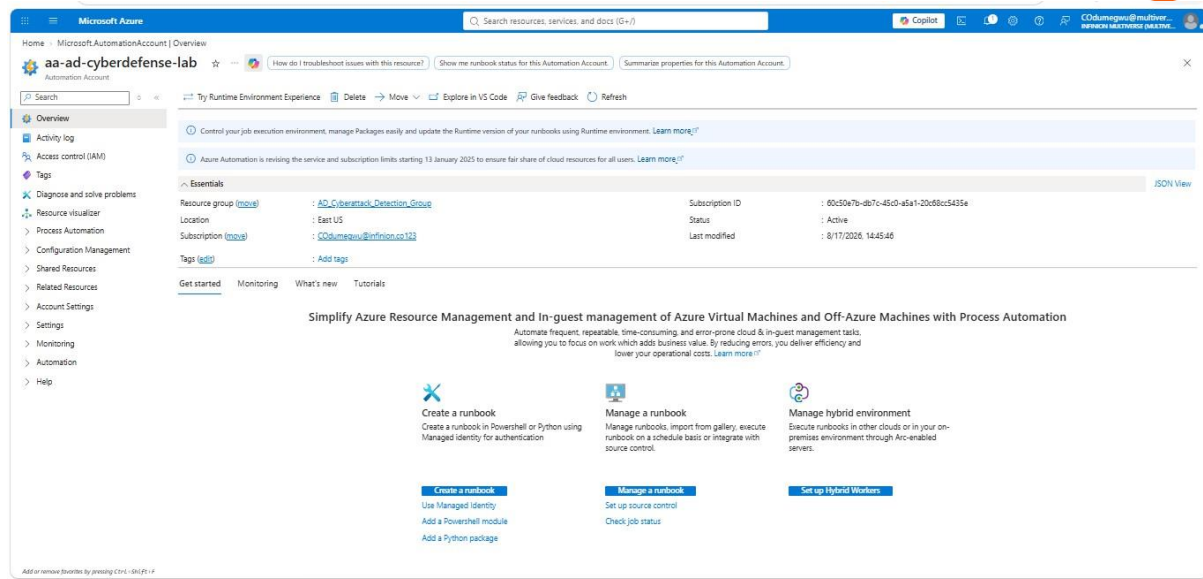


Figure 8.16 — Automation Account overview confirming successful deployment: Active status, East US, ready for runbooks and hybrid workers.

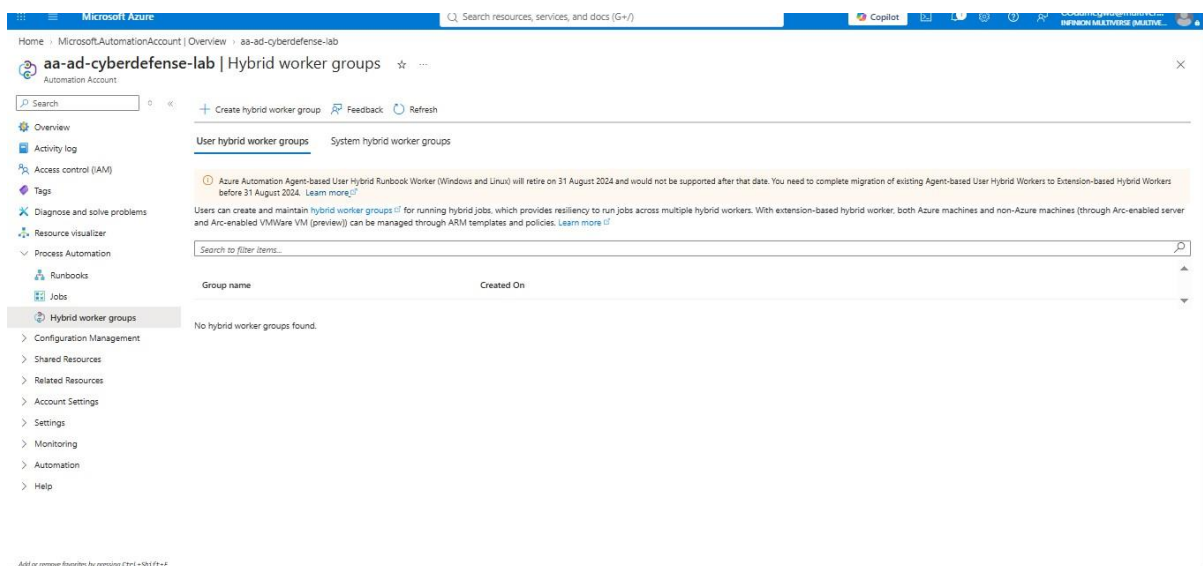


Figure 8.17 — Hybrid worker groups blade, empty before registration, with Azure's notice confirming the legacy Agent-based hybrid worker method is retiring in favour of the extension-based approach used here.



Microsoft Azure

Search resources, services, and docs (G+/)

Home > Microsoft.AutomationAccount | Overview > aa-ad-cyberdefense-lab | Hybrid worker groups

Create Hybrid worker group

Hybrid worker group

* Basics Hybrid workers Review + Create

Create a hybrid worker group that would host Linux or Windows machines as hybrid workers. With [extension based hybrid worker](#), both Azure machines and non-Azure machines (through [Arc-enabled Server](#) and [Arc-enabled VMware VM \(preview\)](#)) can be managed through ARM templates and policies.

Complete the Basics tab then 'Review + create' to create a hybrid worker group or review 'Hybrid Workers' tab to add machines.

Name *

Use Hybrid Worker Credentials ⓘ ☐ Custom ☒ Default

Figure 8.18 — Naming the new extension-based Hybrid worker group DC01-Worker-Group.

Home > Microsoft.AutomationAccount | Overview > aa-ad-cyberdefense-lab | Hybrid worker groups > Create Hybrid worker group

Add machines as hybrid worker

Selected machines would be added to the hybrid worker group and hybrid worker extension would be installed to connect to Automation service. If a machine is not visible or not added to the group, read about the [prerequisites](#) and [troubleshooting steps](#).

Subscription: CODumegwu@infinion.co123 Resource Group: No filter Location: No filter Type: All

Search to filter items...

Name	Subscription	Resource group	Operating System	Location	Status
☐ CLIENT01	CODumegwu@infinion.co123	AD_CYBERATTACK_DETECTION_GROUP	Windows	eastus	VM stopped
☒ DC01	CODumegwu@infinion.co123	AD_CYBERATTACK_DETECTION_GROUP	Windows	eastus	VM stopped
☐ KALI01	CODumegwu@infinion.co123	AD_Cyberattack_Detection_Group	Linux	eastus	VM deallocated
☐ chlamaka-VirtualBox	CODumegwu@infinion.co123	Azure-Arc-Resource-Group1	Linux	eastus	Expired
☐ CHI-DCVM	CODumegwu@infinion.co123	resourcegroup1	Windows	eastus	VM deallocated
☐ CHI-VM1	CODumegwu@infinion.co123	resourcegroup1	Windows	eastus	VM deallocated
☐ CHI-VM2	CODumegwu@infinion.co123	resourcegroup1	Windows	eastus	VM deallocated
☐ CHI-VM3	CODumegwu@infinion.co123	resourcegroup1	Windows	eastus	VM deallocated

Figure 8.19 — Selecting DC01 from the list of eligible machines to add as a hybrid worker.

Home > Microsoft.AutomationAccount | Overview > aa-ad-cyberdefense-lab | Hybrid worker groups

Create Hybrid worker group

Hybrid worker group

* Basics Hybrid workers Review + Create

Select Azure Virtual Machines, Arc-enabled servers or Arc-enabled VMware VM (preview) to host the automation jobs. If you don't select any machines, an empty hybrid worker group will be created. You can still add or remove machines later.

+ Add machines X Remove machines

Machine Name	Subscription Id	Resource Group	Operating System	Location
☐ DC01	60c50a7b-d67c-45c0-a5a1-20d68c5435e	AD_CYBERATTACK_DETECTION_GROUP	Windows	eastus

Review + Create Previous Next

Figure 8.20 — DC01 confirmed as the selected machine for the hybrid worker group, ready for Review + Create.



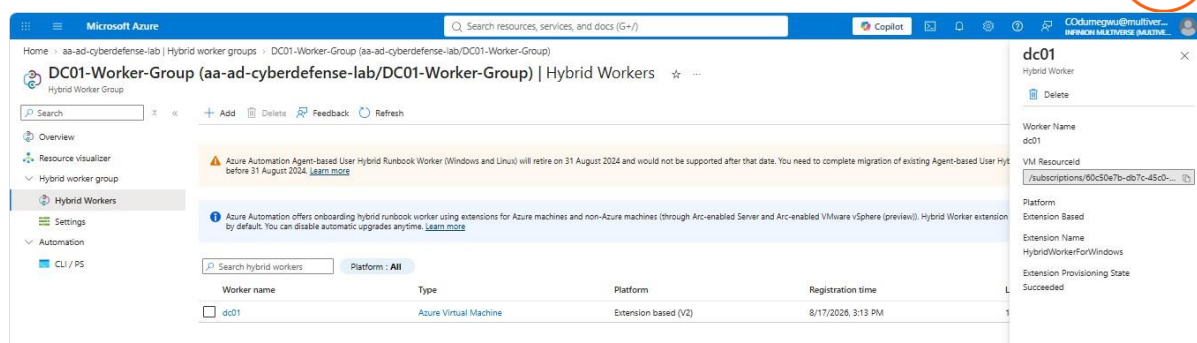


Figure 8.21 — DC01-Worker-Group registered successfully: dc01 shows as an Extension based (V2) hybrid worker, with the HybridWorkerForWindows extension's provisioning state Succeeded.

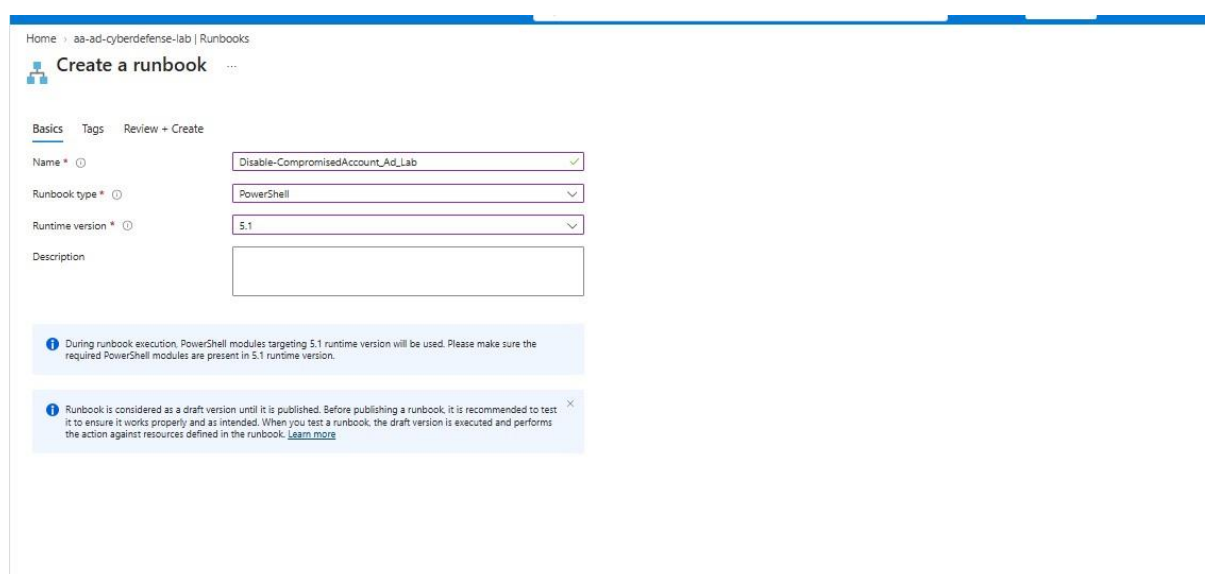


Figure 8.22 — Creating the Disable-CompromisedAccount_Ad_Lab PowerShell runbook, runtime version 5.1.

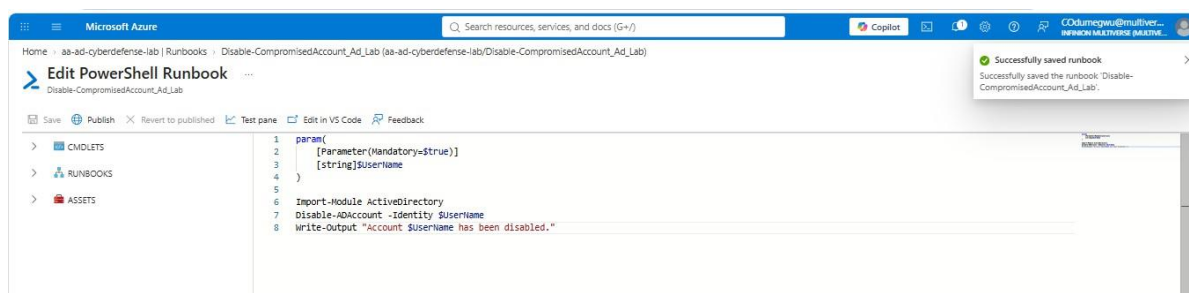


Figure 8.23 — The runbook script, saved successfully: a mandatory \$UserName parameter, importing the ActiveDirectory module, then running Disable-ADAccount against the target identity.

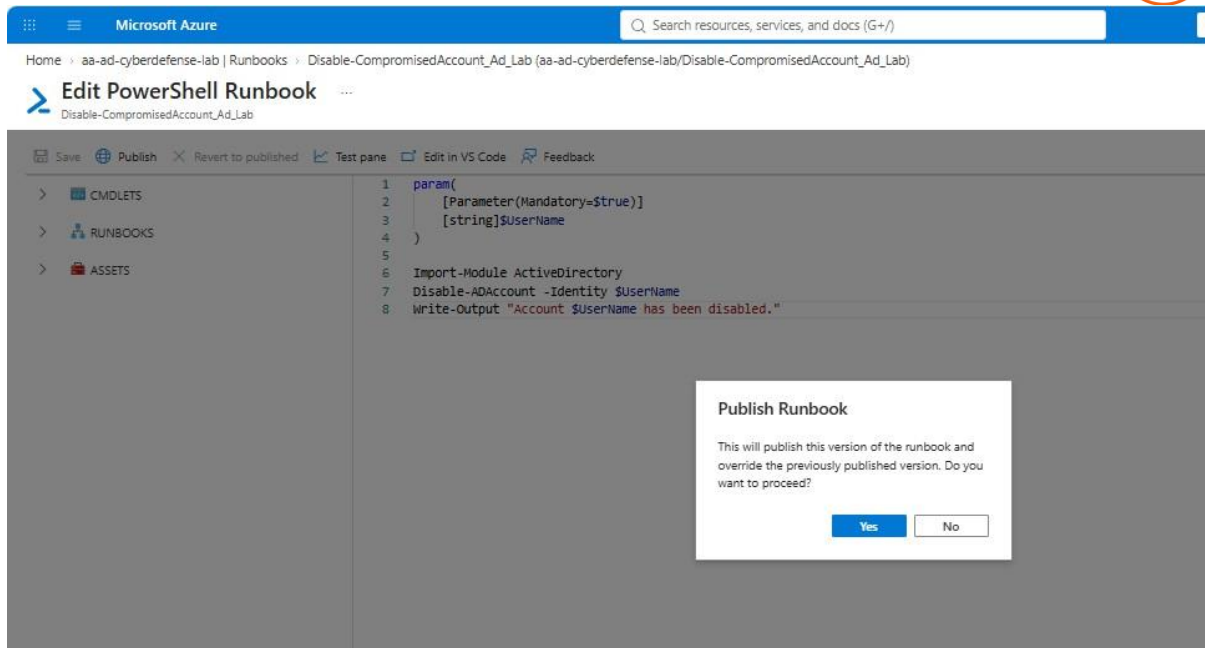


Figure 8.24 — Publishing the runbook, confirming this version becomes the live, callable version.

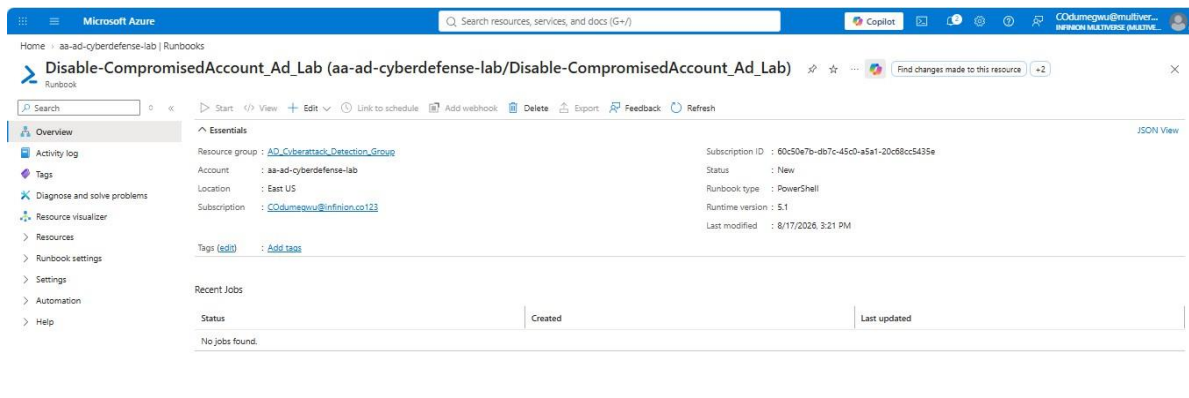


Figure 8.25 — Runbook overview immediately after publishing: Status New, PowerShell 5.1, no jobs run yet.

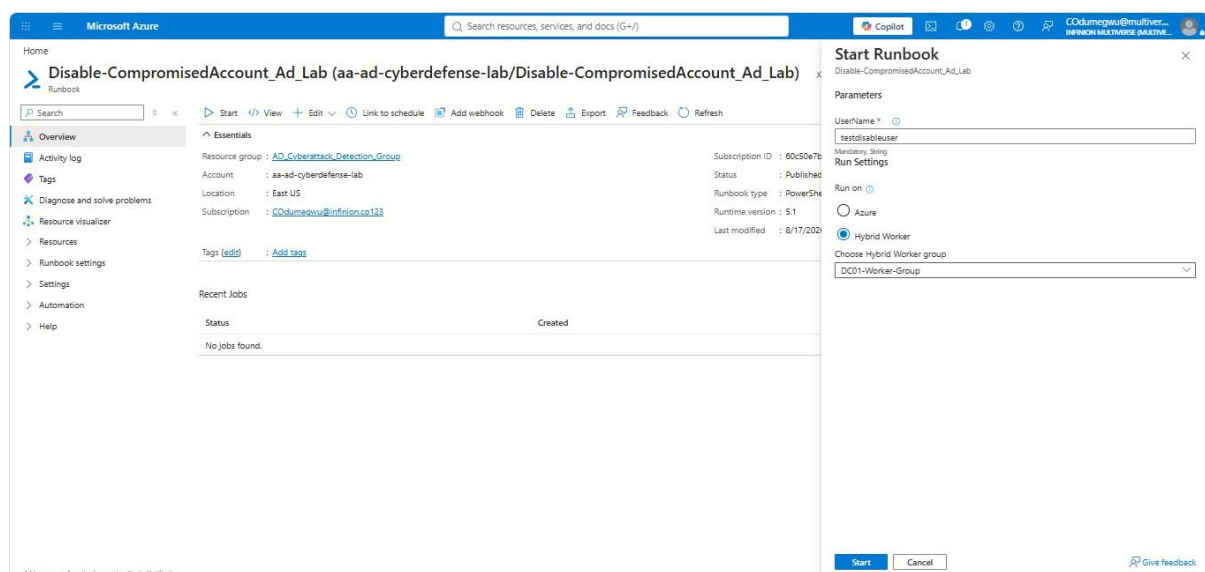


Figure 8.26 — Manually starting the runbook as a test run — UserName "testdisableuser", Run on set to Hybrid Worker, targeting DC01-Worker-Group.



The screenshot shows the 'Job' details page for 'Disable-CompromisedAccount_Ad_Lab' in the Azure portal. The job status is 'Completed' and it was run on 'DC01-Worker-Group' by user 'testdisableuser'. The 'Input' tab is selected, showing the 'UserName' input as 'testdisableuser'.

Essentials	
Id	: 44be3f71-5166-4a94-b9bc-968eaae7340e
Status	: Completed
Ran on	: DC01-Worker-Group
Ran As	: User
Created	: 8/17/2026, 15:51:50
Last Update	: 8/17/2026, 15:52:06
Runbook	: Disable-CompromisedAccount_Ad_Lab
Source snapshot	: View source snapshot

Name	Value
UserName	"testdisableuser"

Figure 8.27 — Job execution details confirming Status: Completed, Ran on: DC01-Worker-Group, with input UserName "testdisableuser".

The screenshot shows the 'Output' tab for the same job. It displays a single line of output: 'Account testdisableuser has been disabled.'.

```
Account testdisableuser has been disabled.
```

Figure 8.28 — The job's Output tab confirming success: "Account testdisableuser has been disabled."

The screenshot shows the 'Jobs' history for the 'Disable-CompromisedAccount_Ad_Lab' runbook. It lists one job with a status of 'Completed'.

Status	Created	Last updated
✓ Completed	8/17/2026, 3:51:50 PM	8/17/2026, 3:52:06 PM

Figure 8.29 — Jobs history for the runbook, confirming the test run completed successfully.



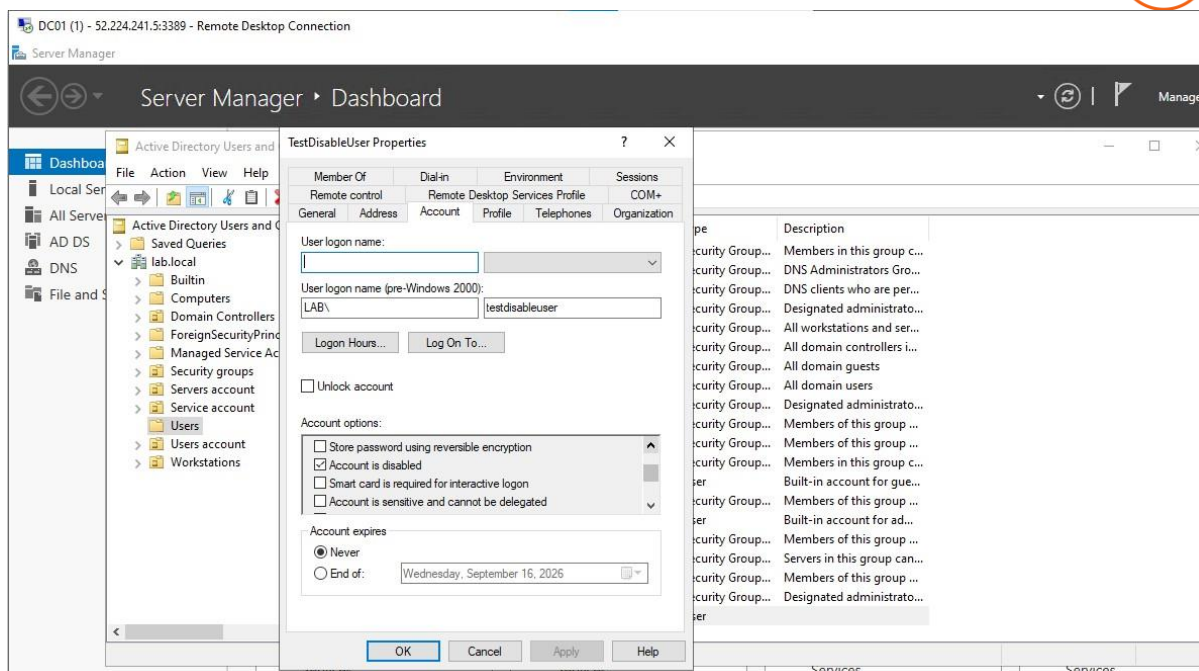


Figure 8.30 — Direct verification in Active Directory Users and Computers on DC01: TestDisableUser's Account properties show "Account is disabled" checked, cross-validating the runbook's output.

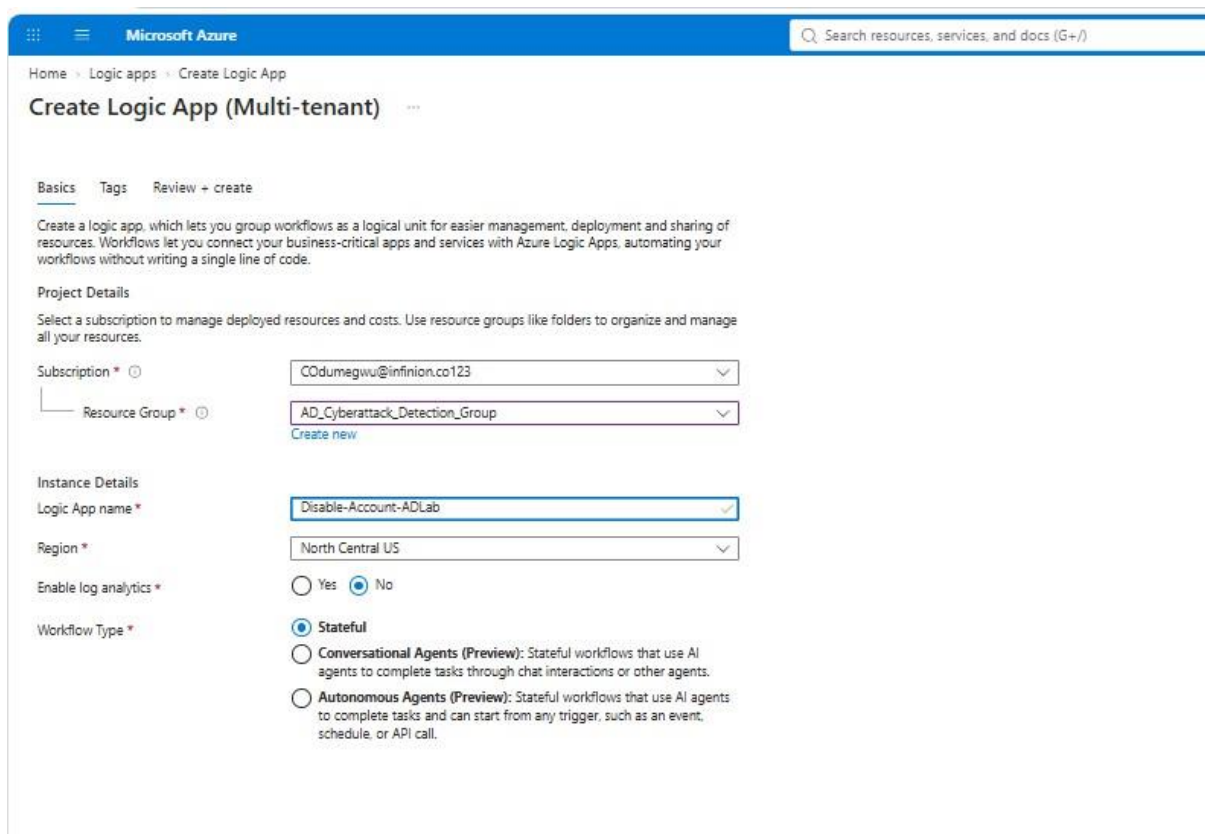


Figure 8.31 — Creating the Disable-Account-ADLab Logic App with a Stateful workflow type.



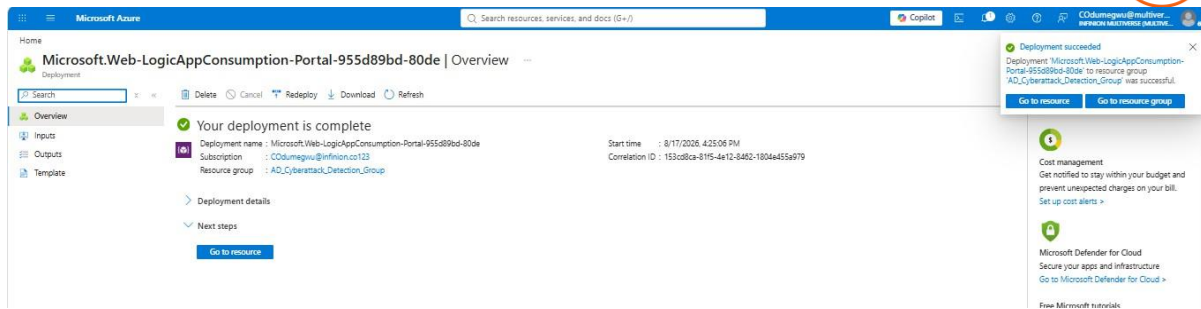


Figure 8.32 — Deployment succeeded confirmation for the new Logic App.

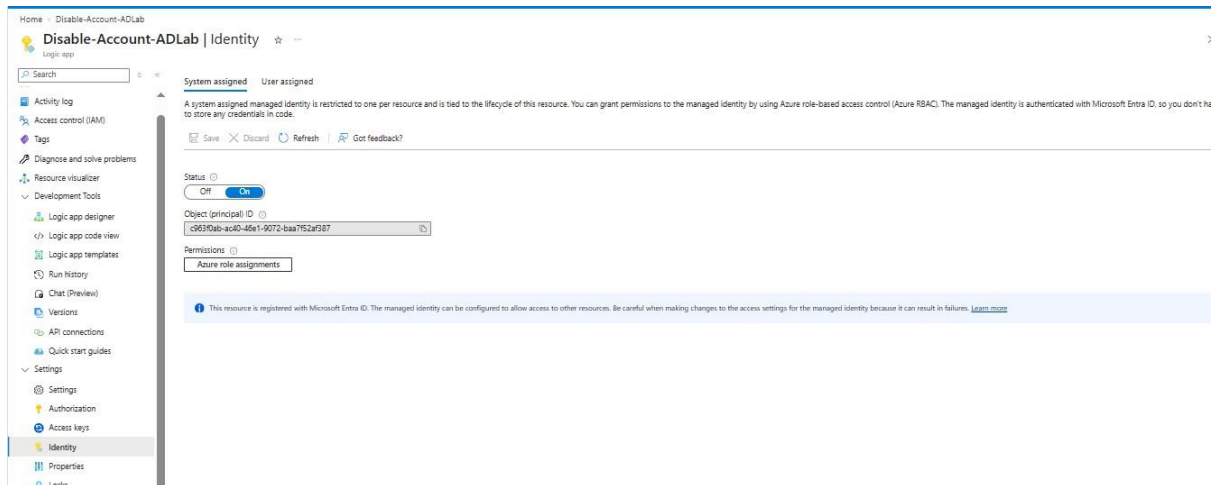


Figure 8.33 — System-assigned managed identity enabled for Disable-Account-ADLab.

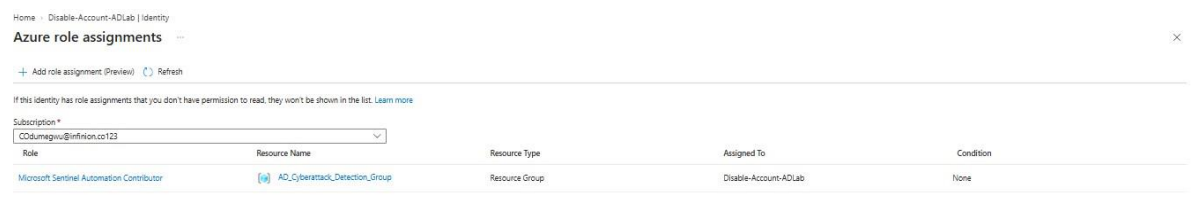


Figure 8.34 — Azure role assignments confirming Disable-Account-ADLab's managed identity holds the Microsoft Sentinel Automation Contributor role.

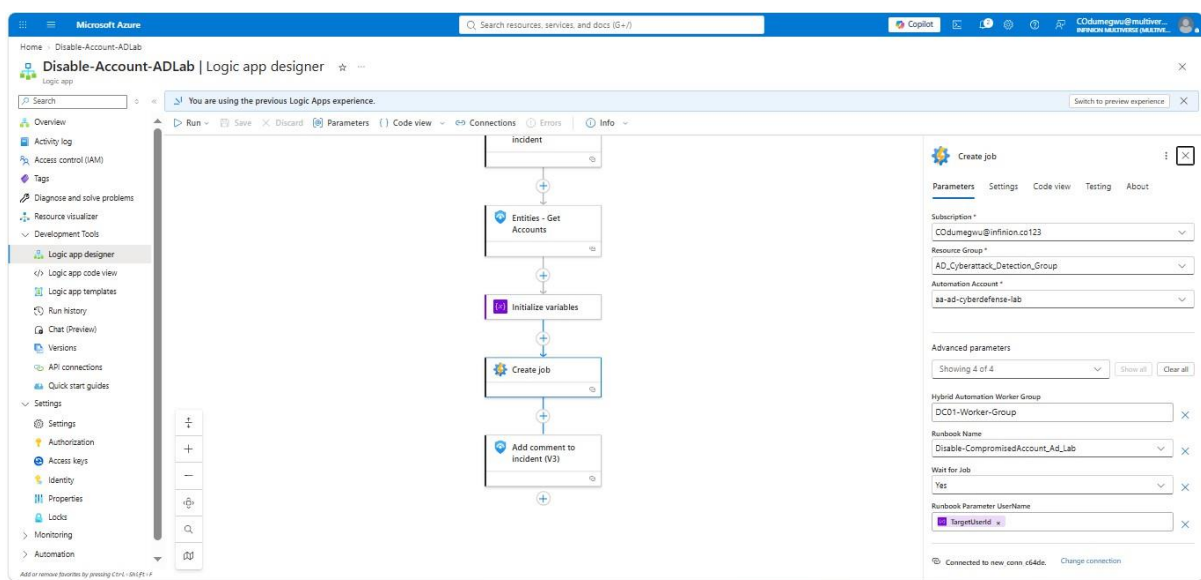


Figure 8.35 — The full Disable-Account-ADLab workflow in Designer: Entities - Get Accounts, Initialize variables, and an Azure Automation "Create job" action targeting Automation Account aa-ad-cyberdefense-lab, Hybrid Worker Group DC01-Worker-Group, runbook Disable-CompromisedAccount_Ad_Lab, with Wait for Job set to Yes and the UserName parameter bound to TargetUserId.

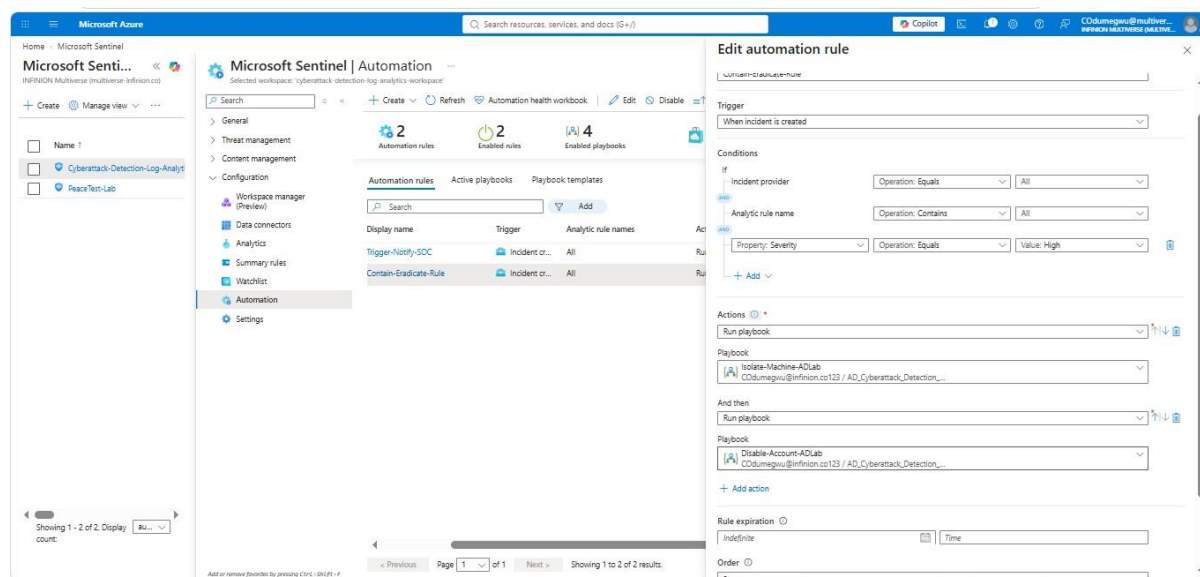


Figure 8.36 — The Contain-Eradicate-Rule fully wired: Severity equals High, running Isolate-Machine-ADLab followed by Disable-Account-ADLab in sequence — now 4 enabled playbooks in total.

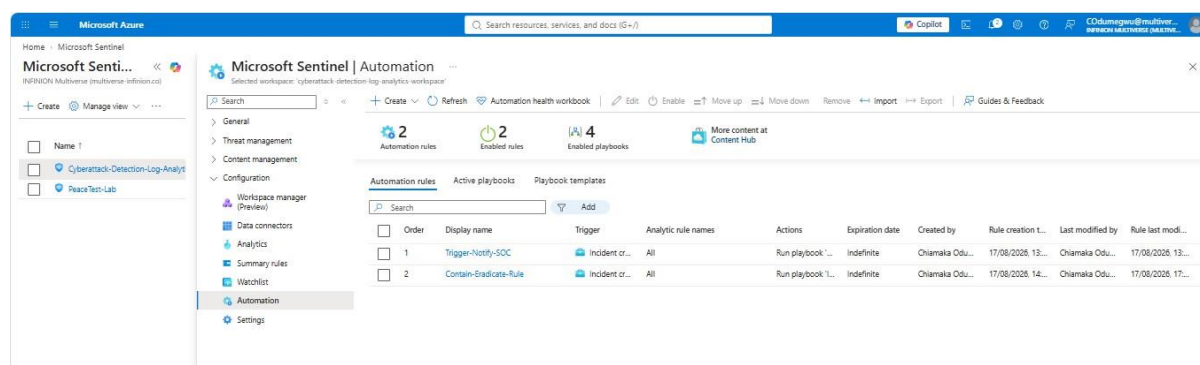


Figure 8.37 — Final Automation blade for Phase 8: both automation rules (Trigger-Notify-SOC, Contain-Eradicate-Rule) live and enabled, 4 enabled playbooks — the complete orchestration confirmed.

Attack Simulation

1. Password Spraying

A deliberately weak password was set on one AD account (john.smith: Summer2026!) while all other accounts (mary.jones, victor.williams, security.analyst, sqlsvc) were configured with strong, complex passwords to create a realistic, controlled testing scenario. From KALI01, netexec was used to spray the weak password against a username list built from known domain accounts, targeting DC01 over SMB. The attack correctly identified john.smith as the only valid account, while all other accounts failed authentication as expected. Detection was validated via Microsoft Defender for Identity's IdentityLogonEvents table (queried through Defender XDR's Advanced Hunting), which captured the full spray pattern — one success against john.smith and multiple logon failures against the remaining accounts — confirming MDI visibility into the attack.

2. Kerberoasting

Using the credentials obtained from the successful password spray (john.smith), an SPN enumeration and ticket request attack was executed from KALI01 against the sqlsvc service account using `impacket-GetUserSPNs`. An initial encryption-type mismatch (`KDC_ERR_ETYPE_NOSUPP`) was encountered and resolved by adjusting the target account's supported Kerberos encryption types, after which a valid, crackable TGS hash was successfully extracted for sqlsvc. Initial detection validation via raw `IdentityLogonEvents` telemetry confirmed the underlying Kerberos activity was logged, though no named alert had yet fired — this was identified as expected behavior tied to Microsoft Defender for Identity's default 30-day learning period for certain behavioral detections. After disabling the learning period via Defender XDR's Advanced Settings, the attack's detection signature was confirmed through multiple named, high-severity MDI alerts: "Possible Kerberoasting attack," "Suspected Kerberos SPN exposure," and "Possible Kerberoasting LDAP reconnaissance," demonstrating that MDI successfully detected both the reconnaissance and exploitation phases of the attack.

3. Attacks Deferred

Due to time constraints, Pass-the-Hash, DCSync, and Lateral Movement simulations were not executed in this session and are documented as pending for a follow-up session.

Outcome: Two full attack techniques from the project's defined scope were successfully executed end-to-end from KALI01 against the lab's Active Directory environment, with detection independently confirmed through Microsoft Defender for Identity and validated via Defender XDR's Advanced Hunting and Alerts interface. This provides direct evidence toward Phase 10 (Detection and Analysis) and satisfies a partial but substantive completion of Phase 9's expected outcome: attacks executed and detected/logged by Microsoft security tools.

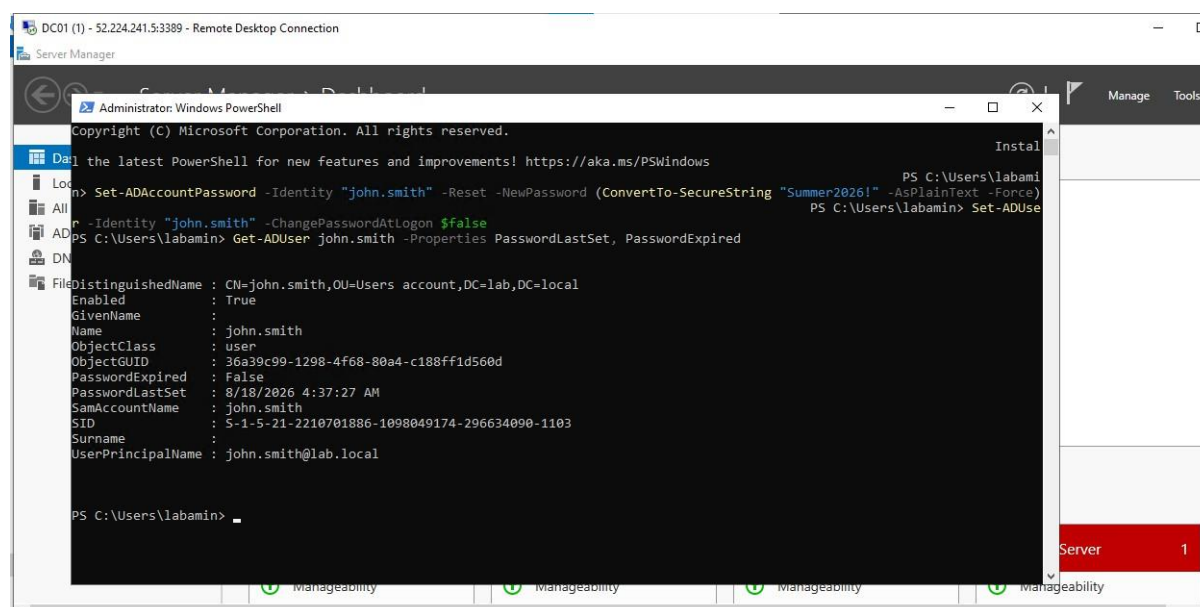


Figure 9.1 — Setting john.smith's deliberately weak password (Summer2026!) via PowerShell on DC01, with `Get-ADUser` confirming `PasswordExpired: False` and the updated `PasswordLastSet` timestamp.



```

https://www.kali.org/docs/troubleshooting/common-cloud-setup/

(Run: "touch ~/.hushlogin" to hide this message)
(kaliuser@ kali)-[~]
$ nano users.txt

(kaliuser@ kali)-[~]
$ cat users.txt
john.smith
mary.jones
victor.williams
security.analyst
sqlsvc

(kaliuser@ kali)-[~]
$

```

Figure 9.2 — On KALI01, building the users.txt target list from the lab's known domain accounts (john.smith, mary.jones, victor.williams, security.analyst, sqlsvc).

```

(kaliuser@ kali)-[~]
$ nxc smb 10.10.1.4 u users.txt -u 'Summer2026!' --continue-on-success
[+] Windows Server 2022 Build 20348 x64 (name:DC01) (domain:lab.local) (signing:True) (SMBv1:None) (Null Auth:True)
[+] lab.local\john.smith:Summer2026!
[-] lab.local\mary.jones:Summer2026! STATUS_LOGON_FAILURE
[-] lab.local\victor.williams:Summer2026! STATUS_LOGON_FAILURE
[-] lab.local\security.analyst:Summer2026! STATUS_LOGON_FAILURE
[-] lab.local\sqlsvc:Summer2026! STATUS_LOGON_FAILURE

(kaliuser@ kali)-[~]

```

Figure 9.3 — Running the netexec (nxc) SMB password spray against DC01 with the weak password Summer2026!: john.smith returns a success ([+]), while every other account fails authentication as expected.

Advanced hunting

Explore your content from Microsoft Sentinel

All data from Microsoft Sentinel, including tables, queries and functions is now available for you to explore.

Schema: Alerts & behaviors, Apps & identities

Query:

```

1 IdentityLogonEvents
2 | where Timestamp > ago(3h)
3 | where AccountName in ("john.smith", "mary.jones", "victor.williams", "security.analyst", "sqlsvc")
4 | project Timestamp, AccountName, ActionType, Application, DeviceName
5 | order by Timestamp desc

```

Results: 9 items

Timestamp	AccountName	ActionType	Application	DeviceName
Aug 18, 2026 6:14:...	sqlsvc	LogonFailed	Active Directory	10.10.1.6
Aug 18, 2026 6:14:...	sqlsvc	LogonFailed	Active Directory	NULL
Aug 18, 2026 6:14:...	security.analyst	LogonFailed	Active Directory	10.10.1.6
Aug 18, 2026 6:14:...	security.analyst	LogonFailed	Active Directory	NULL

Figure 9.4 — Advanced Hunting query against IdentityLogonEvents, filtered to the five spray-targeted accounts — results show LogonFailed events for sqlsvc and security.analyst captured by Microsoft Defender for Identity.



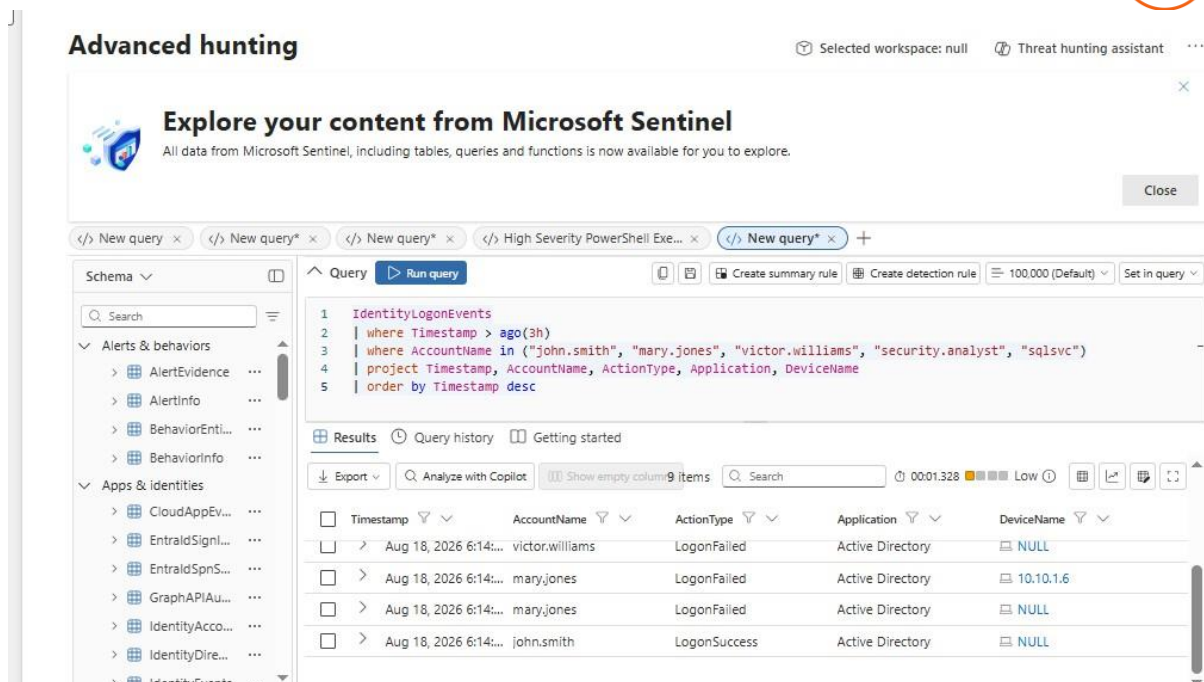


Figure 9.5 — Scrolled results of the same query, showing LogonFailed for victor.williams and mary.jones alongside a LogonSuccess entry for john.smith — matching the spray's real outcome exactly.

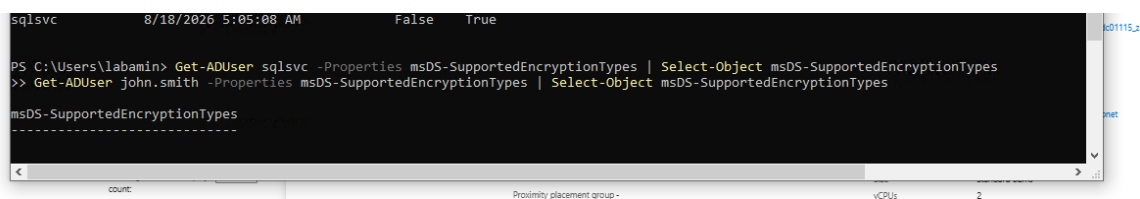


Figure 9.6 — Checking sqlsvc's msDS-SupportedEncryptionTypes attribute via PowerShell (returned empty), diagnosing the encryption-type mismatch hit during the initial Kerberoasting attempt.

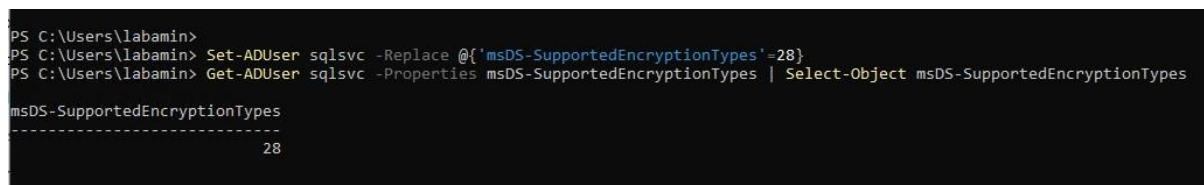


Figure 9.7 — Resolving the mismatch: setting sqlsvc's msDS-SupportedEncryptionTypes to 28 (RC4/AES), confirmed by the follow-up Get-ADUser query.

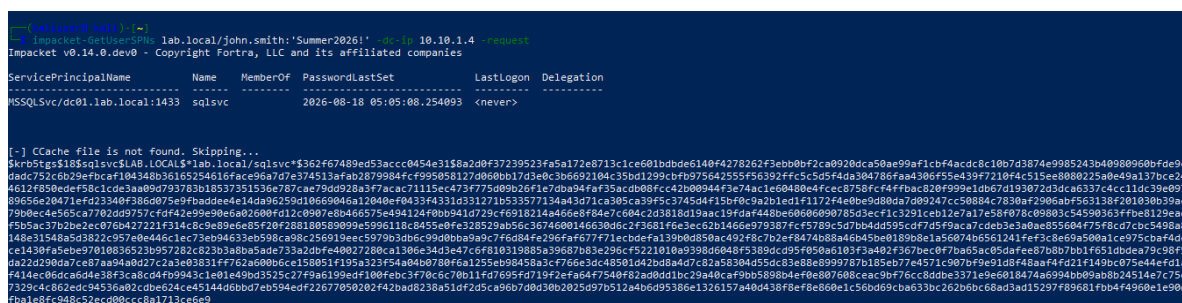


Figure 9.8 — The successful Kerberoasting run: impacket-GetUserSPNs, authenticated as the compromised john.smith account, extracting the sqlsvc SPN and its crackable Kerberos TGS ticket hash from DC01.

Detection and Analysis, Containment and Eradication

Following the attack simulations executed in Phase 9 (Password Spraying and Kerberoasting), this phase validated the lab's detection and analysis capability across Microsoft Defender for Identity, Microsoft Sentinel, and the unified Defender XDR portal.

Detection performance

All expected alerts fired correctly and correlated automatically into a single, well-scoped incident (ID 10054: "Multi-stage incident involving Credential access & Discovery on multiple endpoints"). Four distinct High-severity alerts — spanning LDAP reconnaissance, SPN exposure, anomalous Kerberos authentication, and the Kerberoasting technique itself — were generated across the attack's discovery and credential-access stages, correctly grouped under MITRE ATT&CK techniques (T1558.003 — Steal or Forge Kerberos Tickets: Kerberoasting; T1087 — Account Discovery) with accurate entity attribution to the compromised identity, the targeted service account, and the domain controller.

Automated response validation

Of the three SOAR playbooks wired to respond to High-severity incidents, two — SOC notification and account disablement — executed successfully end-to-end, confirmed via incident activity logs, email delivery, and direct Active Directory state verification. The third, device isolation, initially failed; investigation traced this to a managed identity permissions gap against the Microsoft Defender for Endpoint API, which was diagnosed and remediated.

Key analytical finding

Deeper review of the incident's entity graph revealed that neither device entity present (the attacker's external machine, and the domain controller) represented an actual device compromise — both appeared as entities for observational reasons (traffic source and request recipient, respectively), while the true compromise existed entirely at the identity layer. This distinction informed a design correction to the isolation playbook: an explicit safety exclusion was added to prevent the automation from acting against infrastructure entities that were never actually compromised, rather than relying on the correct outcome occurring by chance.

Outcome: Detection accuracy, incident correlation, and root-cause analysis of automation gaps are confirmed and documented. Custom KQL threat-hunting queries and full MITRE ATT&CK technique mapping remain available as optional extensions beyond the core deliverable documented here.



Advanced hunting

Explore your content from Microsoft Sentinel

All data from Microsoft Sentinel, including tables, queries and functions is now available for you to explore.

Close

Schema

Alerts & behaviors

- AlertEvidence
- AlertInfo
- BehaviorEnti...
- BehaviorInfo

Apps & identities

- CloudAppEv...
- EntraIdSignl...
- EntraIdSpnS...
- GraphAPIAu...
- IdentityAcco...

Query

```
1 IdentityLogonEvents
2 | where Timestamp > ago(1h)
3 | where AccountName == "john.smith"
4 | project Timestamp, AccountName, ActionType, Application, Protocol, DeviceName
5 | order by Timestamp desc
```

Results

Query history

Getting started

Export

Analyze with Copilot

Show empty columns

2 items

Search

00:04.734

Low

Filters

Add filter

Timestamp	AccountName	ActionType	Application	Protocol	DeviceName
> Aug 18, 2026 10:4...	john.smith	LogonSuccess	Active Directory	Kerberos	10.1...
> Aug 18, 2026 10:4...	john.smith	LogonSuccess	Active Directory	Ntlim	NUL

Figure 10.1 — Advanced Hunting query against IdentityLogonEvents, filtered to john.smith in the last hour — confirming both a Kerberos and NTLM LogonSuccess event tied to the password spray.

Home

Find Defender solutions

Home

Cases

Exposure management

- Overview
- Secure now
- Initiatives
- Recommendations
- Vulnerability management
- Attack surface
- Secure score
- Data connectors

Investigation & response

- Incidents & alerts
- Incidents
- Alerts

Alerts

Built-in alert tuning was updated with new rules. These rules help streamline your incident and alert queues by prioritizing high-fidelity, actionable alerts. Review or manage the affected alert types in Alert Tuning. Learn more about built-in rules.

Don't show again

Create

Export

1 Week

468 Alerts

Search for name or ID

Customize columns

Filter set:

Status: New, In progress

Add filter

Reset all

Alert name	Tags	Severity	Investigation state	Status	Category	Detection source
Possible Kerberoasting attack		High	New	New	Credential Access	Defender XDR
Suspected Kerberos SPN exposure		High	New	New	Credential Access	Microsoft Defender
Suspicious Kerberos authentication (AS-REQ)		Medium	New	New	Lateral Movement, Credential Access	Defender XDR
Possible Kerberoasting LDAP reconnaissance		High	New	New	Discovery, Credential Access	Defender XDR
Playbook Test-UPN		Low	New	New	Suspicious activity	Scheduled detection

Figure 10.2 — The Alerts queue in Defender, filtered to New/In progress — all four Kerberoasting-related alerts visible, three at High severity.

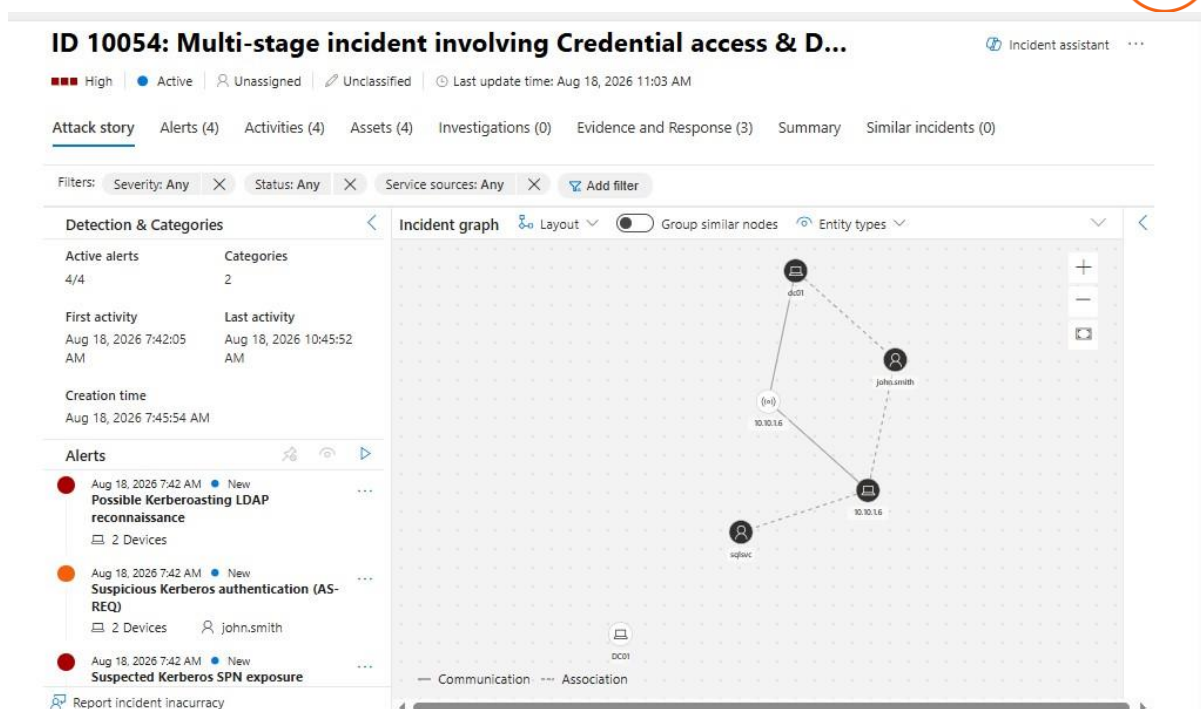


Figure 10.3 — Incident ID 10054, "Multi-stage incident involving Credential access & Discovery on multiple endpoints" (High) — the Attack story graph linking john.smith, sqlsvc, DC01, and KALI01's IP across all four correlated alerts.

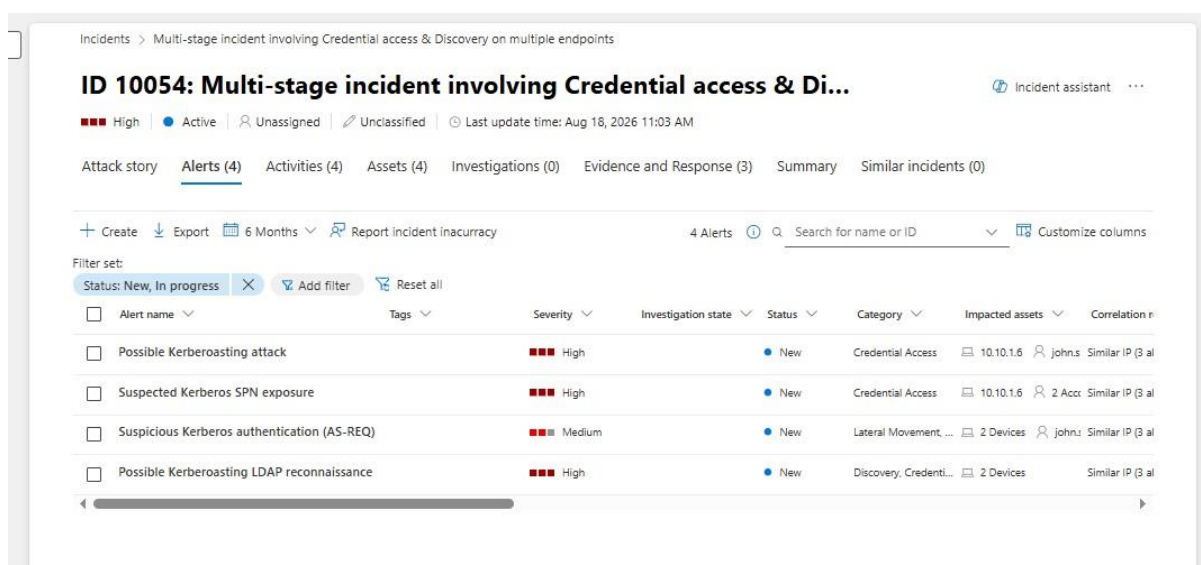


Figure 10.4 — The incident's Alerts tab: all 4 alerts listed with category, impacted assets, and correlation reason (Similar IP).



Incidents > Multi-stage incident involving Credential access & Discovery on multiple endpoints

ID 10054: Multi-stage incident involving Credential access & Di...

Incident assistant ...

High

Active

Unassigned

Unclassified

Last update time: Aug 18, 2026 11:03 AM

Attack story

Alerts (4)

Activities (4)

Assets (4)

Investigations (0)

Evidence and Response (3)

Summary

Similar incidents (0)

+ Create

↓ Export

6 Months

Report incident inaccuracy

4 Alerts

Search for name or ID

Customize columns

Filter set:

Status: New, In progress

Add filter

Reset all

y	Investigation state	Status	Category	Impacted assets	Correlation reas	Detection source	Product name	First activity	Last activity
high		New	Credential Access	10.10.1.6 johns	Similar IP (3 alerts) ...	Defender XDR	Microsoft Defender...	Aug 18, 2026 10:45 AM	Aug 18, 2026 10:45
high		New	Credential Access	10.10.1.6 2 Acc	Similar IP (3 alerts) ...	Microsoft Defender...	Microsoft Defender...	Aug 18, 2026 7:42 AM	Aug 18, 2026 10:45
Medium		New	Lateral Movement ...	2 Devices johns	Similar IP (3 alerts) ...	Defender XDR	Microsoft Defender...	Aug 18, 2026 7:42 AM	Aug 18, 2026 10:45
high		New	Discovery, Credentiali...	2 Devices	Similar IP (3 alerts)	Defender XDR	Microsoft Defender...	Aug 18, 2026 7:42 AM	Aug 18, 2026 10:45

Figure 10.5 — Scrolled view confirming detection source Microsoft Defender XDR / Defender for Identity across all four alerts, with first/last activity timestamps.

Incidents > Multi-stage incident involving Credential access & Discovery on multiple endpoints

ID 10054: Multi-stage incident involving Credential access & Di...

Incident assistant ...

High Active Unassigned Unclassified Last update time: Aug 18, 2026 11:03 AM

Attack story Alerts (4) Activities (4) Assets (4) Investigations (0) Evidence and Response (3) Summary Similar incidents (0)

All assets (4)

- Devices (2)
- Users (2)
- Mailboxes (0)
- Apps (0)
- Cloud Resources (0)
- AI agents (0)

Devices

Export 2 items Customize columns Filter

Device name	Domain	Device id	Risk level
10.10.1.6		10.10.1.6	High
dc01.lab.local	lab.local	e5c549ae9723f44e4986a7e9fed51a67c98fb370	High

Figure 10.6 — The Assets tab: 2 correlated devices — KALI01's IP (10.10.1.6) and dc01.lab.local.

Incidents > Multi-stage incident involving Credential access & Discovery on multiple endpoints

ID 10054: Multi-stage incident involving Credential access & Di...

Incident assistant ...

High Active Unassigned Unclassified Last update time: Aug 18, 2026 11:03 AM

Attack story Alerts (4) Activities (4) Assets (4) Investigations (0) Evidence and Response (3) Summary Similar incidents (0)

All evidence (3)

- Active directory domains (2)
- IP addresses (1)

Export 3 items Customize columns Filter

First seen	Entity	Entity type	Verdict	Remediation status	Impacted assets	D
Aug 18, 2026 7:42 ...	10.10.1.6	IP Address	Suspicious		2 Hosts	
Aug 18, 2026 7:42 ...	lab.local	Active directory domain	Suspicious		2 Hosts	
Aug 18, 2026 7:42 ...	lab.local	Active directory domain	Suspicious		10.10.1.6	

Figure 10.7 — The Evidence and Response tab: KALI01's IP address and the lab.local AD domain, both automatically flagged with a Suspicious verdict.



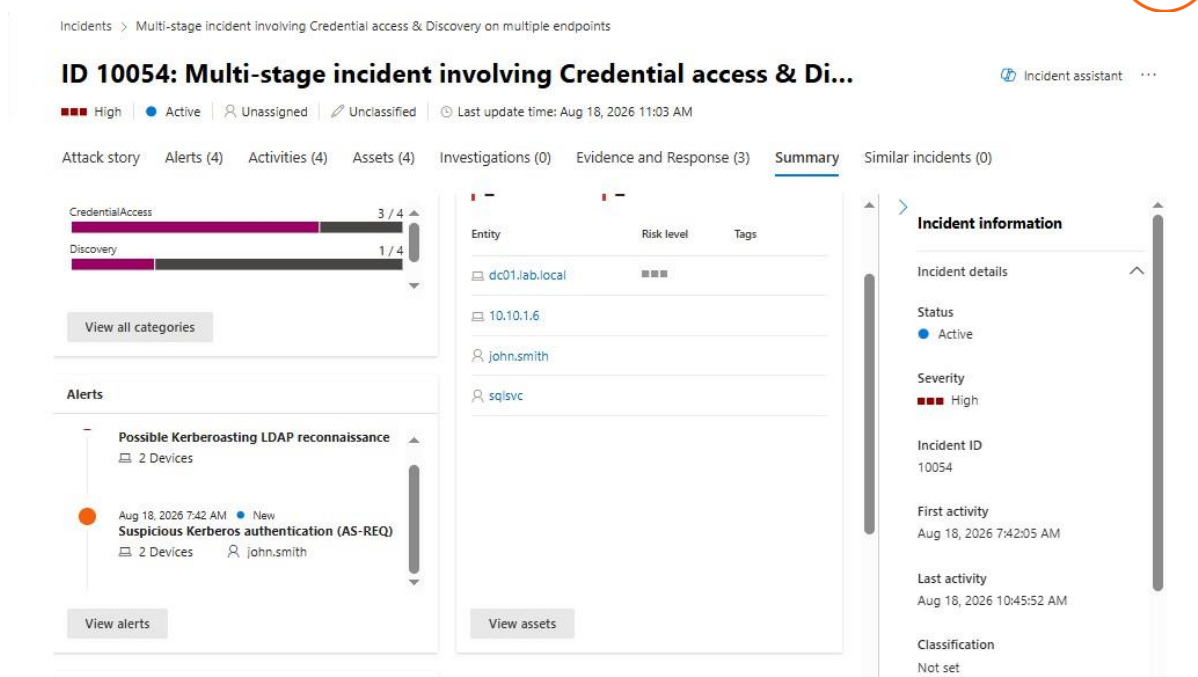


Figure 10.8 — The Summary tab: MITRE ATT&CK category breakdown (Credential Access 3/4, Discovery 1/4) and the full entity list — dc01.lab.local, KALI01's IP, john.smith, sqlsvc.

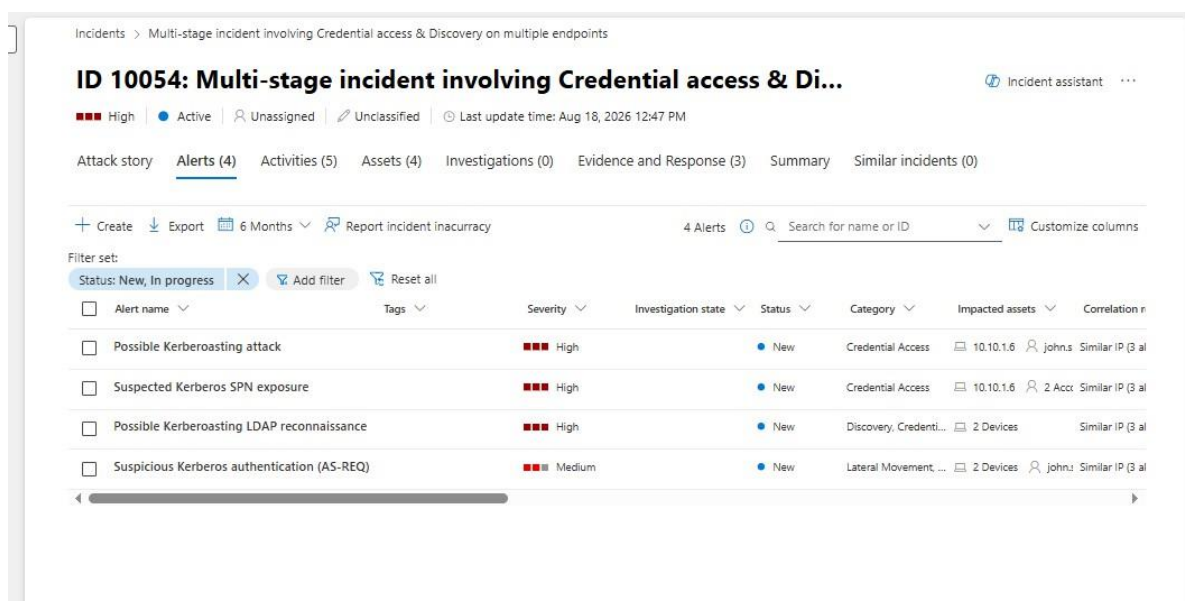


Figure 10.9 — Revisiting the incident later: Activities count has grown to 5 and the alert list is unchanged, confirming continued correlation as the investigation proceeded.



Incidents > Multi-stage incident involving Credential access & Discovery on multiple endpoints

ID 10054: Multi-stage incident involving Credential access & Di...

High Active Unassigned Unclassified Last update time: Aug 18, 2026 12:47 PM

Attack story Alerts (4) Activities (5) Assets (4) Investigations (0) Evidence and Response (3) Summary Similar incidents (0)

Create Export 6 Months Report incident inaccuracy 4 Alerts Search for name or ID Customize columns

Filter set: Status: New, In progress Add filter Reset all

Alerts	Detection source	Product name	First activity	Last activity	Policy name	Classification	Determination	As
Alerts	Defender XDR	Microsoft Defender...	Aug 18, 2026 10:45 AM	Aug 18, 2026 12:42 PM		Not Set	Not Set	Un
Alerts	Microsoft Defender...	Microsoft Defender...	Aug 18, 2026 7:42 AM	Aug 18, 2026 12:42 PM		Not Set	Not Set	Un
Alerts	Defender XDR	Microsoft Defender...	Aug 18, 2026 7:42 AM	Aug 18, 2026 12:42 PM		Not Set	Not Set	Un
Alerts	Defender XDR	Microsoft Defender...	Aug 18, 2026 7:42 AM	Aug 18, 2026 10:45 AM		Not Set	Not Set	Un

Figure 10.10 — Scrolled further right on the Alerts tab — Last activity timestamps updated as the incident stayed active.

Incidents > Multi-stage incident involving Credential access & Discovery on multiple endpoints > Possible Kerberoasting attack

Part of incident: Multi-stage incident involving Credential access & Discovery on multiple endpoints View incident page

10.10.1.6 LAB.LOCAL\john.smith

Manage alert

2919d5804164_aml

MITRE ATT&CK Techniques T1558.003: Kerbe...

Service source Microsoft Defender for Identity

Detection status Unknown

Generated on Aug 18, 2026

Aug 18, 2026 10:49 AM

First activity Aug 18, 2026 10:45:52 AM

Last activity Aug 18, 2026 12:42:26 PM

Workspace

What happened

One or more suspicious Kerberos ticket-granting service requests (TGS-REQ), originating from the IP address 10.10.1.6, have been detected. This activity might indicate a potential Kerberoasting attack...

Read more

Related events

Expand all

8/18/2026 10:45:52 AM Kerberoasting on computer 10.10.1.6

8/18/2026 12:42:26 PM Kerberoasting on computer 10.10.1.6

Figure 10.11 — Drilling into the "Possible Kerberoasting attack" alert: the "What happened" description of the suspicious TGS-REQ requests from 10.10.1.6, MITRE technique T1558.003, and the related Kerberoasting events.



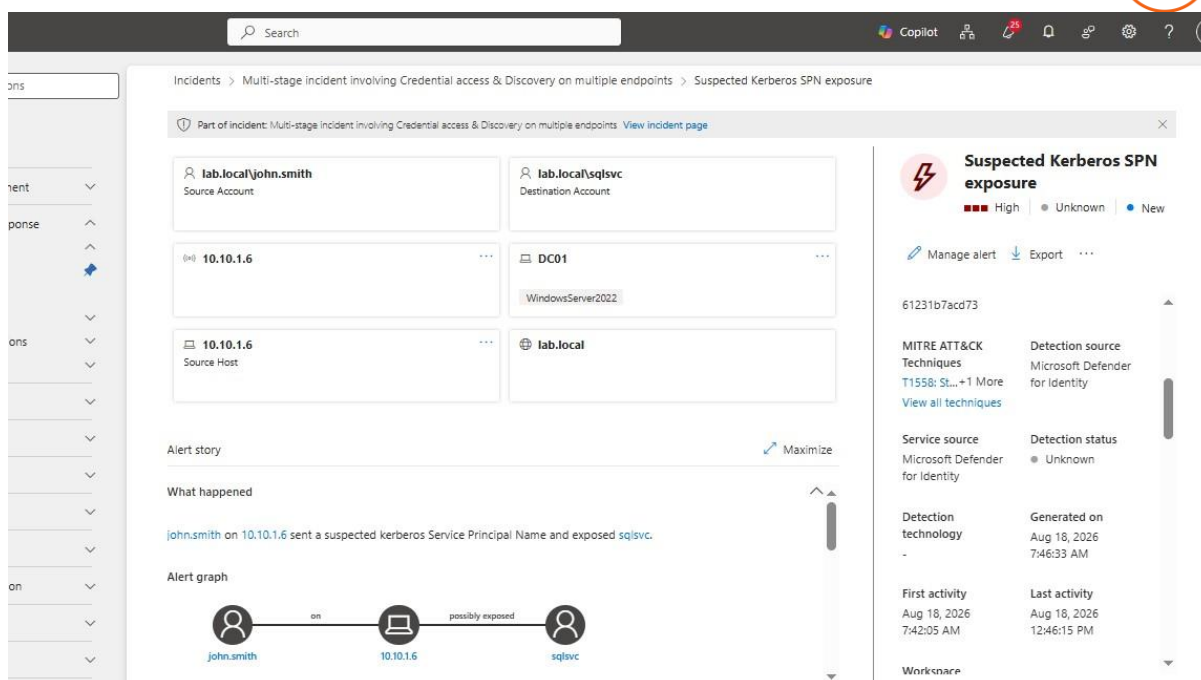


Figure 10.12 — The “Suspected Kerberos SPN exposure” alert, explicitly naming john.smith as the source account that exposed sqlsvc, with a mini alert graph (john.smith → 10.10.1.6 → sqlsvc).

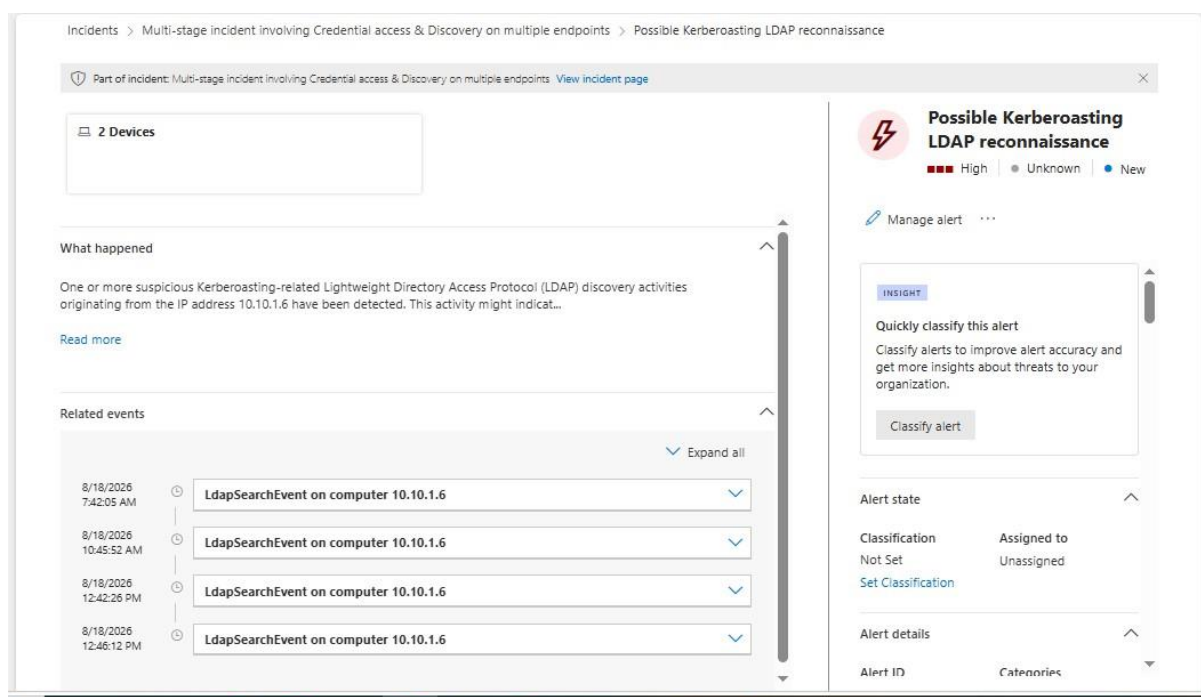


Figure 10.13 — The “Possible Kerberoasting LDAP reconnaissance” alert: four separate LdapSearchEvent entries from 10.10.1.6, spanning the reconnaissance and exploitation phases.



Incidents > Multi-stage incident involving Credential access & Discovery on multiple endpoints > Possible Kerberoasting LDAP reconnaissance

Part of incident: Multi-stage incident involving Credential access & Discovery on multiple endpoints [View incident page](#)

2 Devices

What happened

One or more suspicious Kerberoasting-related Lightweight Directory Access Protocol (LDAP) discovery activities originating from the IP address 10.10.1.6 have been detected. This activity might indicate...

[Read more](#)

Related events

Expand all

Time	Event
8/18/2026 7:42:05 AM	LdapSearchEvent on computer 10.10.1.6
8/18/2026 10:45:52 AM	LdapSearchEvent on computer 10.10.1.6
8/18/2026 12:42:26 PM	LdapSearchEvent on computer 10.10.1.6
8/18/2026 12:46:12 PM	LdapSearchEvent on computer 10.10.1.6

Possible Kerberoasting LDAP reconnaissance

High Unknown New

Manage alert

ri397e8433-be69-4a86-b10a-56d4a256e5c2_aml

Discovery, Credential Access

MITRE ATT&CK Techniques

T1087.00... +1 More

[View all techniques](#)

Detection source

Defender XDR

Service source

Microsoft Defender for Identity

Detection status

Unknown

Detection technology

-

Generated on

Aug 18, 2026 7:45:54 AM

First activity

Aug 18, 2026 7:42:05 AM

Last activity

Aug 18, 2026 12:46:12 PM

Figure 10.14 — The same alert, scrolled to show its categorization: Discovery + Credential Access, MITRE technique T1087 (Account Discovery), detected by Defender for Identity.

Incidents > Multi-stage incident involving Credential access & Discovery on multiple endpoints

ID 10054: Multi-stage incident involving Credential access & Discovery on multiple endpoints

High Active Unassigned Unclassified Last update time: Aug 18, 2026 12:51 PM

Attack story Alerts (4) **Activities (5)** Assets (4) Investigations (0) Evidence and Response (0)

Refresh Export Add comment

Filter set:

Origin: This incident Add filter Reset all

Activity started	Title	Type
☒	Aug 18, 2026 12:47 PM Account disabled via automated playbo...	Comment
☐	Aug 18, 2026 10:49 AM Alert 'Possible Kerberoasting attack' was ...	Alert link
☐	Aug 18, 2026 7:59 AM Alert 'Suspicious Kerberos authentication...	Alert link
☐	Aug 18, 2026 7:46 AM Alert 'Suspected Kerberos SPN exposure'...	Alert link
☐	Aug 18, 2026 7:45 AM Alert 'Possible Kerberoasting LDAP recon...	Alert link

Activity Details

Activity started

Aug 18, 2026 12:47 PM

Title

Comment

Performed by

Playbook-Disable-Account-ADLab

Activity status

Type

Comment

Category

Comment

Trigger

Manual

Comments

Account disabled via automated playbook: john.smith@...

Figure 10.15 — The incident's Activities tab, showing a real automated response already logged: "Account disabled via automated playbook: john.smith@..." performed by Disable-Account-ADLab.



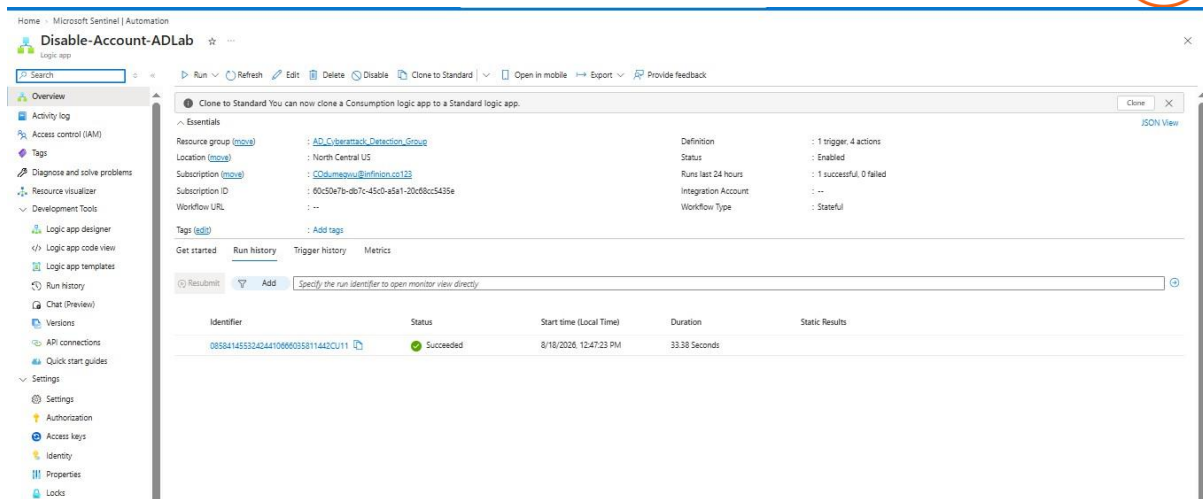


Figure 10.16 — Disable-Account-ADLab's Run history: a genuine successful run (1 successful, 0 failed) at 12:47:23 PM, taking 33.38 seconds, triggered by this incident.

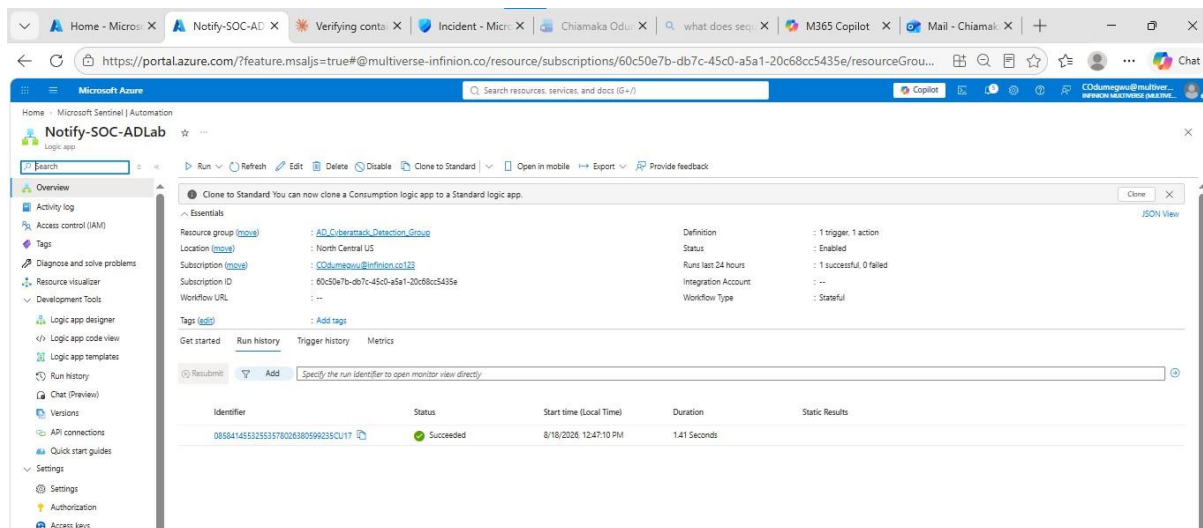


Figure 10.17 — Notify-SOC-ADLab's Run history: also a real successful run (1.41 seconds) at 12:47:10 PM, matching the incident timeline.



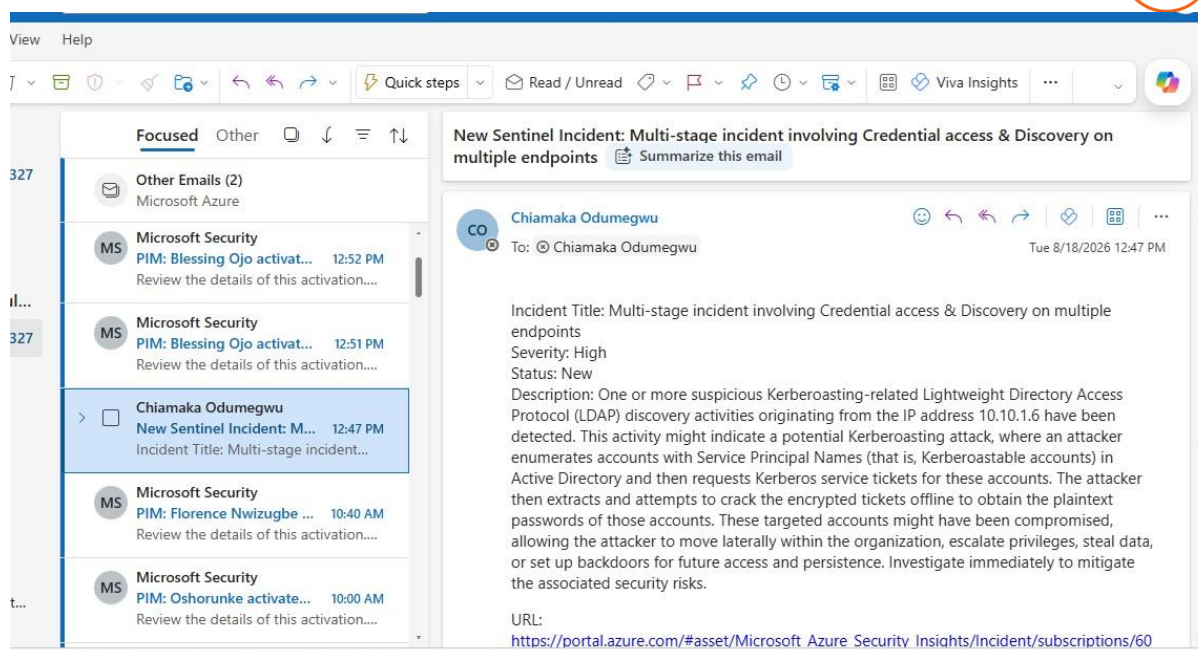


Figure 10.18 — The actual SOC notification email received in Outlook, dynamically populated with Incident Title, Severity (High), Status, Description, and a direct link to the incident.

Disable-Account-ADLab

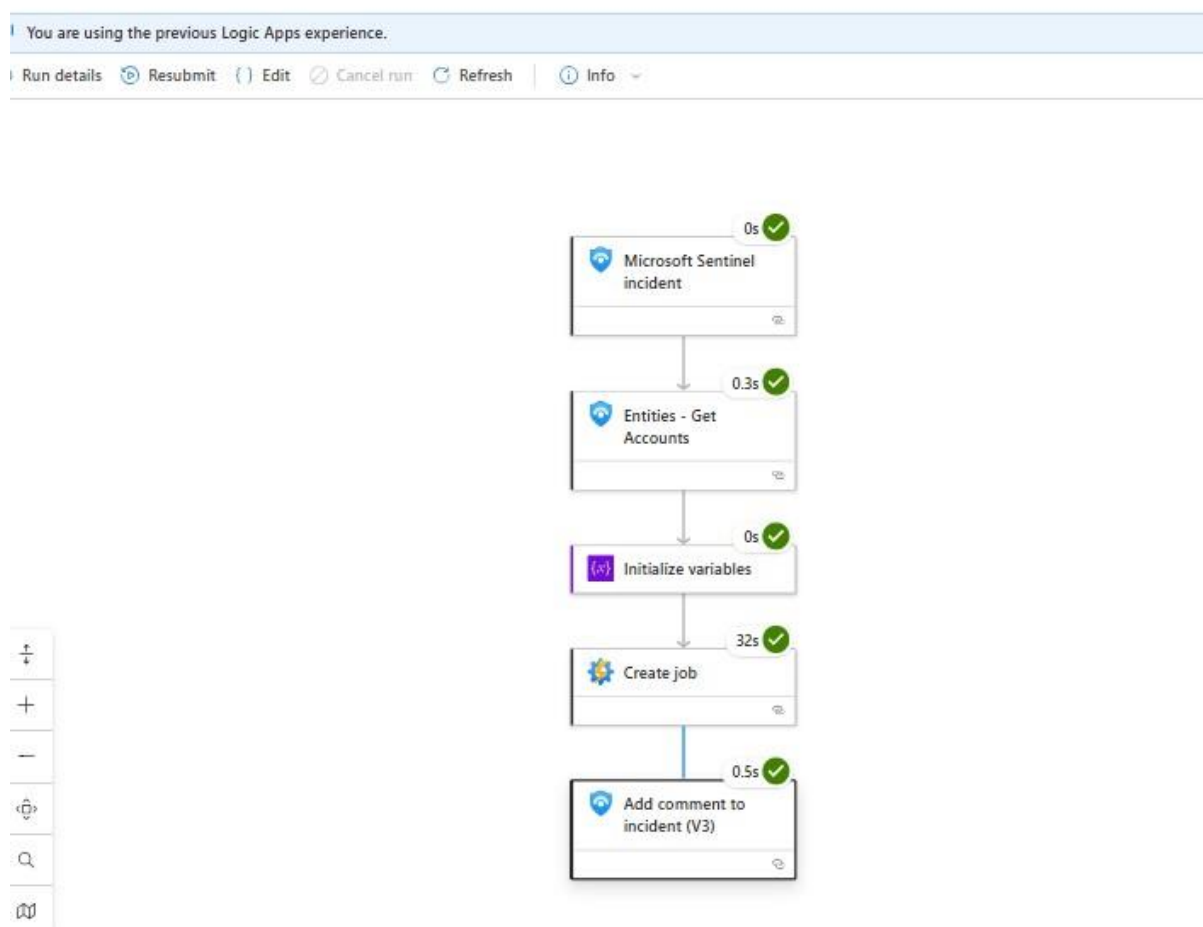


Figure 10.19 — The full run trace for Disable-Account-ADLab: every step — Sentinel incident trigger, Entities - Get Accounts, Initialize variables, Create job, Add comment to incident — completed successfully.

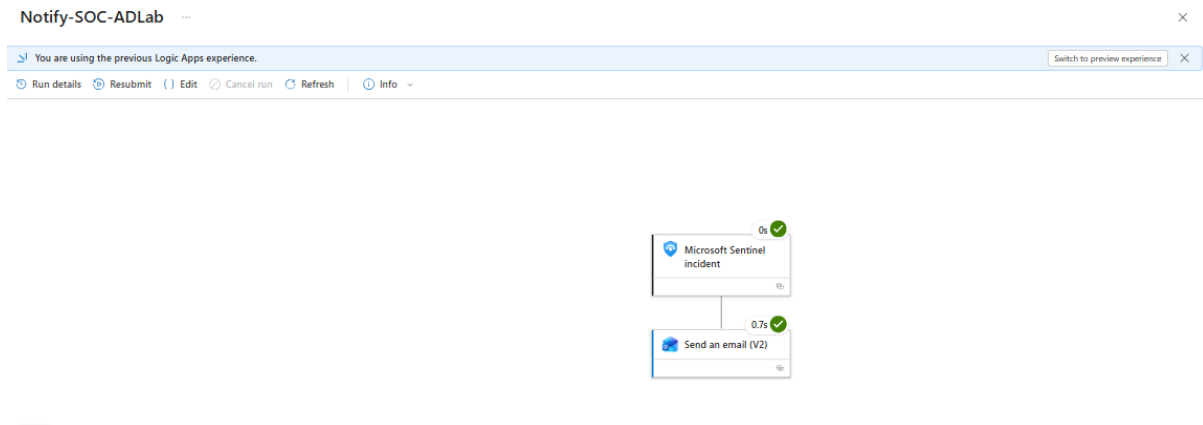


Figure 10.20 — The equivalent run trace for Notify-SOC-ADLab: Sentinel incident trigger → Send an email (V2), both succeeded.

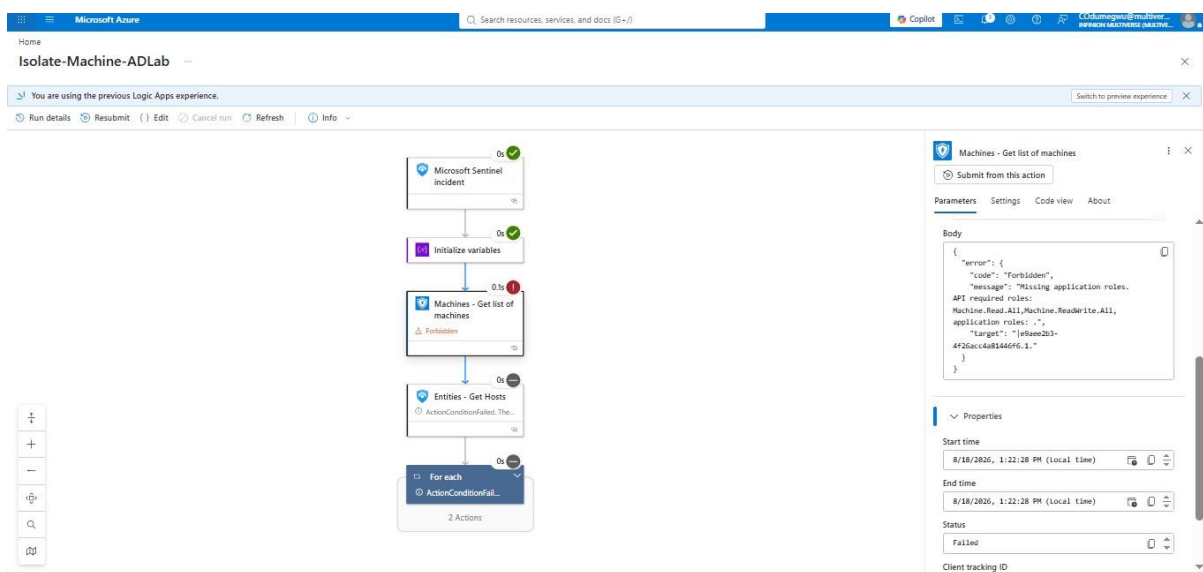


Figure 10.21 — The device isolation failure: Isolate-Machine-ADLab's run stopped at "Machines - Get list of machines" with a 403 Forbidden error — the managed identity is missing the required MDE application roles (Machine.Read.All, Machine.ReadWrite.All).

```

PS /home/chiamaka> $managedIdentityId = 'b52e8bf8-80a6-4424-9a1a-fe0a48dac70a'
PS /home/chiamaka> $myPermissions = @('Machine.Isolate','Machine.Read.All')
PS /home/chiamaka> $msi = Get-MgServicePrincipal -Filter "Id eq '$managedIdentityId'"
PS /home/chiamaka> $msiId = Get-MgServicePrincipal -Filter "AppId eq 'fc788465-2017-40d4-a8c5-307022471b92'" # WindowsDefenderATP
PS /home/chiamaka> foreach ($perm in $myPermissions) {
    >> $role = $msi.AppRoles | where-object Value -eq $perm | select-object -first 1
    >> if ($role) {
    >>     New-MgServicePrincipalAppRoleAssignment `
    >>         -ServicePrincipalId $msi.Id `
    >>         -AppRoleId $role.Id `
    >>     }
}

```

Figure 10.22 — Diagnosing and resolving the gap via Azure Cloud Shell: a PowerShell script assigning the required app roles to the Logic App's managed identity against the WindowsDefenderATP service principal.

AppRoleId	CreatedDateTime	PrincipalDisplayName	PrincipalId	PrincipalType	ResourceDisplayName	ResourceId
Isolate-Machine-ADLab	8/18/2026 12:19:51 PM	Isolate-Machine-ADLab	b52e8bf8-80a6-4424-9a1a-fe0a48dac70a	ServicePrincipal	WindowsDefenderATP	933483c8-4c08-4c05-beF9-0763827758a1
Isolate-Machine-ADLab	8/18/2026 12:19:51 PM	Isolate-Machine-ADLab	b52e8bf8-80a6-4424-9a1a-fe0a48dac70a	ServicePrincipal	WindowsDefenderATP	933483c8-4c08-4c05-beF9-0763827758a1



7th floor Ibukun House, Ademola Adetokunbo street, Victoria Island, Lagos, Nigeria



info@infinion.co



infinion.co

Figure 10.23 — Confirmation output: Get-MgServicePrincipalAppRoleAssignment shows both roles now assigned to Isolate-Machine-ADLab's managed identity.

```
PS /home/chianaka> Get-MgServicePrincipalAppRoleAssignment -ServicePrincipalId $msi.Id | Format-List

AppRoleId       : 7e4e1308-e1b9-4102-88ba-f0cb6e6d5974
CreatedDateTime  : 8/18/2026 12:19:51 PM
DeletedDateTime  :
Id              : -IsutaYA3ESaGv4KSNrHOMTptEE4G9CigkvWg8u7A
PrincipalDisplayName : Isolate-Machine-ADLab
PrincipalId      : b52e8bf8-00a6-4424-9a1a-fe0a48daC70a
PrincipalType    : ServicePrincipal
ResourceDisplayName : WindowsDefenderATP
ResourceId       : 933483c8-6c80-4ca5-8ef9-8761827758a1
AdditionalProperties : {}
```

Figure 10.24 — Detailed role-assignment confirmation: AppRoleId, PrincipalDisplayName (Isolate-Machine-ADLab), and ResourceDisplayName (WindowsDefenderATP) — the permissions fix fully verified.

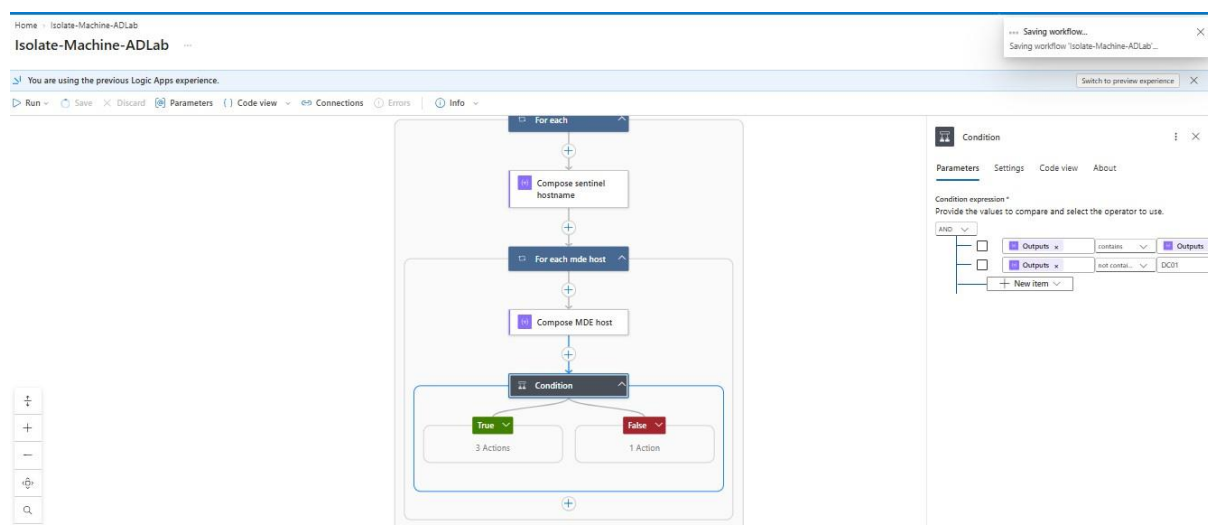


Figure 10.25 — The resulting design correction: a new Condition added to Isolate-Machine-ADLab checking that the target hostname does not contain "DC01", permanently excluding the domain controller from automated isolation.

Containment

Following confirmation of the compromise scope in Phase 10, containment actions were executed to stop the threat from spreading further while preserving the environment for analysis.

Account containment

The Disable-Account-ADLab playbook, triggered automatically by the Contain-Eradicate-Rule automation rule on High-severity incidents, disabled the compromised john.smith account. This action was confirmed both through the incident's activity log and by direct verification of the account's state in Active Directory.

Ticket invalidation

Disabling an account does not automatically invalidate Kerberos tickets already issued while it was active. To fully contain the identity compromise, john.smith's password was reset twice in succession, invalidating any tickets tied to the account's prior key material, closing the gap between "account disabled" and "attacker access fully revoked."

Device isolation — reassessed



7th floor Ibukun House, Ademola Adetokunbo street, Victoria Island, Lagos, Nigeria



info@infinion.co



infinion.co

The Isolate-Machine-ADLab playbook was evaluated as part of containment, but analysis of the incident's entity graph determined that no device in this attack path was actually compromised. Device isolation was correctly identified as unnecessary for this specific incident, and a permanent safety exclusion was added to the playbook to prevent it from acting against the domain controller in this or future incidents where it appears as an observing entity rather than a compromised asset.

Scope verification

A full account audit (Get-ADUser -Filter * across Enabled, LastLogonDate, PasswordLastSet) confirmed the compromise was correctly bounded to john.smith and sqlsvc only. No other domain account showed activity within the attack window.

Outcome: Containment is confirmed complete: the compromised identity was disabled, its tickets invalidated, and the compromise scope verified as limited to the two known accounts.

```

DC01 (1) - 52.224.241.5:3389 - Remote Desktop Connection
Server Manager
Server Manager > Dashboard
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
PS C:\Users\labamin> $temp1 = ConvertTo-SecureString "TempReset1!Xy9" -AsPlainText -Force
PS C:\Users\labamin> Set-ADAccountPassword -Identity john.smith -NewPassword $temp1 -Reset
PS C:\Users\labamin> $temp2 = ConvertTo-SecureString "TempReset2!Zw4" -AsPlainText -Force
PS C:\Users\labamin> Set-ADAccountPassword -Identity john.smith -NewPassword $temp2 -Reset
PS C:\Users\labamin> Get-ADUser -Identity john.smith -Properties PasswordLastSet | Select-Object Name, PasswordLastSet

Name      PasswordLastSet
-----
john.smith 8/19/2026 3:21:36 AM

PS C:\Users\labamin> Get-ADUser -Filter * -Properties LastLogonDate, PasswordLastSet, Enabled |
>>   Select-Object Name, SamAccountName, Enabled, LastLogonDate, PasswordLastSet |
>>   Sort-Object LastLogonDate -Descending

Name           : john.smith
SamAccountName  : john.smith
Enabled         : True
LastLogonDate   : 8/17/2026 6:43:46 PM
PasswordLastSet : 8/19/2026 3:21:36 AM
  
```

Figure 11.1 — Invalidating john.smith's Kerberos tickets: Set-ADAccountPassword run twice in succession with different temporary passwords, followed by the start of the domain-wide account audit (Get-ADUser -Filter *).



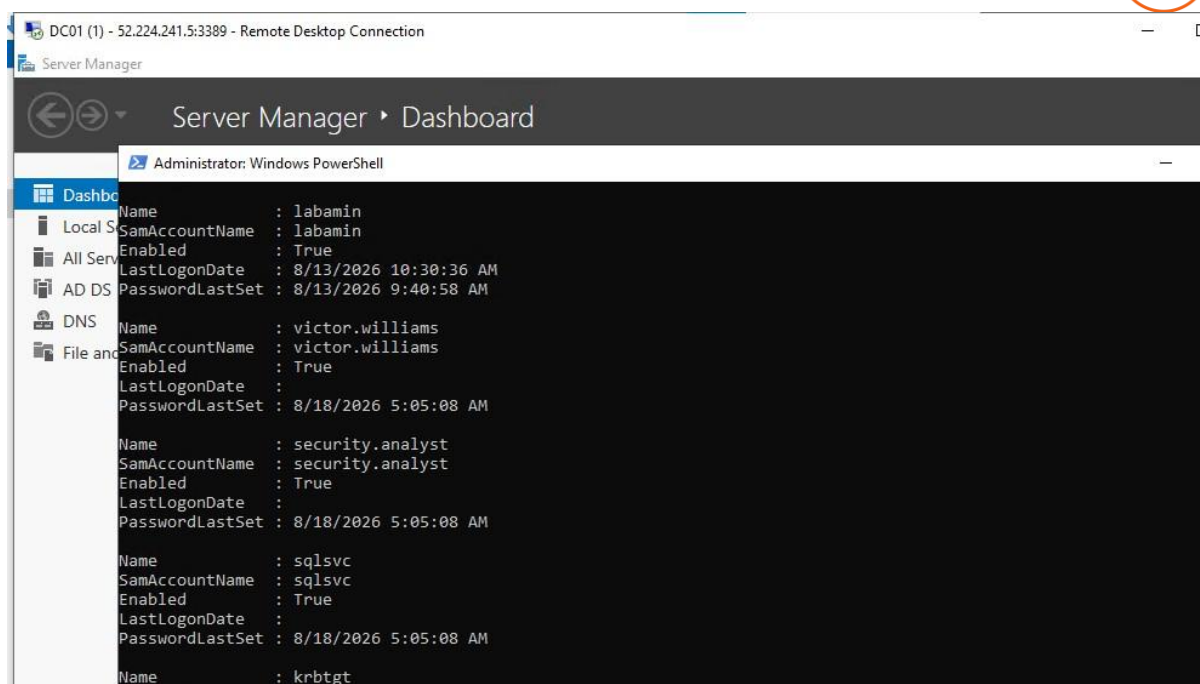


Figure 11.2 — Account audit results, sorted by LastLogonDate descending — victor.williams, security.analyst, and sqlsvc all show no LastLogonDate during the attack window, confirming they were untouched.

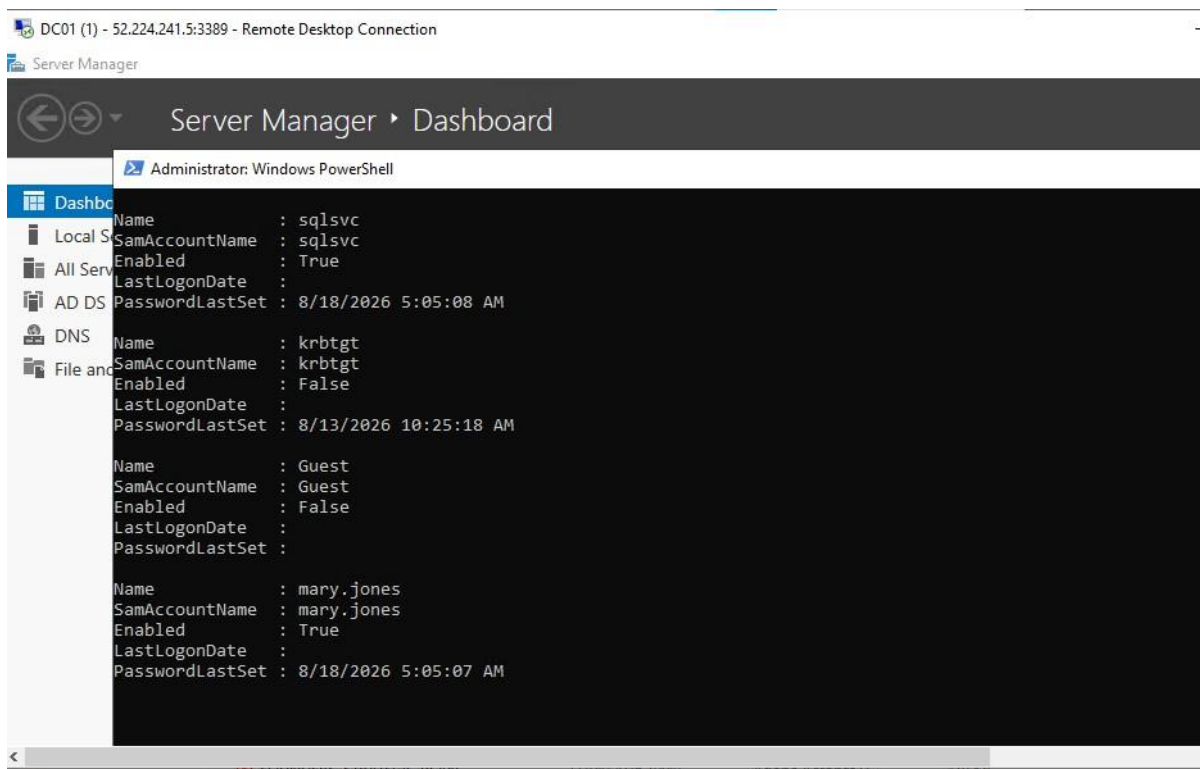


Figure 11.3 — Continued audit output — sqlsvc, krbtgt, Guest, and mary.jones all confirmed with no unexpected activity, closing out the scope-verification sweep.

Eradication

With containment verified, eradication addressed the root cause that made the incident possible in the first place.



7th floor Ibukun House, Ademola Adetokunbo street, Victoria Island, Lagos, Nigeria



info@infinion.co



infinion.co

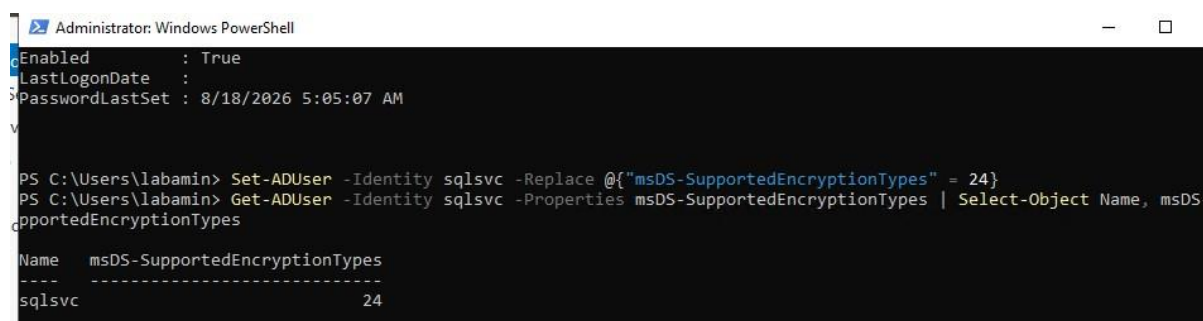
Root cause

The sqlsvc account had msDS-SupportedEncryptionTypes configured to support RC4 encryption (deliberately set to 28 during Phase 9 to demonstrate the Kerberoasting attack against Server 2022's default AES-only Kerberos configuration). RC4-encrypted Kerberos tickets are crackable offline, which is what made the SPN exploitable in the first place.

Remediation

sqlsvc's msDS-SupportedEncryptionTypes was reset to 24 (AES128 + AES256 only, RC4 removed), permanently closing the encryption weakness. This is a one-way fix in this project — the account will not be reverted to a weaker state going forward.

Outcome: The specific vulnerability that enabled this incident — an SPN-bearing account weak enough to permit RC4 ticket cracking — no longer exists in the environment. A future Kerberoasting attempt against sqlsvc would still be able to enumerate the SPN (SPN exposure itself isn't eliminated, since that's inherent to any service account with an SPN), but would no longer be able to obtain a crackable ticket.



```

Administrator: Windows PowerShell

Enabled : True
LastLogonDate :
PasswordLastSet : 8/18/2026 5:05:07 AM

PS C:\Users\labamin> Set-ADUser -Identity sqlsvc -Replace @{msDS-SupportedEncryptionTypes = 24}
PS C:\Users\labamin> Get-ADUser -Identity sqlsvc -Properties msDS-SupportedEncryptionTypes | Select-Object Name, msDS-SupportedEncryptionTypes

Name      msDS-SupportedEncryptionTypes
----      -
sqlsvc    24
  
```

Figure 12.1 — Eradicating the root cause: resetting sqlsvc's msDS-SupportedEncryptionTypes from 28 (RC4+AES) to 24 (AES128+AES256 only), confirmed by the follow-up Get-ADUser query.

Recovery

Recovery restored the environment to normal operational state, confirming both functional restoration and a clean bill of health before closing the incident.

Account restoration

john.smith was re-enabled and assigned a new strong password, replacing both the disabled state and the deliberately weak password (Summer2026!) that enabled the original compromise.

Health verification

A final check confirmed the account carries no lingering LockedOut or PasswordExpired flags that could cause unexpected access issues post-recovery.

Outcome: john.smith is fully restored to normal, secure operating condition — enabled, strong credentials, no residual account-state issues. Combined with the eradication of sqlsvc's encryption weakness, the environment is confirmed both functionally restored and no longer vulnerable to a repeat of this specific attack path.



```

PS C:\Users\labamin> Set-ADUser -Identity john.smith -Enabled $true
PS C:\Users\labamin> Get-ADUser -Identity john.smith -Properties Enabled | Select-Object Name, Enabled

Name      Enabled
-----
john.smith True

PS C:\Users\labamin> $strongPw = ConvertTo-SecureString "P@ssw0rd123!" -AsPlainText -Force
PS C:\Users\labamin> Set-ADAccountPassword -Identity john.smith -NewPassword $strongPw -Reset
PS C:\Users\labamin>

```

Figure 13.1 — Re-enabling john.smith and setting a new strong password via Set-ADAccountPassword, replacing the deliberately weak credential used in Phase 9.

```

DNS john.smith True
File and

PS C:\Users\labamin> $strongPw = ConvertTo-SecureString "P@ssw0rd123!" -AsPlainText -Force
PS C:\Users\labamin> Set-ADAccountPassword -Identity john.smith -NewPassword $strongPw -Reset
PS C:\Users\labamin> Get-ADUser -Identity john.smith -Properties LockedOut, PasswordExpired | Select-Object Name, Enabled, LockedOut, PasswordExpired

Name      Enabled LockedOut PasswordExpired
-----
john.smith True     False      False

PS C:\Users\labamin>

```

Figure 13.2 — Final health check: Get-ADUser confirms john.smith is Enabled, with LockedOut and PasswordExpired both False — recovery closed out cleanly.

Lessons Learned

1. Detection & Environment Tuning

MDI's default 30-day learning period can suppress real alerts on a fresh sensor. The initial Kerberoasting attack didn't trigger an alert, not because detection logic failed, but because a newly deployed sensor defaults to a learning window where it builds a behavioral baseline before alerting. This had to be manually disabled (Settings → Identities → Advanced settings → "Remove learning period") before genuine detections would fire. In a real deployment, this is a timing trap worth knowing in advance — a "quiet" first few weeks after sensor deployment isn't necessarily a healthy signal.

Server 2022 defaults away from RC4 Kerberos encryption, which meant the classic Kerberoasting demonstration didn't work out of the box — sqlsvc had to have msDS-SupportedEncryptionTypes deliberately weakened to 28 to make the account roastable at all. This is itself informative: modern AD environments are more resistant to Kerberoasting by default than commonly assumed, and an attacker succeeding at it in a real environment likely indicates a legacy or misconfigured account, not a fundamental Kerberos weakness.

2. Identity vs. Device Compromise — Containment Design

The Kerberoasting incident's entity graph included two device entities (KALI01, DC01) alongside the compromised identity. Neither device was actually compromised — one was the external attacker's own machine, the other simply received and logged malicious requests. The real compromise existed entirely at the identity layer.

Implication: automated containment playbooks shouldn't assume every incident warrants every available action. A device appearing in an incident's entity list doesn't mean that device needs isolating — SOAR logic needs to distinguish *why* an entity is present (source of attack, recipient of attack, or actual compromised asset) rather than reacting to presence alone.



3. Infrastructure Gaps That Produce Silent Failures

Managed identity permissions for the Microsoft Defender for Endpoint API can't be granted through the Azure portal's standard API Permissions UI. Unlike most Azure RBAC scenarios, these specific app roles (Machine.Isolate, Machine.Read.All) require direct assignment via Microsoft Graph PowerShell against the WindowsDefenderATP service principal. This is an easy gap to miss since the portal gives no obvious indication that this permission model works differently — a playbook can look fully configured and still fail silently at runtime.

A Logic App run reporting "Succeeded" doesn't guarantee its logic actually executed meaningfully. One run showed a clean success status while every step inside its loop was actually skipped, because the loop had nothing to iterate over (no incident data was present). This distinction — between "ran without error" and "actually did the thing it's meant to do" — only surfaced by manually tracing into individual action outputs rather than trusting the top-level status.

Sentinel's connection to the Defender XDR unified portal affects automation timing in ways that aren't always obvious. Early in testing, automation rules appeared not to fire at all; this was ultimately traced to a combination of workspace connectivity timing and the permission/logic gaps above, rather than a single root cause — a reminder that "automation didn't fire" can have layered causes that need to be isolated one at a time rather than assumed to be a single issue.

4. Non-Entra-Synced AD Requires Different Automation Architecture

Because lab.local is a standalone AD DS environment (not synced to Entra ID), standard Sentinel playbook patterns that call Microsoft Graph directly to manage user accounts don't apply. This required building out a Hybrid Runbook Worker on DC01 and an Azure Automation runbook to execute AD changes locally, with the Sentinel playbook acting as an orchestration trigger rather than a direct actor. This is a meaningfully different — and more involved — architecture than cloud-native identity response, and worth calling out explicitly since many published SOAR examples assume Entra-synced environments.

Troubleshooting Guidance

This section documents real issues encountered during the build and their resolutions, intended as a reference for anyone reproducing or extending this lab.

1. Kerberoasting attempt returns KDC_ERR_ETYPE_NOSUPP *Symptom*: impacket-GetUserSPNs fails against a target SPN account rather than returning a crackable hash. *Cause*: Windows Server 2022 defaults away from RC4 Kerberos encryption, which is required for the classic offline-crackable ticket format. *Fix*: Temporarily set the target account's msDS-SupportedEncryptionTypes to 28 (RC4 + AES128 + AES256) via Set-ADUser -Replace. Revert to 24 (AES-only) once testing is complete, see Rollback/Recovery below.

2. Automation rule appears configured but playbook never runs *Symptom*: A Sentinel automation rule exists, is enabled, and is correctly wired to a playbook, but the Logic App shows zero runs even across multiple qualifying incidents. *Cause*: In this project, this was ultimately linked to timing between Sentinel's connection to the Defender XDR unified portal and incident-creation events; incidents sourced from Defender products (like MDI) need that connector fully established before Sentinel's own automation rules can evaluate against them. *Fix*: Confirm the Defender XDR connector shows "Connected" status in Sentinel workspace settings before assuming a rule or playbook is broken. If recently connected, allow time and retest with a fresh incident rather than debugging playbook logic first.

3. Logic App run shows "Succeeded" but nothing actually happened *Symptom*: A playbook run completes with a green success status, but no downstream action (isolation, disablement, notification) is observed. *Cause*: A "Succeeded" status only means no step threw an error, it does not confirm the workflow's loops actually had data to process. In this project, one such



run resulted from manually triggering the playbook without a real incident payload, so the "For each host" loop had nothing to iterate over and every downstream step was silently skipped. *Fix:* Never trust the top-level run status alone. Click into the run and inspect individual action outputs, specifically confirm entity-retrieval steps (e.g., "Entities - Get Hosts") returned non-empty results before concluding a test was meaningful.

4. Managed identity call to Defender for Endpoint API returns 403 Forbidden ("Missing application roles") *Symptom:* A Logic App with a correctly configured system-assigned managed identity fails at a Defender for Endpoint API call (e.g., "Machines - Get list of machines") with a 403 error. *Cause:* Defender for Endpoint API application roles (Machine.Isolate, Machine.Read.All, etc.) cannot be granted to a managed identity through the Azure portal's standard API Permissions blade, this permission model requires direct assignment via Microsoft Graph PowerShell against the WindowsDefenderATP service principal. *Fix:* Use `New-MgServicePrincipalAppRoleAssignment` to grant the required roles from the WindowsDefenderATP service principal directly to the managed identity's Object ID. *Caution:* confirm you are using the Object ID of the correct Logic App's identity — during this project, the role was initially and mistakenly granted to the wrong playbook's managed identity, which produced no error but also no fix, and cost time to diagnose.

5. Role assignment applied correctly, but the API call still fails immediately after *Symptom:* `Get-MgServicePrincipalAppRoleAssignment` confirms the correct role is assigned, but re-running the playbook (via `Resubmit`) still returns 403. *Cause:* Azure AD app role propagation is not instantaneous, it can take several minutes to fully apply. Additionally, using "Resubmit" on a previously failed run may reuse a cached authentication token issued before the role was granted. *Fix:* Wait 10–15 minutes after granting the role, then trigger a genuinely fresh run (not a `Resubmit`) to force new token acquisition.

6. SSH connection to KALI01 fails with "Identity file not accessible" *Symptom:* SSH command using a Windows-style file path (e.g., `C:\Users\...\KALI01-key.pem`) fails when run from a Linux/Kali terminal. *Cause:* Mixing a Windows filesystem path with a Linux-based SSH client, the path doesn't resolve inside the Linux environment. *Fix:* Reference the key using its actual path within the Linux filesystem (e.g., `~/Downloads/KALI01-key.pem`), not the Windows path where it may have originally been downloaded.

Known Limitations and Risks

1. Isolation playbook lacked a critical infrastructure safeguard by default. The Isolate-Machine-ADLab playbook's Condition logic, as originally built, contained no exclusion preventing it from targeting the domain controller (DC01). Had the managed identity permissions issue not coincidentally blocked execution, an early automated run could have isolated DC01 for real — cutting off RDP/JIT administrative access to the domain controller itself. This was remediated by adding an explicit hostname exclusion, but it represents a real risk class: automation with sufficient privilege and no target-awareness can cause self-inflicted outages.

2. `LastLogonDate` is an unreliable indicator for network-based authentication. This AD attribute is lazily replicated and does not consistently capture non-interactive authentication types such as the SMB/NTLM-based password spray used in this project. Relying on it alone during a scope-verification check could produce a false sense of "no activity" for an account that was, in fact, targeted. Incident scope assessments should prioritize alert and event-log evidence over this attribute.

3. The lab's standalone AD architecture (lab.local, non-Entra-synced) required custom automation that would not directly transfer to a hybrid or cloud-native environment. The Disable-Account-ADLab playbook relies on an Azure Automation Hybrid Runbook Worker registered on DC01, since standard Microsoft Graph-based account management (used in cloud-synced environments) is not applicable here. This is architecturally sound for this



scenario but represents additional operational overhead (a runbook worker to maintain, patch, and monitor) that would not exist in an Entra-synced deployment.

4. Defender for Endpoint API permission management for managed identities is not discoverable through the Azure portal UI. This is a genuine platform limitation, not a lab-specific issue: nothing in the portal indicates that Defender for Endpoint app roles must be assigned outside the normal API Permissions blade. Any automation calling this API is at risk of an identical silent failure unless the operator specifically knows to check for it.

5. This lab currently demonstrates only credential/identity-based compromise, not device-based compromise. Every attack executed (Password Spraying, Kerberoasting) targeted the identity layer exclusively; no domain-joined endpoint was ever actually compromised. As a result, the Isolate-Machine-ADLab playbook's core containment action (isolating a genuinely compromised device) has been fixed and safety-tested, but not yet validated against a real positive case. This is a coverage gap in the testing performed, not a flaw in the automation itself.

6. Deliberately weakened security settings (weak passwords, RC4 Kerberos encryption) were introduced for demonstration purposes and must not persist post-lab. Any environment built from this lab's configuration as a starting point should not retain these deliberately weakened settings — see Rollback/Recovery below for the specific reversal steps.

Rollback or Recovery Steps

These steps document how to return the lab environment to a fully secure baseline state, reversing any condition deliberately introduced for attack simulation purposes.

1. Restore sqlsvc to hardened Kerberos encryption (if not already applied):

```
Set-ADUser -Identity sqlsvc -Replace @{"msDS-SupportedEncryptionTypes" = 24}
```

This removes RC4 support, leaving only AES128/AES256 — closing the Kerberoasting weakness permanently.

2. Confirm john.smith is enabled with a strong, non-guessable password:

```
Set-ADUser -Identity john.smith -Enabled $true
```

```
$strongPw = ConvertTo-SecureString "<strong-password-here>" -AsPlainText -Force
```

```
Set-ADAccountPassword -Identity john.smith -NewPassword $strongPw -Reset
```

Do not leave the original demonstration password (Summer2026!) in place beyond the testing window.

3. Verify no residual account-state issues:

```
Get-ADUser -Identity john.smith -Properties Enabled, LockedOut, PasswordExpired |
```

```
Select-Object Name, Enabled, LockedOut, PasswordExpired
```

Confirm Enabled = True, LockedOut = False, PasswordExpired = False.

4. Confirm no device isolation is active on any lab machine: security.microsoft.com → Assets → Devices → confirm DC01 and CLIENT01 show no "Isolated" status. If an isolation action is found active in error, use Actions → Release from isolation immediately.

5. Rotate the KALI01 SSH key if it was ever exposed outside the intended environment, since it provides direct access to the attack machine.

6. Review and, if desired, disable the Isolate-Machine-ADLab, Disable-Account-ADLab, and Notify-SOC-ADLab playbooks if the lab is being decommissioned or left idle for an extended period, to prevent unintended future triggering against a live environment.



Recommendations for Future Improvement

1. Extend attack coverage to include genuine device-based compromise. Executing Pass-the-Hash, DCSync, or Lateral Movement (as originally planned but deferred) would allow the isolation playbook's core containment action to be validated against a real positive case, rather than only a correctly-negative one. This would close the project's most significant remaining coverage gap.
2. Add compromise-type-aware routing to the automation rule layer, rather than relying solely on a static exclusion condition inside one playbook. A more scalable design would have the automation rule itself inspect entity types/roles before invoking device-oriented playbooks at all, reducing reliance on every individual playbook independently implementing its own safety logic.
3. Build a small custom KQL threat-hunting library covering the attack patterns demonstrated here (anomalous Kerberos TGS-REQ volume, LDAP reconnaissance patterns, password spray failure/success ratios), so detection isn't solely dependent on MDI's built-in analytics — this would also serve as a reusable asset beyond this specific lab.
4. Formalize a managed-identity least-privilege review as a standing step, given that this project encountered both an under-permissioned identity (the original 403 error) and an over-permissioned one (the accidental grant to the wrong Logic App's identity). A documented review checklist would catch both failure modes earlier.
5. Add MITRE ATT&CK technique mapping as a first-class documentation artifact rather than an optional extension, since Defender already surfaces this data on each alert — capturing it systematically would strengthen the project's alignment with industry-standard threat modeling frameworks for a portfolio audience.
6. Consider migrating the standalone lab.local AD environment to a hybrid Entra-synced configuration in a future iteration, to demonstrate and compare both the cloud-native and on-premises-only automation patterns side by side — this would broaden the project's applicability to the wider range of real-world environments Microsoft security consultants typically encounter.

